ARMENU

AR / 3D RESTORAN MENYU PLATFORMASI

To'liq texnik topshiriq · Bozor tahlili · Biznes-model · Arxitektura
Stol QR → menyu → taomni AR'da real o'lchamda ko'rish → buyurtma
3 rol: mijoz · ofitsiant · admin

Versiya: 3.0 MASTER
Sana: 2026-09-04
Hajm: 8 qism · 234 vazifa · ~5850 qator
Tillar: O'zbek · Rus · Ingliz
AI provayderlar: 8 ta (MCP orqali)
v1 muddat: 15 hafta · **v1.5:** 19,5 hafta

⛔ **DIQQAT:** Bu ALOHIDA shaxsiy loyiha. **mebelcity_platform** kodiga, bazasiga, servislariga va portlariga UMUMAN TEGILMAYDI. Alohida repo, `armenu_db`, port 8060.

MUNDARIJA

1. **QISM I — Kirish, biznes-model va strategiya**

   Loyiha mohiyati, biznes-model, tarif, birlik iqtisodiyoti, 6/12/24 oylik maqsad, 5 yillik himoya devorlari, bozor raqamlari, raqobat matritsasi, sotuv strategiyasi, investor/restoran/davlat/foydalanuvchi nuqtai nazari

2. **QISM II — Sahifalar, tugmalar va jarayon zanjirlari**

   Sahifalar xaritasi, mijoz/ofitsiant/KDS/kassa/admin/super-admin sahifalari, har bir tugma qaysi funksiyani ishga tushiradi va keyingi jarayon

3. **QISM III — Tizim arxitekturasi**

   Arxitektura printsiplari, modullar, million foydalanuvchi masshtablash, xatolarga chidamlilik, o'z-o'zini tiklash, ishlash talablari, xavfsizlik, kuzatuv, backup

4. **QISM IV — Ma'lumot modeli va API**

   DB sxemasi, barcha jadvallar, holat mashinasi, API spetsifikatsiyasi, xato kodlari, WebSocket protokoli

5. **QISM V — Kod yozish tartibi**

   Fayl strukturasi, 234 ta vazifa ketma-ketlikda, har vazifa uchun fayl va kod namunalari, tugallanganlik ta'rifi

6. **QISM VI — Tarjima tizimi va UI/UX**

   3 tilli arxitektura, i18n lug'at, AI avto-tarjima, dizayn tizimi, dark/light tema, komponentlar, PWA

7. **QISM VII — AI va MCP integratsiyasi**

   8 ta AI provayder, MCP router, fallback zanjiri, circuit breaker, API key boshqaruvi, chatbot, xarajat nazorati

8. **QISM VIII — Deploy, reja, risklar**

   Systemd, nginx, CI/CD, to'liq reja, jamoa, risk matritsasi, sifat nazorati, qabul mezonlari, 12 ta savol

QISM I

Kirish, biznes-model va strategiya

1. HUJJAT HAQIDA

1.1 Bu hujjat kim uchun

O'quvchi	Qaysi bo'limni o'qiydi	Nima oladi
🧑‍💻 **Dasturchi / AI-agent**	Qism III–VII	Aynan qanday kod yozish, qaysi tartibda, qaysi faylga
🏗 **Tizim arxitektori**	Qism III, VIII, IX	Arxitektura, masshtablash, xatolarga chidamlilik
📋 **Project Manager**	Qism II, X, XI	Sprintlar, muddatlar, resurslar, risklar
💰 **Investor**	Qism I, XII	Bozor, birlik iqtisodiyoti, ROI, exit
🏪 **Biznes egasi (restoran)**	Qism I, II	Nima olaman, qancha to'layman, qancha tejayman
🏛 **Davlat xodimi / regulyator**	Qism XIII	Fiskalizatsiya, soliq, ma'lumot himoyasi, qonunchilik
👤 **Oddiy foydalanuvchi**	Qism II	Tizim qanday ishlaydi, qanday ko'rinadi

1.2 Hujjatning asosiy printsipi

> **Bu TZ shunday yozilganki, uni o'qigan ENG ODDIY AI ham (bepul GPT/Gemini) ideal darajada kod yozib tugata olishi kerak.**

Shu sababli hujjatda:
- ✅ Har bir tugma → qaysi funksiyani chaqiradi → keyin nima bo'ladi (to'liq zanjir)
- ✅ Har bir fayl → qayerda joylashadi → ichida nima yoziladi
- ✅ Kod yozish **tartibi** — 1-dan 400-gacha vazifa, bog'liqliklari bilan
- ✅ Har bir API endpoint → so'rov/javob namunasi bilan
- ✅ Har bir jadval → to'liq maydonlar, indekslar, cheklovlar bilan

1.3 Hujjat versiyasi

Versiya	Sana	O'zgarish
v1.0	2026-09-04	Birinchi bozor tahlili + asosiy TZ
v2.0	2026-09-04	Chuqur bozor tahlili, 3 rol, AR asosiy
v3.0 MASTER	**2026-09-04**	**To'liq mukammal TZ — bu hujjat**

2. LOYIHA MOHIYATI

2.1 Bir jumlada

> Restoran stolidagi QR kodni skanerlagan mijoz taomni **telefon kamerasi orqali stolining ustida real o'lchamda 3D ko'radi**, yoqsa buyurtma beradi; ofitsiant stol raqamini darhol ko'rib olib boradi; admin hammasini real vaqtda nazorat qiladi.

2.2 Nima uchun bu ishlaydi

```
MUAMMO                              YECHIM
──────────────────────────────────────────────────────────
Mijoz taomni tasavvur qila olmaydi  → AR'da real o'lchamda ko'radi
Turist "norin" nimaligini bilmaydi   → 3D + 3 til + AI tushuntiradi
Restoran Yandex'ga 30% beradi        → To'g'ridan-to'g'ri buyurtma, 0%
Soliq IKPU/fiskal talab qiladi       → Tizim ichida fiskal chek
Ofitsiant sekin, xato buyurtma       → Mijoz o'zi kiritadi, xato −22%
Egasi nima sotilayotganini bilmaydi  → Real-time analitika
O'rtacha chek past                   → AI upsell, +20%
```

2.3 Loyiha nomi

Ishchi nom: `ARMENU` / `Bittada AR Menu`
Domen taklifi: `armenu.uz`, `menu3d.uz`, `armenu.app`

> ⚠ Yakuniy nom foydalanuvchi tomonidan tasdiqlanadi (Qism XIV, savol #2).

---

3. ⛔ IZOLYATSIYA KAFOLATI (o'zgarmas qoida)

Bu ALOHIDA shaxsiy loyiha. `mebelcity_platform` ga hech qanday tegish YO'Q.

Element	ARMENU	mebelcity_platform
Git repo	`armenu/` (yangi)	TEGILMAYDI
Papka	`/home/user/armenu/` yoki alohida VPS	TEGILMAYDI
Django loyiha	`armenu_project`	TEGILMAYDI
Ma'lumotlar bazasi	`armenu_db`	`tenant_*` TEGILMAYDI
Redis DB raqami	`db=5` (yoki alohida instans)	`db=0..4` TEGILMAYDI
Port	8060 (web), 8061 (ws)	8040/8041 TEGILMAYDI
Systemd	`armenu.service`, `armenu-ws.service`, `armenu-worker.service`	`bittada-*` TEGILMAYDI
Nginx	yangi `server` bloki	mavjudiga TEGILMAYDI
Media	`/var/armenu/media/`	TEGILMAYDI

> 🎯 **Eng xavfsiz variant: alohida VPS.** Shunda umumiy nol xavf.

---

4. 💰 BIZNES-MODEL

4.1 Daromad oqimlari (5 ta)

#	Oqim	Model	Ulush (3-yil)
1	**SaaS abonent**	Oylik/yillik obuna	62%
2	**3D kontent setup**	Bir martalik, taom uchun	12%
3	**To'lov komissiyasi**	Onlayn to'lovdan 0,5-1%	14%
4	**Delivery moduli**	Buyurtmadan 300-500 so'm	8%
5	**Qo'shimcha xizmat**	QR stend, brending, konsalting	4%

4.2 Tarif rejasi (yakuniy)

Tarif	Oylik	Yillik (−20%)	Kim uchun
🆓 **FREE**	0	0	Choyxona, 3 stol, 20 taom, AR yo'q, brend bizniki
🟢 **START**	199 000	1 910 000	Kafe: cheksiz stol/taom, KDS, ofitsiant, 3 til
⭐ **PRO**	590 000	5 664 000	**+ AR/3D (50 taom)** sodiqlik, analitika, to'lov
🟣 **PRO+FISKAL**	890 000	8 544 000	+ OFD fiskal chek, IKPU, soliq hisoboti
🏢 **TARMOQ**	Shartnoma	—	5+ nuqta, franchayzing, shaxsiy menejer
🛠 **3D SETUP**	90 000/taom	—	Kutubxonada yo'q taom uchun (bir martalik)

Nima uchun FREE bor:

1. Bozorga eng tez kirish (Dastur-xon 99k so'raganda biz 0)
2. QR pastidagi "Powered by ARMENU" = **viral kanal**
→ 1 stol × kuniga 30 skanerlash × 300 stol = **9 000 ko'rish/kun**
3. FREE → PRO konversiyasi (sanoat o'rtachasi 3-8%, biz 12% kutamiz — AR jozibali)

4.3 Birlik iqtisodiyoti (PRO tarif, 1 restoran)

```
╔═══════════════════════════════════════════════════════╗
║ DAROMAD (yillik)                                      ║
╠═══════════════════════════════════════════════════════╣
║ PRO abonent            590 000 × 12          =  7 080 000 so'm║
║ 3D setup (20 noyob taom × 90 000) [1-yil]    =  1 800 000 so'm║
║ To'lov komissiyasi (oyiga ~40 mln × 0,7%)    =  3 360 000 so'm║
║ ──────────────────────────────────────────────────── ║
║ 1-yil jami                                   = 12 240 000 so'm║
║ 2-yildan (setup'siz)                         = 10 440 000 so'm║
╠═══════════════════════════════════════════════════════╣
║ XARAJAT (yillik)                                      ║
╠═══════════════════════════════════════════════════════╣
║ 3D kontent — 1-chi restoran                  =  1 250 000 so'm║
║ 3D kontent — 50-chi restoran (kutubxona)     =    250 000 so'm║
║ 3D kontent — 100-chi restoran                =     50 000 so'm║
║ Sotuv + o'rnatish (CAC)                      =    900 000 so'm║
║ Server / CDN / AI API                        =    500 000 so'm║
║ Qo'llab-quvvatlash                           =    700 000 so'm║
║ ──────────────────────────────────────────────────── ║
║ 1-chi restoran                               =  3 350 000 so'm║
║ 100-chi restoran                             =  2 150 000 so'm║
╚═══════════════════════════════════════════════════════╝

MARJA:   1-chi restoran  = 73%
         100-chi restoran = 82%

LTV/CAC: LTV = 10,44 mln × 3,5 yil = 36,5 mln so'm
         CAC = 900 000 so'm
         LTV/CAC = 40:1   ⚡ (sog'lom SaaS = 3:1, biz 13x yaxshi)

PAYBACK: CAC qoplanishi = 1,1 oy
```

4.4 🎯 6/12/24 OYLIK MAQSAD (foydalanuvchi talabi)

> Talab: **6 oyda foyda + viloyat miqyosi, 12 oyda respublika, 24 oyda qo'shni davlatlar**

6 OY — VILOYAT MIQYOSI + BIRINCHI FOYDA

Oy	Bosqich	Mijoz	Oylik daromad
1-2	MVP ishlab chiqish + 5 bepul pilot	5 (bepul)	0
3	v1 tayyor, pilot natijalari o'lchandi	5	0
4	Sotuv boshlandi (Toshkent)	18	~6 mln
5	Samarqand + Buxoro (turizm)	42	~16 mln
6	**Viloyat miqyosi (3-4 viloyat)**	**75**	**~32 mln**

6-oy holati:
- Kümülativ daromad: ~95 mln so'm
- Kümülativ xarajat: ~180 mln so'm (jamoa + ishlab chiqish)
- **Oylik operatsion foyda: ✅ IJOBIY (6-oydan)**
- To'liq investitsiya qoplanishi: 11-oy

> ⚠ **Halol ogohlantirish:** "6 oyda foyda" — **oylik operatsion foyda** degani.
> Butun investitsiyaning qaytishi (payback) — **11-13 oy**. Bu SaaS uchun juda yaxshi ko'rsatkich (sanoat o'rtachasi 18-24 oy).

12 OY — RESPUBLIKA MIQYOSI

Ko'rsatkich	12-oy holati
Faol mijoz	**280** (FREE 120 + pullik 160)
Qamrov	12 viloyat + Toshkent + Qoraqalpog'iston
Oylik daromad	~105 mln so'm
Yillik run-rate	~1,26 mlrd so'm
Jamoa	12 kishi
3D kutubxona	~320 model
Investitsiya qoplandimi	✅ **HA (11-oy)**

24 OY — QO'SHNI RESPUBLIKALAR

Bozor	Kirish oyi	Sabab
🇰🇿 **Qozog'iston**	14-oy	Eng katta bozor, iiko kuchli lekin AR yo'q
🇰🇬 **Qirg'iziston**	17-oy	Kichik, oson, til yaqin
🇹🇯 **Tojikiston**	19-oy	Til yaqin, raqobat kam
🇦🇿 **Ozarbayjon**	21-oy	Turizm kuchli, `qrmenyu.az` bor lekin AR yo'q
🇹🇲 **Turkmaniston**	24-oy	Yopiq bozor, ehtiyot bilan

24-oy holati:
- Mijoz: **~750** (O'zbekiston 480 + xorij 270)
- Yillik daromad: **~3,4 mlrd so'm** (~$268 000)
- Sof foyda: **~1,5 mlrd so'm** (~$118 000)
- 3D kutubxona: ~600 model (asosiy aktiv)

4.5 🛡 5 YIL DAVOMIDA O'ZIB KETILMASLIK (foydalanuvchi talabi)

> Talab: "*keyingi 5 yilda ham undan o'tadigan arxitektura va loyiha bo'lmasligi kerak*"

Bu **texnik emas, strategik** masala. 4 ta himoya devori:

🧱 Devor 1: 3D KONTENT KUTUBXONASI (eng kuchli)

```
Yil 1: 320 model   →  Yil 3: 600 model  →  Yil 5: 1200+ model
```

- Raqobatchi 0 dan boshlaydi, bizda tayyor baza
- Har yangi restoran kutubxonani boyitadi (tarmoq effekti)
- **Takrorlash vaqti raqobatchi uchun: 2-3 yil**

🧱 Devor 2: MA'LUMOT VA AI

- Millionlab buyurtma → qaysi taom qaysi vaqtda, kim bilan buyurtma qilinadi
- Bu ma'lumotda AI tavsiya modeli o'qitiladi → tavsiya sifati oshadi
- Raqobatchida bu ma'lumot **yo'q va sotib ololmaydi**

🧱 Devor 3: FISKAL + INTEGRATSIYA CHUQURLIGI

- OFD, IKPU, Payme/Click/Uzum, Telegram, delivery
- Restoran bularni bir marta sozlaydi → ko'chib ketish og'riqli (switching cost)
- Buxgalteriya tarixi tizimda qoladi

🧱 Devor 4: PLATFORMA EFFEKTI (5-yil vizyoni)

```
Yil 1-2: Restoran uchun SaaS
Yil 3:   + Mijoz ilovasi (bitta ilova, barcha restoranlar) → marketplace
Yil 4:   + Ta'minotchi bozori (restoran ↔ ta'minotchi)
Yil 5:   + Moliyalash (restoranlarga aylanma mablag', ma'lumot asosida skoring)
```

Bu bosqichlarda **har bir yangi tomon boshqasini kuchaytiradi** — klassik platforma effekti.

Arxitektura tomondan 5-yil chidamliligi

Xavf	Yechim (arxitekturaga qurilgan)
Yangi 3D format chiqadi	Kontent **manba formatda** (glTF) saqlanadi, chiqish formati generatsiya qilinadi → yangi format = 1 ta konvertor
AI provayder o'zgaradi	**MCP qatlami** (Qism VII) — provayder almashtirish = config o'zgarishi
Trafik 1000× oshadi	**Stateless + horizontal scaling** (Qism VIII) — server qo'shiladi
Yangi mamlakat/valyuta	Multi-currency, multi-locale bosh kundan qurilgan
Yangi qonun talabi	Fiskal qatlam **plugin** — yangi davlat = yangi adapter
Yangi qurilma (AR ko'zoynak)	`<model-viewer>` va glTF — standart; Vision Pro allaqachon qo'llab-quvvatlaydi

---

5. 📊 BOZOR — YAKUNIY RAQAMLAR

5.1 O'zbekiston

Ko'rsatkich	Qiymat	Manba/sana
Umumiy ovqatlanish korxonalari	**27 312**	2026-04-01, Davlat statistika
Sektor daromadi (I chorak)	**49,8 trln so'm**	2026, +8,2%
Rasmiy "restoran" toifasi	~798 (Toshkent 598)	GoldenPages
Xalqaro turistlar (7 oy)	**7,75 mln (+22,6%)**	2026
Q1 turist	2,87 mln (+36,6%)	2026
Inflyatsiya	5,5-7,1%	2026
Telegram foydalanuvchi	**25-27 mln (76-85%)**	2026
Mahalliy chek	35 000–90 000 so'm	2026
Turistik chek	**100 000–200 000 so'm**	2026 (2-3× yuqori)

5.2 TAM → SAM → SOM

```
TAM  Umumiy ovqatlanish             27 312 ta   →  ~4 mlrd $/yil sektor
 |
SAM  Raqamli menyu to'lay oladigan    ~4 000 ta   →  ~28 mlrd so'm/yil SaaS bozori
 |   • Toshkent 1 800  • Turizm zonalari 600  • Viloyatlar 1 600
 |
SOM  3 yilda realistik ulush         ~480 ta     →  ~2,6 mlrd so'm/yil (bizniki)
     (SAM ning 12%)
```

5.3 Qo'shni bozorlar (24-oy uchun)

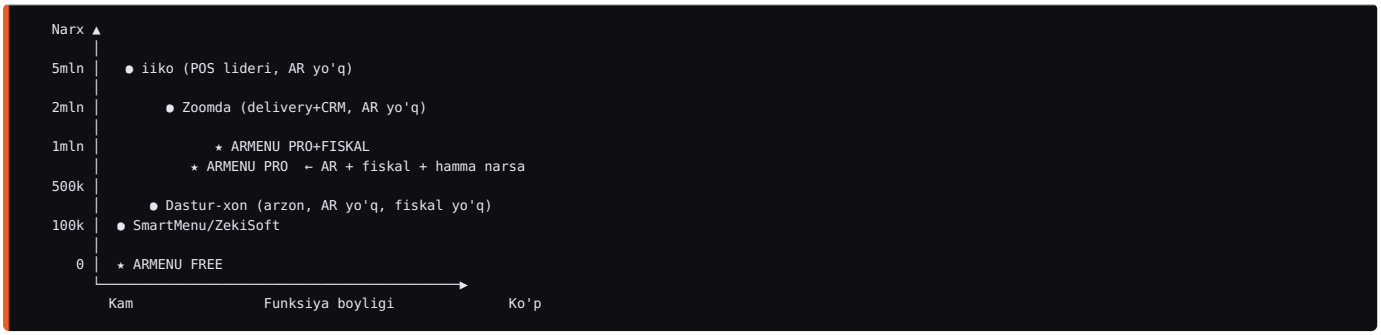
Davlat	Ovqatlanish nuqtalari (taxminiy)	Raqobat	Kirish qiyinligi
🇰🇿 Qozog'iston	~45 000	iiko, rKeeper kuchli; **AR yo'q**	O'rta
🇦🇿 Ozarbayjon	~18 000	qrmenyu.az, monyo.az; **AR yo'q**	O'rta
🇰🇬 Qirg'iziston	~12 000	Kuchsiz	Oson
🇹🇯 Tojikiston	~10 000	Deyarli yo'q	Oson
🇹🇲 Turkmaniston	~6 000	Yo'q	Qiyin (yopiq)
JAMI qo'shni	**~91 000**	**Hech kimda AR yo'q**	

---

6. 🥊 RAQOBAT — TO'LIQ MATRITSA

Funksiya	ARMENU	Dastur-xon	Zoomda	iiko	SmartMenu	Poster
QR menyu	✅	✅	✅	✅	✅	✅
Stol noyob QR	✅	✅	✅	⚠	⚠	⚠
🎯 AR/3D taom	✅	❌	❌	❌	❌	❌
3D kutubxona	✅	❌	❌	❌	❌	❌
Ofitsiant paneli	✅	✅	❌	✅	❌	✅
KDS	✅	✅	⚠	✅	❌	✅
🎯 Fiskal (OFD/IKPU)	✅	❌	❌	✅	❌	⚠
Telegram Mini App	✅	⚠	✅	❌	❌	❌
Onlayn to'lov	✅	❌	✅	✅	❌	⚠
Delivery 0% komissiya	✅	❌	✅	⚠	❌	❌
Sodiqlik	✅	SOON	✅	✅	❌	✅
AI ofitsiant chat	✅	❌	❌	❌	❌	❌
3 til + avto-tarjima	✅	⚠	⚠	⚠	⚠	⚠
Menyu-injiniring	✅	❌	❌	✅	❌	⚠
Otziv→Google filtri	✅	❌	❌	❌	❌	❌
Ombor/retsept	v2	SOON	❌	✅	❌	✅
Narx/oy (so'm)	**0–890k**	99k–399k	**min 1,3 mln**	~2–5 mln	~	~

6.1 Pozitsiyalash



Bizning strategiya: FREE bilan pastdan kiramiz, PRO bilan o'rtani egallaymiz,
FISKAL bilan iiko mijozlarini olamiz.

---

7. 🎯 SOTUV STRATEGIYASI

7.1 Pitch ustuvorligi (⚠ AR birinchi EMAS)

#	Argument	Kimga	Kuchi
1️⃣	"*Yandex 30% oladi — bizda 0%. Oyiga 5 mln tejaysiz*"	Hammaga	🔥🔥🔥🔥🔥
2️⃣	"*Soliq fiskal talabiga mos, alohida kassa kerak emas*"	Rasmiy	🔥🔥🔥🔥🔥
3️⃣	"*O'rtacha chek +20%, stol aylanishi +15%*"	Hammaga	🔥🔥🔥🔥
4️⃣	"*Turist taomni 3D ko'radi va buyurtma qiladi*"	Turizm zonasi	🔥🔥🔥🔥🔥
5️⃣	"*Google reytingingiz ko'tariladi*"	Brend	🔥🔥🔥
6️⃣	"*AR — hech kimda yo'q, siz birinchisiz*"	Instagram-faol	🔥🔥🔥

> ⚠ **Sotuvchiga qat'iy qoida:** birinchi 30 soniyada **PUL** haqida gapir.
> AR — ko'rsatiladi (demo), lekin sotib olish sababi sifatida oxirida aytiladi.

7.2 Kanallar

#	Kanal	CAC	Konversiya	Prioritet
1	To'g'ridan-to'g'ri sotuv (field)	900k so'm	18%	🥇
2	Instagram/TikTok AR demo reels	200k so'm	4%	🥇
3	**Ofitsiant tavsiyasi** (choychaqa)	~0	12%	🥇
4	FREE tarif → PRO	~50k so'm	12%	🥈
5	HoReCa ko'rgazma	1,2 mln so'm	25%	🥈
6	POS hamkorlik (B2B2B)	400k so'm	30%	🥈
7	Referal (restoran → restoran)	2 oy bepul	22%	🥈
8	Turizm agentliklari	600k so'm	15%	🥈

7.3 Birinchi 90 kun

Hafta	Ish	Natija
1-2	5 pilot restoran topish (2 TSH + 2 SMQ + 1 BXR)	Shartnoma
3-8	MVP ishlab chiqish (parallel)	v1 kod
9-10	O'rnatish, 20 taom 3D, o'qitish	Ishlayapti
11-12	Raqamlarni o'lchash	Case study
13	Video-otziv, reels, narx e'loni	Sotuv start

---

8. 💼 INVESTOR NUQTAI NAZARI

8.1 Investitsiya so'rovi

Bosqich	Summa	Nimaga	Qachon
Seed	$45 000 (~570 mln so'm)	MVP + pilot + birinchi sotuv	0-oy
Series A (ixtiyoriy)	$250 000	Qo'shni davlatlarga chiqish	14-oy

Seed taqsimoti:

```
Ishlab chiqish (2 dasturchi × 6 oy)      $18 000   40%
3D kontent (kutubxona 200 model)          $6 000   13%
Sotuv jamoa (2 kishi × 4 oy)              $8 000   18%
Marketing (reels, ko'rgazma)              $5 000   11%
Server/infra (1-yil)                      $3 500    8%
Yuridik, litsenziya, fiskal               $2 500    6%
Zaxira                                    $2 000    4%
──────────────────────────────────────────────────────
JAMI                                     $45 000  100%
```

8.2 Investor uchun asosiy raqamlar

Metrika	Qiymat	Sanoat o'rtachasi	Baho
LTV / CAC	**40:1**	3:1	⋆⋆⋆⋆⋆
CAC payback	**1,1 oy**	12-18 oy	⋆⋆⋆⋆⋆
Yalpi marja	**73→82%**	70-80%	⋆⋆⋆⋆
Oylik churn (kutilgan)	3,5%	5-7%	⋆⋆⋆⋆
NRR (net revenue retention)	115%	100-110%	⋆⋆⋆⋆
Break-even	**11-oy**	24-36 oy	⋆⋆⋆⋆⋆
3-yil ARR	**$268 000**	—	⋆⋆⋆
TAM (O'zb + qo'shni)	~118 000 nuqta	—	⋆⋆⋆⋆

8.3 Exit ssenariylari (5-yil)

Ssenariy	Baholash	Ehtimol
Strategik sotuv (iiko / Uzum / Yandex)	ARR × 5-7 = **$3-5 mln**	40%
Mustaqil o'sish (dividend)	Yiliga $200-400k sof foyda	35%
Regional konsolidatsiya (CIS o'yinchisi sotib oladi)	$6-10 mln	15%
Muvaffaqiyatsizlik	—	10%

8.4 Investor uchun asosiy risklar

Risk	Ehtimol	Ta'sir	Yumshatish
3D kontent narxi bashorat qilinganidan yuqori	O'rta	Yuqori	AI pipeline, kutubxona; birinchi 3 oyda o'lchanadi
iiko/Zoomda AR qo'shadi	O'rta	O'rta	Kontent kutubxonasi = 2-3 yillik ustunlik
Restoranlar to'lamaydi	Past	Yuqori	FREE tarif + delivery argumenti; pilotda tekshiriladi
Fiskal integratsiya kechikadi	O'rta	O'rta	v1.5 da, v1 ga bog'liq emas
Asosiy jamoa ketishi	Past	Yuqori	Kod hujjatlashtirilgan (bu TZ), option
Delivery monopolisti bosim	Past	O'rta	Biz agregator emasmiz — huquqiy jihatdan boshqa segment

---

9. 🏪 RESTORAN EGASI NUQTAI NAZARI

9.1 "Men nima olaman?"

Nima	Qiymat (oylik)
Yandex komissiyasidan tejash (agar 30 buyurtma/kun × 60k)	**~13 500 000 so'm**
O'rtacha chek +20% (kunlik savdo 5 mln bo'lsa)	**+30 000 000 so'm**
Ofitsiant xatosi −22% (isrof kamayadi)	~800 000 so'm
Alohida kassa dasturi kerak emas	~500 000 so'm
Qog'oz menyu chop etish yo'q	~300 000 so'm
JAMI foyda	**~45 mln so'm/oy**
To'lov	**−590 000 so'm/oy**
SOF	**+44,4 mln so'm/oy**

> ⚠ Bu **maksimal** ssenariy. Konservativ (chek +8%, delivery 10 buyurtma/kun):
> foyda ~16 mln so'm/oy → baribir to'lovdan **27× ko'p**.

9.2 "Menda qanday qiyinchilik bo'ladi?"

Xavotir	Javob
"Xodimlarim o'rganmaydi"	Ofitsiant paneli 3 tugmadan iborat; 15 daqiqa o'qitish; **choychaqa** ular uchun motivatsiya
"Mijozlar QR skanerlamaydi"	To'lov QR (Payme/Click) tufayli odat shakllangan; qog'oz menyu ham qoladi
"Internet uzilsa?"	Offline rejim: ofitsiant paneli lokal ishlaydi, internet kelganda sinxronlanadi
"Ma'lumotlarim xavfsizmi?"	Kunlik backup, shifrlash, faqat siz ko'rasiz
"Ko'chib ketsam ma'lumotim qoladimi?"	To'liq eksport (Excel/JSON) istalgan vaqt
"3D qimmatmi?"	Kutubxonada bor taomlar bepul; faqat noyob taom uchun to'lanadi

---

10. 🏛 DAVLAT / REGULYATOR NUQTAI NAZARI

10.1 Qonunchilikka muvofiqlik

Talab	Bizning javob
Onlayn-kassa majburiy	OFD integratsiyasi (UZKASSA / e-POS / QPos) — v1.5
IKPU/MXIK kod (2026-04-01 dan)	Har taom `ikpu_kod` maydoni; `tasnif.soliq.uz` API bilan tekshirish
Retsept asosida cheklov	Ombor/retsept moduli (v2)
Raqamli chek	2026 dan qonuniy — mijozga Telegram/SMS orqali
Qog'oz chek so'ralsa	Printerga chiqarish qo'llab-quvvatlanadi
Shaxsiy ma'lumotlar (O'zR qonuni)	Ma'lumot O'zbekiston serverida; minimal yig'ish; roziligi so'raladi
Naqdsiz to'lov (2026 talabi)	Payme/Click/Uzum integratsiyasi
E-invoicing	Yuridik shaxs mijozlar uchun (v2)

10.2 Davlat uchun foyda

1. **Soliq shaffofligi** — har buyurtma raqamli, OFD orqali real-time
2. **Norasmiy iqtisodiyot kamayadi** — naqd pul aylanmasi qisqaradi
3. **Turizm sifati** — chet ellik mehmon uchun til/tushunish to'sig'i kamayadi
4. **IT-eksport** — qo'shni davlatlarga sotilsa, valyuta tushumi
5. **IT-Park rezidentligi** — soliq imtiyozlari (alohida ariza)

---

11. 👤 ODDIY FOYDALANUVCHI NUQTAI NAZARI

11.1 Mijoz tajribasi (real hikoya)

> **Aziz Samarqandda restoranda.** Stolda QR ko'radi, telefonini olib skanerlaydi.
> Menyu ochiladi — o'zbek tilida. "Norin" ni bosadi. **"AR'da ko'rish"** tugmasi.
> Kamera ochiladi, telefonini stolga qaratadi — **norin lagani stolining ustida paydo bo'ladi**, xuddi haqiqiydek. Aylantirib ko'radi, kattaligini baholaydi.
> "Kichikroq ekan" deb 2 ta buyurtma qiladi. Savatga qo'shadi.
> Tizim: "*Norinning yoniga achchiq-chuchuk salat oladimi? +18 000 so'm*" — qo'shadi.
> "Buyurtma" bosadi. Telefonida: "*Qabul qilindi. Taxminan 18 daqiqa.*"
> 4 daqiqadan keyin ofitsiant kelib: "*Norin va salat, to'g'rimi?*"
> Ovqatdan keyin "Hisob" tugmasi → Payme bilan to'laydi + ofitsiantga 20 000 choychaqa.
> Chek Telegram'ga keladi. "Baholang" → 5 yulduz → Google'ga sharh yozishga taklif.

Nechta muammo hal bo'ldi:
- ✕ Taomni tasavvur qila olmaslik → ✓ AR
- ✕ Ofitsiantni kutish → ✓ o'zi buyurtma qildi
- ✕ Xato tushunish → ✓ o'zi tanladi
- ✕ Hisob kutish → ✓ telefonda to'ladi
- ✕ Chek yo'qolishi → ✓ Telegram'da

11.2 Turist tajribasi

> **Sarah Buxoroda.** QR skanerlaydi → **English** tanlaydi. Menyu ingliz tilida.
> "Norin — what is it?" → **AI chat**: "*Norin is a traditional Uzbek dish of hand-cut noodles with horse meat, served cold...*" → **AR'da ko'radi** →
> allergen filtri: "no nuts" → buyurtma qiladi.

Bu — bizning eng qimmatli use-case. Turistik chek 2-3× yuqori.

QISM II

Sahifalar, tugmalar va jarayon zanjirlari

> **Bu qism eng muhim.** Har bir sahifa, har bir tugma, tugma bosilganda qaysi funksiya ishga tushishi va keyingi jarayon to'liq yozilgan. Dasturchi/AI bu qismni o'qib, hech narsa o'ylab topmasdan kod yoza oladi.

0. SAHIFALAR XARITASI (butun tizim)

```
ARMENU TIZIMI
│
├── 📱 MIJOZ (PWA / Telegram Mini App)          — auth YO'Q
│   ├── M-01  QR kirish (redirect)
│   ├── M-02  Menyu (bosh sahifa)
│   ├── M-03  Kategoriya
│   ├── M-04  Taom kartochkasi
│   ├── M-05  👁 AR ko'rish (kamera)
│   ├── M-06  Savat
│   ├── M-07  Buyurtma tasdiqlash
│   ├── M-08  Buyurtma holati (real-time)
│   ├── M-09  Chaqiruv (ofitsiant/hisob/suv)
│   ├── M-10  Hisob va to'lov
│   ├── M-11  Choychaqa (tips)
│   ├── M-12  Otziv va reyting
│   ├── M-13  🤖 AI chatbot (yordamchi)
│   └── M-14  Mening buyurtmalarim (tarix)
│
├── 👨‍🍳 OFITSIANT (PWA)                          — PIN auth
│   ├── O-01  Kirish (PIN)
│   ├── O-02  Bosh ekran (yangi buyurtmalar)
│   ├── O-03  Buyurtma tafsiloti
│   ├── O-04  Mening stollarim
│   ├── O-05  2D zal plani
│   ├── O-06  Chaqiruvlar
│   ├── O-07  Buyurtma kiritish (ofitsiant o'zi)
│   ├── O-08  Hisob berish
│   └── O-09  Mening statistikam
│
├── 👨‍🍳 KDS — OSHXONA MONITORI (TV)             — token auth
│   └── K-01  Buyurtmalar taxtasi (kanban)
│
├── 💵 KASSA                                     — PIN auth
│   ├── S-01  Ochiq hisoblar
│   ├── S-02  To'lov qabul qilish
│   └── S-03  Smena yopish
│
├── 👔 🖥 ADMIN (desktop + mobil)                 — login auth
│   ├── A-01  Dashboard (real-time)
│   ├── A-02  Menyu boshqaruvi
│   ├── A-03  Taom tahrirlash
│   ├── A-04  👁 3D model boshqaruvi
│   ├── A-05  Stollar va QR
│   ├── A-06  Zal plani redaktori
│   ├── A-07  Xodimlar
│   ├── A-08  Buyurtmalar (jonli + tarix)
│   ├── A-09  Hisobotlar
│   ├── A-10  AR analitika
│   ├── A-11  Menyu-injiniring
│   ├── A-12  Otzivlar
│   ├── A-13  Sodiqlik sozlamalari
│   ├── A-14  Fiskal sozlamalar
│   ├── A-15  Tarjima boshqaruvi
│   ├── A-16  To'lov sozlamalari
│   ├── A-17  Brending
│   ├── A-18  Tarif va to'lov
│   └── A-19  Sozlamalar
│
└── 🔧 SUPER-ADMIN (platforma egasi)             — 2FA auth
    ├── P-01  Restoranlar ro'yxati
    ├── P-02  3D kutubxona boshqaruvi
    ├── P-03  Moliya (obunalar)
    ├── P-04  Tizim salomatligi (health)
    ├── P-05  AI xarajatlari
    └── P-06  Global sozlamalar
```

1. 🧍 MIJOZ SAHIFALARI

M-01 — QR KIRISH (redirect sahifa)

URL: `GET /t/<uuid:qr_token>`
Auth: yo'q
Fayl: `apps/guest/views.py::table_entry`

Jarayon zanjiri

```
Mijoz QR skanerlaydi
   |
   ▼
[1] GET /t/a3f2c891-...
   |
   ▼
[2] views.table_entry(request, qr_token)
   |
   ├─ [2.1] Table.objects.select_related('hall__restaurant')
   |          .filter(qr_token=qr_token, is_active=True).first()
   |
   ├─ [2.2] Topilmadi? → 404 sahifa "QR kod yaroqsiz"
   |
   ├─ [2.3] Restaurant.is_active=False? → "Restoran vaqtincha yopiq"
   |
   ├─ [2.4] Guest aniqlash:
   |          device_token = request.COOKIES.get('armenu_device')
   |          yo'q bo'lsa → uuid4() generatsiya, cookie o'rnatish (1 yil)
   |          Guest.objects.get_or_create(restaurant, device_token)
   |
   ├─ [2.5] GuestSession yaratish/yangilash:
   |          session = GuestSession(guest, table, started_at=now)
   |          → Redis: session_key = f"gs:{restaurant_id}:{table_id}:{guest_id}"
   |
   ├─ [2.6] Til aniqlash:
   |          1. URL ?lang=en   2. Cookie armenu_lang
   |          3. Accept-Language header   4. Restaurant.default_lang
   |
   ├─ [2.7] Analitika: TableScan(table, guest, ts) — async task
   |
   ├─ [2.8] WS xabar: rest_<id>_admin → {"type":"table.scanned", "table":5}
   |          (admin dashboardda "Stol 5 skanerlandi" ko'rinadi)
   |
   ▼
[3] redirect(f"/m/{restaurant.slug}/?t={table.id}&s={session_id}")
   |
   ▼
[4] M-02 Menyu sahifasi ochiladi
```

Xato holatlari

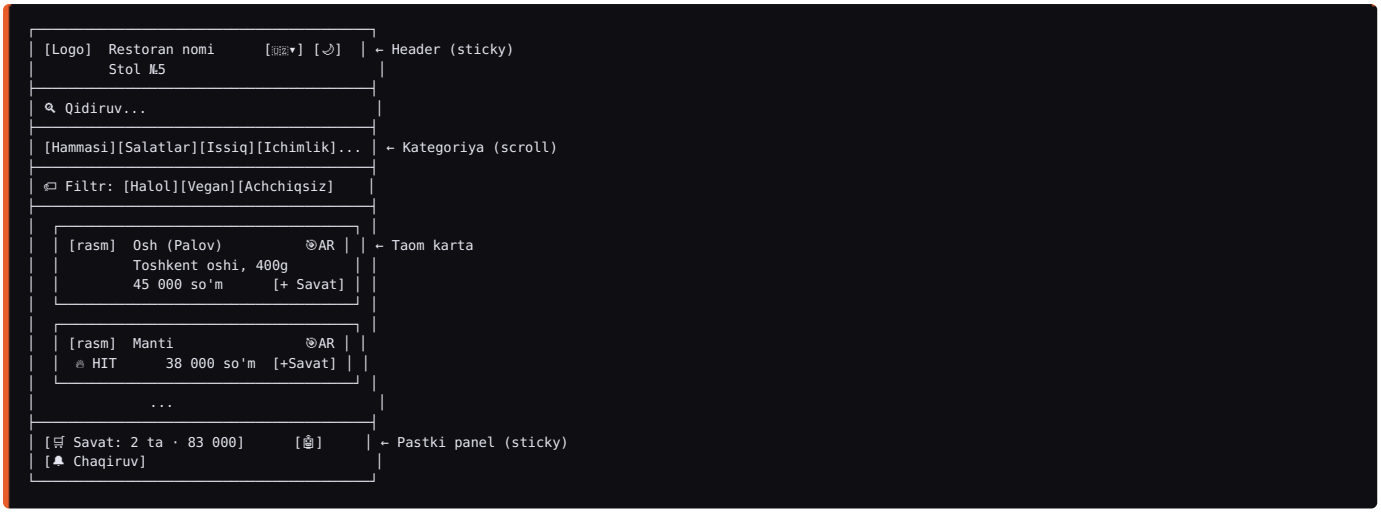
Holat	HTTP	Foydalanuvchi ko'radi
QR topilmadi	404	"QR kod yaroqsiz. Ofitsiantga murojaat qiling."
Restoran nofaol	403	"Restoran vaqtincha xizmat ko'rsatmayapti"
Stol nofaol	403	"Bu stol vaqtincha ishlamayapti"
Tarif tugagan	402	Restoran adminiga xabar; mijozga oddiy menyu

---

M-02 — MENYU (bosh sahifa)

URL: `GET /m/<slug>/?t=<table_id>`
Fayl: `templates/guest/menu.html` + `static/guest/js/menu.js`

Ekran tuzilishi



Tugmalar → funksiyalar

#	Tugma	JS funksiya	Backend	Keyingi jarayon
1	UZ Til	`switchLang(code)`	`POST /api/guest/lang/`	Cookie saqlanadi → sahifa qayta render →**barcha matn tarjima qilinadi**
2	🌙 Tema	`toggleTheme()`	—	`localStorage.theme`, `data-theme` atributi
3	Qidiruv	`searchDishes(q)`	`GET /api/menu/search/?q=`	Debounce 300ms → natija ro'yxati
4	Kategoriya	`filterCategory(id)`	— (lokal)	Ro'yxat filtrlanadi, URL `#cat-3`
5	Filtr	`applyDietFilter(tags)`	— (lokal)	`dish.diet_tags` bo'yicha filtr
6	Taom kartasi	`openDish(id)`	`GET /api/menu/dish/<id>/`	**M-04** modal ochiladi
7	**AR**	`openAR(dishId)`	`GET /api/ar/<dish_id>/`	**M-05** — quyida batafsil
8	+ Savat	`addToCart(dishId, 1)`	— (lokal)	Savat yangilanadi, toast, upsell tekshiriladi
9	Savat	`openCart()`	—	**M-06**
10	Chaqiruv	`openCallMenu()`	—	**M-09**
11	AI	`openAIChat()`	—	**M-13**

Ma'lumot yuklash (birinchi kirish)

```
// static/guest/js/menu.js
async function initMenu() {
  // 1. Menyu ma'lumotlari (bitta so'rov, hammasi)
  const data = await fetch(`/api/menu/${SLUG}/?lang=${LANG}`).then(r=>r.json());
  //    → {categories:[...], dishes:[...], restaurant:{...}, upsell_rules:[...]}
  //    ⚡ Cache: Redis 60s, ETag, brauzer localStorage 5 daq

  // 2. Render (virtual scroll agar 100+ taom)
  renderCategories(data.categories);
  renderDishes(data.dishes);

  // 3. Savatni tiklash
  restoreCart();                // localStorage

  // 4. WebSocket ulanish (buyurtma holati uchun)
  connectWS(`/ws/guest/${SESSION_ID}/`);

  // 5. Rasmlarni lazy-load (IntersectionObserver)
  initLazyImages();

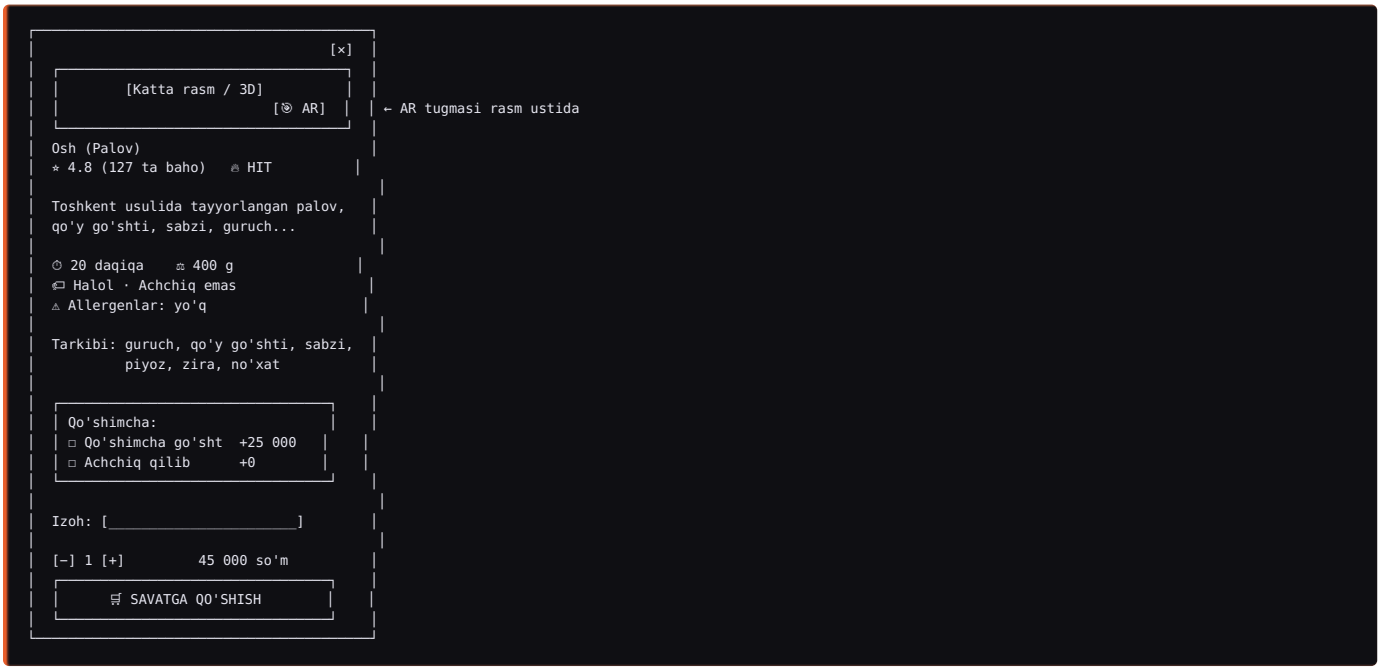
  // 6. 3D poster'larni oldindan yuklash (faqat ko'rinadiganlar)
  prefetchARPosters();
}
```

⚡ Ishlash talablari

Metrika	Maqsad
First Contentful Paint	< 1,2 s (3G)
Time to Interactive	< 2,5 s
Menyu API javob	< 150 ms (p95)
Rasm hajmi	< 60 KB (WebP, 400px)
Umumiy sahifa (birinchi)	< 400 KB

M-04 — TAOM KARTOCHKASI (modal)

URL: `GET /api/menu/dish/<id>/?lang=uz`



Tugmalar

Tugma	Funksiya	Jarayon
🎯 AR	`openAR(dishId)`	→ **M-05**
☐ Qo'shimcha	`toggleModifier(id)`	Narx qayta hisoblanadi (real-time)
− / +	`changeQty(delta)`	`Math.ceil()`, min 1, maks 99
Izoh	—	Maks 200 belgi
🛒 Savatga	`addToCart(payload)`	Savatga qo'shiladi → **upsell tekshiruvi** → toast → modal yopiladi

Upsell mantiq (savatga qo'shilgandan keyin)

```
function checkUpsell(addedDish) {
  // Qoidalar admin panelda sozlanadi (A-02)
  const rules = MENU_DATA.upsell_rules;
  //   [{trigger_dish_ids:[1,2], suggest_dish_id:15, text_uz:"...", ...}]

  const rule = rules.find(r => r.trigger_dish_ids.includes(addedDish.id));
  if (!rule) return;

  // Savatda allaqachon bormi?
  if (cart.some(i => i.dish_id === rule.suggest_dish_id)) return;

  showUpsellToast({
    text: rule[`text_${LANG}`],       // "Oshga achchiq-chuchuk salat?"
    dish: getDish(rule.suggest_dish_id),
    onAccept: () => { addToCart(rule.suggest_dish_id, 1); trackUpsell(rule.id, true); },
    onDismiss: () => trackUpsell(rule.id, false)
  });
}
```

> 📊 **Analitika:** har upsell ko'rsatilishi va qabul qilinishi yoziladi →
> A-10 da "Upsell konversiyasi: 34%" ko'rinadi.

M-05 — 🎯 AR KO'RISH (eng muhim sahifa)

URL: `GET /api/ar/<dish_id>/`
Fayl: `static/guest/js/ar.js`

To'liq jarayon zanjiri

```
[1] Mijoz "👁 AR" tugmasini bosadi
    │
    ▼
[2] openAR(dishId) — static/guest/js/ar.js
    │
    ├─ [2.1] Qurilma imkoniyatini tekshirish
    │         const support = await detectARSupport();
    │         // iOS Safari + ARKit → 'quick-look'
    │         // Android Chrome + ARCore → 'scene-viewer'
    │         // WebXR mavjud → 'webxr'
    │         // hech biri → '3d-only'
    │
    ├─ [2.2] GET /api/ar/<dish_id>/  →
    │         {
    │           "glb":  "https://cdn.../dish_15.glb",
    │           "usdz": "https://cdn.../dish_15.usdz",
    │           "poster":"https://cdn.../dish_15.webp",
    │           "scale_m": 0.32,          // real o'lcham metrda
    │           "name": "Osh (Palov)",
    │           "ready": true
    │         }
    │
    ├─ [2.3] ready=false? → "3D model tayyorlanmoqda" + oddiy rasm
    │
    ├─ [2.4] Modal ochiladi, <model-viewer> render qilinadi:
    │
    │   <model-viewer
    │     src="dish_15.glb"                 ← Android/web
    │     ios-src="dish_15.usdz"            ← iOS
    │     poster="dish_15.webp"
    │     ar ar-modes="webxr scene-viewer quick-look"
    │     ar-scale="fixed"                  ⚡ REAL O'LCHAM (auto EMAS)
    │     ar-placement="floor"
    │     camera-controls                   ⚡ sichqoncha/barmoq bilan
    │     touch-action="pan-y"
    │     shadow-intensity="1"
    │     environment-image="neutral"
    │     loading="eager">
    │       <button slot="ar-button">🪑 Stolga qo'yib ko'rish</button>
    │   </model-viewer>
    │
    ├─ [2.5] Yuklanish kuzatuvi:
    │         'progress' → progress bar
    │         'load'     → ARView yozuvi boshlanadi (timer start)
    │         'error'    → fallback: oddiy rasm + "3D yuklanmadi"
    │
    ▼
[3] Mijoz "🪑 Stolga qo'yib ko'rish" bosadi
    │
    ├─ iOS  → AR Quick Look ochiladi (tizim ilovasi), .usdz
    ├─ And. → Scene Viewer ochiladi, .glb
    └─ WebXR→ Sahifa ichida kamera
    │
    ▼
[4] Kamera ochiladi → yuza (stol/pol) aniqlanadi → taom qo'yiladi
    │  Mijoz: barmoq bilan aylantiradi, yaqinlashtiradi, yuradi
    │  ⚡ ar-scale="fixed" → REAL o'lchamda (lagan Ø32 sm = 32 sm)
    │
    ▼
[5] Mijoz AR'dan chiqadi
    │
    ├─ [5.1] POST /api/ar/view/  {dish_id, session_id, duration_sec, mode}
    │         → ARView modeliga yoziladi (async, Celery/thread emas — Redis queue)
    │
    ├─ [5.2] Modal ichida: [🛒 SAVATGA QO'SHISH] tugmasi ko'rinadi
    │         (⚡ AR ko'rgandan keyin darhol buyurtma imkoniyati — konversiya!)
    │
    ▼
[6] Savatga qo'shsa → OrderItem.ar_viewed = True
    │
    ▼
[7] Analitika: A-10 da "AR ko'rgan mijoz cheki: 68 000 vs ko'rmagan: 52 000 (+31%)"
```

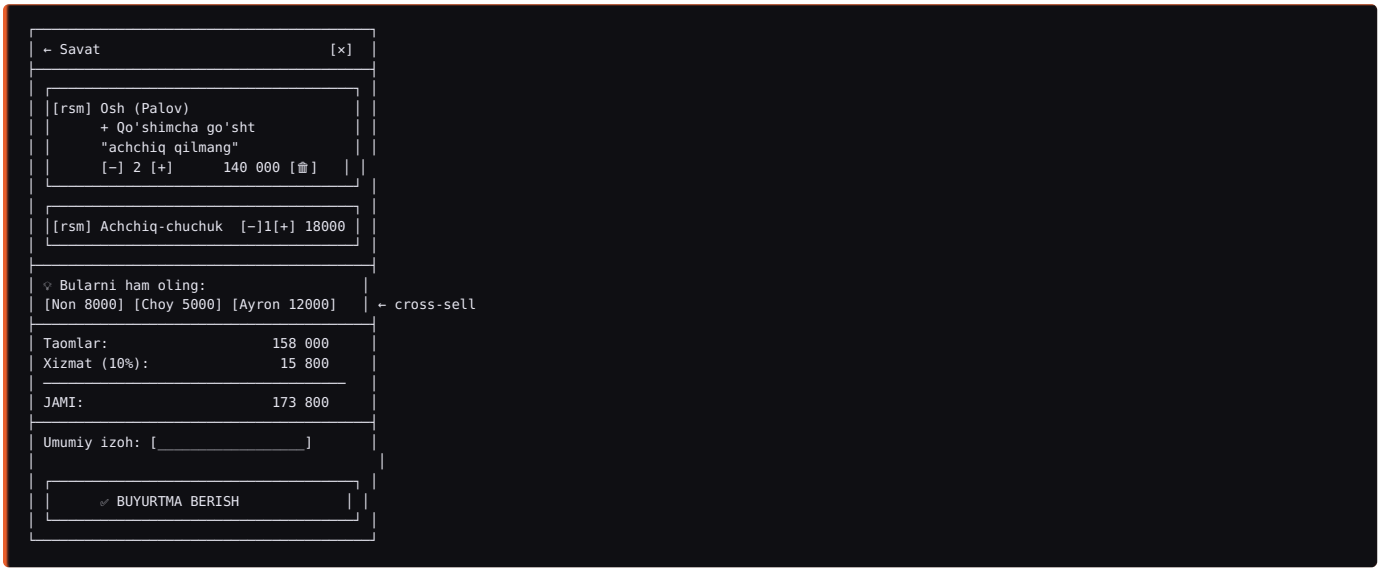
Fallback zanjiri (muhim!)

```
1. To'liq AR (WebXR/QuickLook/SceneViewer)   ← eng yaxshi
   ↓ qo'llab-quvvatlanmasa
2. 3D aylantirish (<model-viewer> AR'siz)     ← yaxshi
   ↓ WebGL yo'q / sekin qurilma
3. 360° rasm karusel (24 kadr)                ← qoniqarli
   ↓ yo'q
4. Oddiy yuqori sifatli rasm                  ← minimal
```

3D fayl talablari (qat'iy)

Parametr	Chegara	Nima uchun
GLB hajmi	**≤ 3 MB**	3G da 8 soniyada yuklanadi
USDZ hajmi	**≤ 5 MB**	iOS Quick Look limiti
Poligon soni	≤ 60 000 tri	Eski telefonlar
Tekstura	≤ 1024×1024, WebP/KTX2	Xotira
Materiallar	≤ 4 ta	Render tezligi
Siqish	**Draco** (glTF), meshopt	Hajm −70%
Real o'lcham	**metrda, aniq**	AR 1:1 chiqishi uchun
Markaz nuqta	Model tagi (Y=0)	Stolga to'g'ri qo'yiladi

M-06 — SAVAT



Tugma	Funksiya	Jarayon
− / +	`updateQty(idx, delta)`	localStorage → jami qayta hisob
🗑	`removeItem(idx)`	Tasdiq → o'chirish
Cross-sell	`addToCart(id)`	Qo'shiladi
✅ Buyurtma	`submitOrder()`	⬇ quyida

`submitOrder()` — to'liq zanjir

```
[1] submitOrder() bosildi
    │
    ├─ [1.1] Frontend validatsiya:
    │         • savat bo'shmi? → "Savat bo'sh"
    │         • miqdorlar butun sonmi? → Math.ceil()
    │         • summa > 0?
    │
    ├─ [1.2] Tugma bloklanadi (double-submit oldini olish)
    │         idempotency_key = uuid4()  ← ⚡ MUHIM
    │
    ▼
[2] POST /api/orders/create/
    Headers: X-Idempotency-Key: <uuid>
    Body: {
      session_id, table_id,
      items: [{dish_id, qty, modifiers:[], note, ar_viewed}],
      note: "umumiy izoh"
    }
    │
    ▼
[3] Backend: apps/orders/views.py::create_order
    │
    ├─ [3.1] Idempotency tekshiruvi (Redis, TTL 24s)
    │         key mavjud? → mavjud order qaytariladi (dublikat yo'q)
    │
    ├─ [3.2] Validatsiya (serializers.py):
    │         • Table faol va shu restoranga tegishlimi?
    │         • Har dish faol va shu restoranmi? (⚡ tenant tekshiruvi)
    │         • Dish.is_available=True? (stop-list)
    │         • Narx SERVERDA qayta hisoblanadi ⚡ (frontendga ishonilmaydi)
    │         • Modifikatorlar shu taomga tegishlimi?
    │
    ├─ [3.3] Transaksiya:
    │         with transaction.atomic():
    │             order = Order.objects.create(...)
    │             OrderItem.objects.bulk_create([...])
    │             order.total = calculated_total
    │             order.number = next_order_number(restaurant)  # kunlik #001
    │             order.save()
    │
    ├─ [3.4] Post-commit hooks (transaction.on_commit):
    │         ├─ WS → rest_<id>_kds:      {"type":"order.new", order:{...}}
    │         ├─ WS → rest_<id>_waiters:  {"type":"order.new", table:5, ...}
    │         ├─ WS → rest_<id>_admin:    {"type":"order.new", ...}
    │         ├─ WS → guest_<session>:    {"type":"order.created", id, number}
    │         ├─ Telegram push (ofitsiantga)      → queue
    │         ├─ Printer (oshxona cheki)          → queue
    │         └─ Analitika event                  → queue
    │
    ▼
[4] Javob: 201 {order_id, number:"#047", eta_min:18, status:"new"}
    │
    ▼
[5] Frontend:
    ├─ Savat tozalanadi (localStorage)
    ├─ Animatsiya: ✅ "Buyurtmangiz qabul qilindi!"
    └─ redirect → M-08 (buyurtma holati)
```

⚠ Kritik xato holatlari

Xato	Kod	Foydalanuvchi ko'radi	Tizim harakati
Taom tugagan	409	"Osh tugadi. Boshqa tanlang."	Savatdan olib tashlanadi
Narx o'zgargan	409	"Narxlar yangilandi. Tekshiring."	Savat yangilanadi
Internet yo'q	—	"Ulanish yo'q. Qayta urinilmoqda..."	3× retry, keyin offline queue
Server xato	500	"Xatolik. Ofitsiantni chaqiring."	Sentry'ga log, tugma qayta faol
Dublikat	200	(jim) mavjud buyurtma ko'rsatiladi	Idempotency ishladi

---

M-08 — BUYURTMA HOLATI (real-time)



WebSocket orqali holat yangilanishi

```
ws.onmessage = (e) => {
  const msg = JSON.parse(e.data);
  switch (msg.type) {
    case 'order.status':          // new → cooking → ready → served
      updateProgressBar(msg.status);
      updateETA(msg.eta_min);
      if (msg.status === 'ready') {
        vibrate([200,100,200]);
        showToast("🎉 Buyurtmangiz tayyor!");
        playSound('ready.mp3');
      }
      break;
    case 'order.item_unavailable': // taom tugab qoldi
      showAlert(`${msg.dish_name} tugadi. Ofitsiant keladi.`);
      break;
    case 'bill.ready':
      redirect('/bill/');
      break;
  }
};

// ⚡ WS uzilsa — avtomatik qayta ulanish (exponential backoff)
// + fallback: har 15 soniyada polling GET /api/orders/<id>/status/
```

M-09 — CHAQIRUV



Zanjir:

```
Tugma → POST /api/calls/ {table_id, type}
      → Call.objects.create(...)
      → WS → rest_<id>_waiters: {"type":"call.new", table:5, call_type:"water"}
      → Ofitsiant telefonida: 🔔 tovush + vibratsiya + push
      → Mijozga: "Ofitsiant chaqirildi ✓" (30s cooldown, spam oldini olish)
      → Ofitsiant "Bajardim" bossa → Call.resolved=True → mijozga bildirishnoma
```

⚡ **Cooldown:** bir stol 60 soniyada bir marta chaqira oladi (Redis rate-limit).

M-10 — HISOB VA TO'LOV



To'lov zanjiri (Payme misolida)

```
[1] "TO'LASH" bosildi
    ▼
[2] POST /api/payments/create/ {order_id, provider:"payme", use_bonus:false}
    ▼
[3] Backend (apps/payments/services.py):
    ├─ Payment.objects.create(status='pending', provider='payme')
    ├─ paytechuz orqali invoice yaratish
    └─ javob: {payment_url: "https://checkout.paycom.uz/..."}
    ▼
[4] Frontend: window.location = payment_url
    ▼
[5] Mijoz Payme ilovasida to'laydi
    ▼
[6] Payme → bizning webhook: POST /api/payments/webhook/payme/
    ├─ Imzo tekshiriladi (⚡ SHART)
    ├─ Idempotency (transaction_id)
    ├─ Payment.status = 'paid'
    ├─ Order.status = 'paid'
    ├─ 🧾 FISKAL CHEK: OFD'ga yuborish (async, retry bilan)
    ├─ 🎁 Bonus hisoblash: Guest.bonus += total × cashback%
    ├─ WS → rest_<id>_cashier, _waiters, _admin
    └─ WS → guest_<session>: {"type":"payment.success"}
    ▼
[7] Mijoz qaytadi → "✅ To'lov qabul qilindi"
    ├─ Fiskal chek ko'rsatiladi (QR + raqam)
    ├─ Telegram'ga chek yuborish taklifi
    ▼
[8] → M-11 (choychaqa) → M-12 (otziv)
```

⚠ To'lov xavfsizligi (qat'iy)

1. **Summa faqat serverda hisoblanadi** — frontenddan kelgan summa **ishonilmaydi**
2. **Webhook imzosi tekshiriladi** — har provayder uchun
3. **Idempotency** — bitta to'lov 2 marta hisoblanmaydi
4. **Timeout** — 15 daqiqada to'lanmasa `expired`
5. **Reconciliation** — kuniga 1 marta provayder bilan solishtirish (cron)
6. **PCI DSS** — karta ma'lumotlari **bizda saqlanmaydi** (faqat provayder)

M-11 — CHOYCHAQA (TIPS) ⚡



> ⚡ **Bu funksiya strategik muhim:** ofitsiantlar qo'shimcha daromad oladi → tizimni **o'zlari targ'ib qiladi** → xodim qarshiligi yo'qoladi (SaaS #1 o'ldiruvchi).

Zanjir: `POST /api/tips/` → `Tip(order, staff, amount)` → to'lov → `Staff.tips_balance += amount` → ofitsiantga push: "*Dilshod, sizga 17 480 so'm choychaqa! 🎉*"

M-12 — OTZIV VA REYTING ⭐



⭐ Otziv filtri mantiq (juda muhim sotuv argumenti)



> 💰 **Sotuv argumenti:** "*Google reytingingiz ko'tariladi, salbiy sharhlar internetga chiqmasdan sizga keladi.*" — restoran egalari buni juda qadrlaydi.

M-13 — 🤖 AI CHATBOT (yordamchi)



AI chatbot arxitekturasi (batafsil — Qism VII)

```
Savol
  |
  ├─ [1] Rate limit (10 savol / 5 daqiqa / mijoz)
  |
  ├─ [2] Kontekst tayyorlash:
  |       • Restoran menyusi (kesilgan: nom, narx, tarkib, teglar)
  |       • Til (uz/ru/en)
  |       • Savat holati
  |       • Cheklov: "faqat menyu haqida javob ber"
  |
  ├─ [3] MCP orqali AI chaqiruvi (Qism VII: 6+ provayder)
  |       Prioritet: arzon → qimmat
  |       1. Gemini Flash    (eng arzon)
  |       2. Haiku 4.5
  |       3. GPT-4o-mini
  |       4. Groq Llama      (tez, arzon)
  |       5. DeepSeek
  |       6. Sonnet          (murakkab savol)
  |       ⚡ Biri ishlamasa → keyingisiga o'tadi (fallback zanjiri)
  |
  ├─ [4] Javob validatsiyasi:
  |       • Menyuda yo'q taom taklif qilmasin
  |       • Narx to'g'rimi (menyudan tekshiriladi)
  |       • Zararli kontent yo'qmi
  |
  ├─ [5] Javobga tugmalar biriktiriladi ([AR], [Buyurtma])
  |
  └─ [6] Xarajat yoziladi: AIUsage(restaurant, provider, tokens, cost)
          → P-05 da ko'rinadi
```

---

2. 👨‍🍳 OFITSIANT SAHIFALARI

O-01 — KIRISH (PIN)

```
┌──────────────────────────────────┐
│        [Restoran logo]           │
│        Xodim kirishi             │
│                                  │
│      ┌──┐ ┌──┐ ┌──┐ ┌──┐        │
│      │● │ │● │ │● │ │  │        │
│      └──┘ └──┘ └──┘ └──┘        │
│                                  │
│        [1] [2] [3]               │
│        [4] [5] [6]               │
│        [7] [8] [9]               │
│        [⌫] [0] [✓]               │
└──────────────────────────────────┘
```

Zanjir: 4 raqamli PIN → `POST /api/staff/login/` → JWT (24s) → `localStorage.staff_token` → rolra qarab redirect (waiter→O-02, cook→K-01, cashier→S-01)

⚠ **Xavfsizlik:** 5 xato → 15 daqiqa blok (Redis); PIN bcrypt bilan hash.

O-02 — BOSH EKRAN (yangi buyurtmalar)

```
┌──────────────────────────────────┐
│ 👨‍🍳 Dilshod          [Smena: 4s 20d]  │
│ Bugun: 23 stol · 4,2 mln · 💎 87 000  │
├──────────────────────────────────┤
│ 🔔 3 ta yangi                    │
├──────────────────────────────────┤
│ ┌──────────────────────────────┐ │
│ │ 🪑 #047      🕐 STOL 5    2 daq  │ │ ← katta stol raqami
│ │ Osh × 2, Achchiq-chuchuk × 1     │ │
│ │ 💬 "achchiq qilmang"             │ │
│ │ 173 800 so'm                     │ │
│ │ [✓ QABUL QILDIM]                 │ │
│ └──────────────────────────────┘ │
│ ┌──────────────────────────────┐ │
│ │ 👨‍🍳 #046      🕐 STOL 12   8 daq  │ │
│ │ Tayyorlanmoqda...                │ │
│ └──────────────────────────────┘ │
│ ┌──────────────────────────────┐ │
│ │ ✓ #045      🕐 STOL 3    TAYYOR!  │ │ ← yashil, pulsatsiya
│ │ [🍽 OLIB BORDIM]                 │ │
│ └──────────────────────────────┘ │
├──────────────────────────────────┤
│ [🛍 Buyurtma] [🗺 Zal] [🔔2] [📊]      │
└──────────────────────────────────┘
```

Tugmalar zanjiri

Tugma	Endpoint	Jarayon
✅ QABUL QILDIM	`POST /api/orders/<id>/accept/`	status= `accepted` , waiter biriktiriladi → WS: KDS'ga "tayyorlashni boshla", mijozga "qabul qilindi"
🍽 OLIB BORDIM	`POST /api/orders/<id>/serve/`	status= `served` → mijozga "Yoqimli ishtaha!" → stol holati `dining`
🗺 Zal	—	**O-05**
🔔	—	**O-06** chaqiruvlar
📊	—	**O-09** statistika

⚡ Real-time bildirishnoma

```
ws.onmessage = (e) => {
  const m = JSON.parse(e.data);
  if (m.type === 'order.new') {
    // ⚡ Faqat MENING stolim bo'lsa
    if (!MY_TABLES.includes(m.table_id) && m.assigned_to !== MY_ID) return;
    playSound('new_order.mp3');
    vibrate([300,100,300]);
    prependOrderCard(m.order);
    updateBadge();
    if (document.hidden) showNotification(`Stol ${m.table_no} — yangi buyurtma`);
  }
  if (m.type === 'order.ready') { /* yashil pulsatsiya + tovush */ }
  if (m.type === 'call.new')    { /* chaqiruv */ }
};
```

O-05 — 2D ZAL PLANI

```
┌──────────────────────────────────────┐
│ 🗺 Zal — 1-qavat        [1▾]         │
├──────────────────────────────────────┤
│                                      │
│    ┌──┐      ┌──┐      ┌──┐         │
│    │ 1 │     │ 2 │     │ 3 │        │
│    │🟢 │     │🟢 │     │🟢 │        │
│    └──┘      └──┘      └──┘         │
│                                      │
│    ┌──┐    ╔═══╗      ┌──┐         │
│    │ 4 │   ║ 5 ║      │ 6 │        │
│    │🟢 │   ║🟡 ║      │🟢 │        │
│    └──┘    ╚═══╝      └──┘         │
│           ↑ TAYYOR!                 │
│         (pulsatsiya)                 │
│                                      │
│   ┌──────────── BAR ──────────┐      │
│                                      │
├──────────────────────────────────────┤
│ 🟢 Bo'sh  🟡 Band  🔴 Kutmoqda       │
│ 🟣 Tayyor 🔵 Hisob                   │
└──────────────────────────────────────┘
```

Stol rangi mantiq:

```
function tableColor(t) {
  if (t.has_ready_order)   return 'red';     // 🟣 tayyor — pulsatsiya
  if (t.has_pending_call)  return 'purple';  // 🔵 chaqiruv
  if (t.has_active_order)  return 'yellow';  // 🟡 kutmoqda
  if (t.is_occupied)       return 'blue';    // 🟡 band
  return 'green';                            // 🟢 bo'sh
}
```

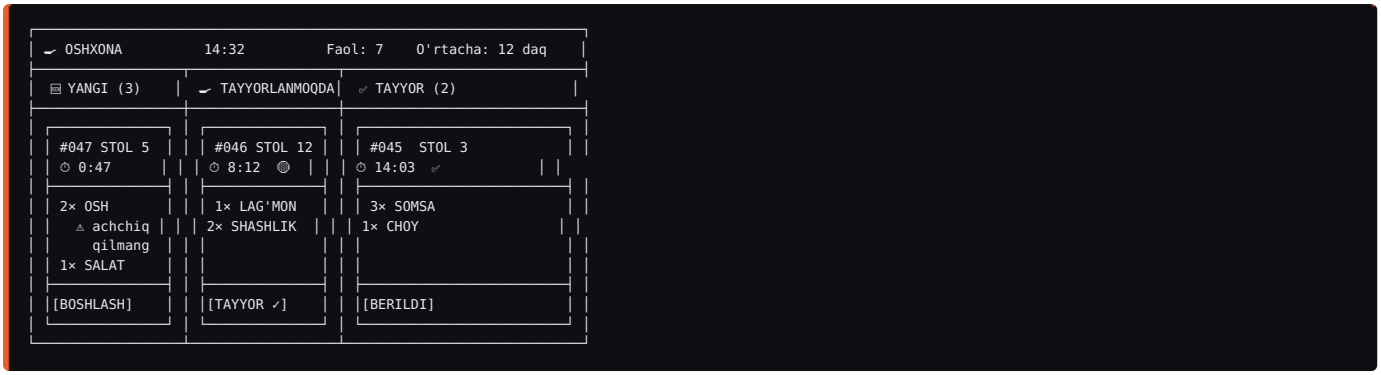
Stol bosilsa: → **O-03** buyurtma tafsiloti / yangi buyurtma kiritish

> 📌 **v3 da:** shu 2D plan → 2.5D izometrik → to'liq 3D + navigatsiya.
> Ma'lumot modeli (x, y, shakl) bosh kundan 3D uchun tayyor.

3. 📺 KDS — OSHXONA MONITORI

K-01 — BUYURTMALAR TAXTASI

URL: `/kds/?token=<kds_token>` (TV brauzerida, doim ochiq)
Uslub: dark, katta shrift, 3 metrdan o'qiladigan (`ai agent.mp4` uslubi)



Vaqt ranglari (oshpaz uchun bosim)

Vaqt	Rang	Ma'no
0-50% ETA	🟢 yashil	Normal
50-80%	🟡 sariq	Shoshiling
80-100%	🟠 to'q sariq	Kechikmoqda
>100%	🔴 qizil + tovush	**KECHIKDI**

Tugmalar

Tugma	Endpoint	Natija
BOSHLASH	`POST /api/orders/<id>/cook/`	`cooking`, taymer start → mijozga "Tayyorlanmoqda"
TAYYOR ✓	`POST /api/orders/<id>/ready/`	`ready` → 🔔 ofitsiantga tovush+push → mijozga "Tayyor!"
BERILDI	`POST /api/orders/<id>/serve/`	`served`
Taom bosilsa	`POST /api/dishes/<id>/stop/`	Stop-list: menyudan yashiriladi (tugagan)

⚡ **KDS hech qachon yopilmaydi:** WS uzilsa avtomatik ulanadi, offline'da lokal holat ko'rsatiladi, sahifa har 6 soatda avtomatik yangilanadi (xotira uchun).

---

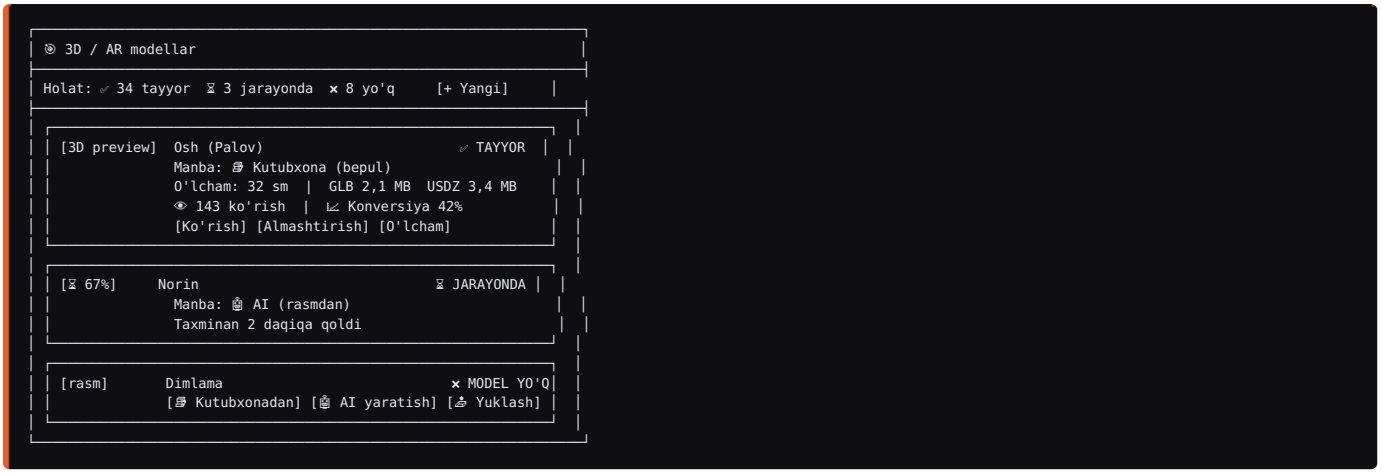
4. 👨‍💼 ADMIN SAHIFALARI

A-01 — DASHBOARD (real-time)



⚡ **Real-time:** WS `rest_<id>_admin` — barcha hodisalar jonli.

A-04 — 🎯 3D MODEL BOSHQARUVI (eng muhim admin sahifa)



"AI yaratish" tugmasi — to'liq zanjir

```
[1] Admin "AI yaratish" bosadi
    ▼
[2] Modal: "Taom rasmini yuklang (3-6 ta turli burchakdan yaxshiroq)"
    [rasm yuklash] + o'lcham kiritish: [32] sm
    ▼
[3] POST /api/ar/generate/ {dish_id, images:[...], real_size_cm:32}
    ▼
[4] Backend:
    ├─ [4.1] Tarif tekshiruvi: PRO/PRO+FISKAL bo'lishi kerak
    │         START/FREE → 402 "Tarifni oshiring"
    ├─ [4.2] Limit: PRO = 50 model; oshgan → to'lov taklifi
    ├─ [4.3] Model3DJob.objects.create(status='queued')
    ├─ [4.4] Redis navbatga qo'yish: queue:model3d
    └─ [4.5] javob: 202 {job_id}
    ▼
[5] ⚙ WORKER (armenu-worker.service) navbatdan oladi
    │
    ├─ [5.1] KUTUBXONA QIDIRUVI (birinchi navbatda!)
    │         • Nom bo'yicha: "norin" → DishLibrary
    │         • AI embedding bo'yicha (semantik)
    │         • Rasm bo'yicha (CLIP similarity)
    │         TOPILDI? → nusxa olinadi, 0 xarajat, 5 soniya ✓ TUGADI
    │
    ├─ [5.2] Topilmadi → AI GENERATSIYA
    │         MCP orqali (Qism VII):
    │         1. Meshy API      (image-to-3d)   ~$0.5
    │         2. ishlamasa → Tripo API         ~$0.8
    │         3. ishlamasa → Rodin API         ~$1.5
    │         → natija: raw .glb
    │
    ├─ [5.3] OPTIMIZATSIYA (gltf-transform)
    │         • Draco siqish
    │         • Tekstura → 1024px WebP
    │         • Poligon → ≤60k (simplify)
    │         • O'lcham → real_size_cm ga masshtablash ⚡
    │         • Markaz → Y=0 (model tagi)
    │         • Natija tekshiruvi: ≤3 MB?
    │
    ├─ [5.4] USDZ konvertatsiya (usd-from-gltf)
    │         → dish_15.usdz (≤5 MB)
    │
    ├─ [5.5] Poster render (headless, 800×800 WebP)
    │
    ├─ [5.6] CDN'ga yuklash (S3/MinIO + Cloudflare)
    │
    ├─ [5.7] DishModel3D saqlanadi, status='ready'
    │
    ├─ [5.8] KUTUBXONAGA QO'SHISH (agar umumiy taom bo'lsa) ⚡
    │         DishLibrary.objects.create(key='norin', ...)
    │         → keyingi restoranlar uchun BEPUL
    │
    └─ [5.9] WS → admin: {"type":"model3d.ready", dish_id:15}
              + Telegram: "✓ Norin 3D modeli tayyor"
    ▼
[6] Admin panelda ✓ TAYYOR ko'rinadi, mijozlar AR'da ko'ra oladi
```

⚠ Xato holatlari

Xato	Harakat
AI provayder ishlamadi	Keyingi provayderga (3 ta zanjir)
Hammasi ishlamadi	`status='failed'`, admin xabardor, qo'lda yuklash taklifi
Model 3 MB dan katta	Qayta optimizatsiya (agressiv), baribir katta → sifat pasaytiriladi
Rasm sifatsiz	"Yaxshiroq rasm yuklang" + tavsiyalar
Limit tugagan	To'lov taklifi

A-05 — STOLLAR VA QR

```
┌──────────────────────────────────────────────────────────────┐
│ 🪑 Stollar va QR kodlar          [+ Stol]  [🖨 Barcha QR — PDF]  │
├──────────────────────────────────────────────────────────────┤
│ Zal: [1-qavat ▾]                                             │
├──────┬──────────┬─────────┬──────────┬──────────┬────────────┤
│ №    │ Zal      │ Sig'im  │ Holat    │ QR       │ Amallar    │
├──────┼──────────┼─────────┼──────────┼──────────┼────────────┤
│ 1    │ 1-qavat  │ 4       │ 🟢 Bo'sh │ [▪▫▪] a3f2..│ [PDF][♻][✏] │
│ 2    │ 1-qavat  │ 2       │ 🔴 Band  │ [▪▫▪] 7b1e..│ [PDF][♻][✏] │
│ 5    │ 1-qavat  │ 6       │ 🟡 Tayyor│ [▪▫▪] e9c0..│ [PDF][♻][✏] │
└──────┴──────────┴─────────┴──────────┴──────────┴────────────┘
```

QR generatsiya zanjiri

```
[+ Stol] → POST /api/tables/
   ├─ qr_token = uuid4()          ⚡ HAR STOL NOYOB
   ├─ url = f"https://{domain}/t/{qr_token}"
   ├─ QR rasm generatsiya (qrcode kutubxonasi, ECC=H, logo markazda)
   ├─ PNG (300 DPI) + SVG saqlanadi
   └─ Table saqlanadi

[🖨 PDF] → GET /api/tables/qr-pdf/?hall=1&layout=a4_6
   ├─ ReportLab/WeasyPrint bilan PDF
   ├─ Har stol uchun: logo + STOL №(katta) + QR + 3 tilda matn
   ├─ Layout: A4 da 1/2/4/6 ta, yoki stol standi (10×15 sm)
   └─ Yuklab olinadi: qr_codes_1-qavat.pdf

[♻] → POST /api/tables/<id>/regenerate-qr/
   ├─ ⚠ Tasdiq: "Eski QR ishlamay qoladi!"
   ├─ qr_token = uuid4() (yangi)
   └─ Eski token 24 soat "grace period" ishlaydi (mijoz ovqatlanayotgan bo'lishi mumkin)
```

A-10 — AR ANALITIKA (sotuv argumentining asosi)

```
┌──────────────────────────────────────────────────────────────┐
│ 👁 AR analitika                    [Bu oy ▾]                  │
├──────────────────────────────────────────────────────────────┤
│ ┌──────────────┐ ┌──────────────┐ ┌──────────────┐           │
│ │ AR KO'RISH   │ │ KONVERSIYA   │ │ CHEK TA'SIRI │           │
│ │   2 847      │ │   38,4%      │ │  +31%        │           │
│ │  ▲ +34%      │ │  ▲ +5,2pp    │ │ ⚡ ASOSIY    │           │
│ └──────────────┘ └──────────────┘ └──────────────┘           │
├──────────────────────────────────────────────────────────────┤
│ 💰 AR KO'RGAN vs KO'RMAGAN MIJOZ                             │
│                                                              │
│   AR ko'rgan:     ████████████████ 68 400 so'm               │
│   AR ko'rmagan:   ██████████ 52 100 so'm                     │
│                                  ⚡ +31,3% farq              │
│   → Bu oy AR qo'shimcha keltirdi:  +14 200 000 so'm          │
│   → Tarif to'lovi: 590 000 so'm  →  ROI = 24×                │
├──────────────────────────────────────────────────────────────┤
│ TOP AR taomlar                                               │
│ 1. Osh        487 ko'rish  →  42% buyurtma                   │
│ 2. Shashlik   356 ko'rish  →  51% buyurtma  ★ eng yaxshi     │
│ 3. Norin      289 ko'rish  →  28% buyurtma                   │
├──────────────────────────────────────────────────────────────┤
│ ⚠ TAVSIYA                                                    │
│ • "Dimlama" 3D modeli yo'q, lekin 89 marta ochilgan → yarating │
│ • "Lag'mon" AR konversiyasi 12% — model sifati past bo'lishi mumkin│
└──────────────────────────────────────────────────────────────┘
```

> 💰 Bu sahifa — **sotuvchining eng kuchli quroli**. Restoran egasi
> "AR menga 24× qaytim beryapti" ko'rsa, hech qachon ketmaydi (churn ↓).

A-15 — 🌐 TARJIMA BOSHQARUVI (3 til talab)

```
┌──────────────────────────────────────────────────────────────┐
│ 🌐 Tarjimalar          [🇺🇿 O'zbek] [🇷🇺 Rus] [🇬🇧 Ingliz]      │
├──────────────────────────────────────────────────────────────┤
│ Holat: 🇺🇿 100%   🇷🇺 87%   🇬🇧 62%    [🤖 AI bilan to'ldirish]  │
├──────────────┬──────────────────┬────────────────────────────┤
│ 🇺🇿 O'zbek    │ 🇷🇺 Rus           │ 🇬🇧 Ingliz                  │
├──────────────┼──────────────────┼────────────────────────────┤
│ Osh (Palov)  │ Плов             │ Pilaf                   ✍  │
│ Toshkent oshi.. │ Ташкентский пл.. │ Tashkent-style pilaf...  ✍  │
├──────────────┼──────────────────┼────────────────────────────┤
│ Norin        │ Норин            │ [🤖 tarjima qilish]     ⚠  │
├──────────────┼──────────────────┼────────────────────────────┤
│ Dimlama      │ [🤖]          ⚠  │ [🤖]                    ⚠  │
└──────────────┴──────────────────┴────────────────────────────┘
```

Tarjima tizimi (har bir harf/matn tarjima qilinadi — talab)

3 qatlam:

```
1. INTERFEYS matnlari (statik)
   → Django i18n: locale/uz/LC_MESSAGES/django.po
   → JS: static/i18n/{uz,ru,en}.json
   → 100% qo'lda tarjima qilingan (sifat kafolati)

2. KONTENT matnlari (dinamik — taom nomi, tavsif)
   → DB da 3 ta ustun: name_uz, name_ru, name_en
   → Admin qo'lda kiritadi YOKI 🤖 AI tarjima qiladi
   → AI tarjima "tasdiqlanmagan" belgisi bilan (admin tasdiqlaydi)

3. AI GENERATSIYA (chatbot javoblari)
   → Foydalanuvchi tilida generatsiya qilinadi (tarjima emas)
```

🤖 AI bilan to'ldirish zanjiri:

```
[🤖 AI bilan to'ldirish] bosildi
   → POST /api/i18n/auto-translate/ {target_lang:'en'}
   → Worker: bo'sh maydonlarni yig'adi (batch 50 ta)
   → MCP AI: "Quyidagi o'zbek taom nomlarini ingliz tiliga tarjima qil.
              Milliy taomlar uchun transliteratsiya + qisqa tushuntirish ber."
   → Natija: {name_en:"Norin", desc_en:"Traditional hand-cut noodles with horse meat"}
   → is_auto_translated=True bilan saqlanadi
   → Admin panelda ⚠ belgisi (tasdiqlash uchun)
   → Admin tasdiqlasa → ✍
```

---

5. 🔧 SUPER-ADMIN (platforma egasi)

P-02 — 3D KUTUBXONA BOSHQARUVI ⚡ (asosiy aktiv)

```
┌──────────────────────────────────────────────────────────────┐
│ 🏛 Umumiy 3D kutubxona      312 model   [+ Qo'shish] [📊 Hisobot]│
├──────────────────────────────────────────────────────────────┤
│ Qidiruv: [osh_______]  Kategoriya: [Milliy ▾]                │
├──────────┬──────┬────────┬──────────┬──────────────────────┤
│ Model    │ Nom  │ O'lcham│ Ishlatildi│ Sifat                │
├──────────┼──────┼────────┼──────────┼──────────────────────┤
│ [3D]     │ Osh  │ 32 sm  │ 87 rest. │ ★★★★★ foto           │
│ [3D]     │ Manti│ 24 sm  │ 76 rest. │ ★★★★ AI              │
│ [3D]     │ Somsa│ 12 sm  │ 71 rest. │ ★★★★★ foto           │
│ [3D]     │ Norin│ 28 sm  │ 43 rest. │ ★★★ AI               │
└──────────┴──────┴────────┴──────────┴──────────────────────┘

💡 TAVSIYA: "Chuchvara" 34 marta so'ralgan lekin kutubxonada yo'q → yarating
```

Kutubxona qiymati hisoblagichi:

```
312 model × o'rtacha 70 000 so'm ishlab chiqarish narxi = 21,8 mln so'm aktiv
Har yangi restoran o'rtacha 38 model kutubxonadan oladi = 2,66 mln so'm tejaydi
```

P-04 — TIZIM SALOMATLIGI (health)

```
┌──────────────────────────────────────────────────────┐
│ ❤ Tizim salomatligi                  ◎ SOG'LOM       │
├──────────────┬──────────────┬──────────────────────────┤
│ Web (8060)   ◎ 4/4 instans   CPU 34%  RAM 2,1/8 GB  p95 87ms  │
│ WS (8061)    ◎ 2/2 instans   3 421 ulanish                    │
│ Worker       ◎ 3/3           Navbat: 12   O'rtacha 4,2 s      │
│ PostgreSQL   ◎ Primary+2 replica  Conn 45/200  Lag 0,3s       │
│ Redis        ◎ Cluster 3 node     Xotira 1,2/4 GB             │
│ CDN          ◎ Hit rate 94,2%                                 │
│ AI providers ◎ Meshy sekin (12s)  Boshqalar normal            │
│ OFD          ◎ Navbat 0   Oxirgi 2s oldin                     │
├──────────────┴──────────────┴──────────────────────────┤
│ ⚠ OGOHLANTIRISH                                       │
│ • Meshy API javob vaqti 12s (normal 4s) → Tripo'ga o'tildi    │
│ • Restoran #47 — 3 kun faol emas → churn xavfi                │
└──────────────────────────────────────────────────────┘
```

QISM III

Tizim arxitekturasi

> Bu qism tizim arxitektori uchun. Million+ foydalanuvchi, xatolarga chidamlilik, o'z-o'zini tiklash, 5 yillik chidamlilik.

1. ARXITEKTURA PRINTSIPLARI (o'zgarmas qoidalar)

#	Printsip	Nima uchun	Amalda
1	**Stateless application**	Gorizontal masshtablash	Session → Redis, fayl → S3, hech narsa diskda emas
2	**Shared-nothing**	Bir node yiqilsa boshqasi ishlaydi	Har instans mustaqil
3	**Graceful degradation**	Qism yiqilsa tizim to'liq o'lmaydi	Har xizmat uchun fallback
4	**Idempotency**	Qayta urinish xavfsiz	Har yozuv operatsiyasida key
5	**Event-driven**	Bo'shashgan bog'lanish	Domain event → handlers
6	**CQRS-lite**	O'qish/yozish alohida masshtablanadi	Read replica, materialized view
7	**Cache-first**	DB yuklamasi kamayadi	Redis → DB, TTL + invalidation
8	**Circuit breaker**	Yiqilgan xizmat butun tizimni tortmaydi	Har tashqi API uchun
9	**Bulkhead**	Bir modul boshqasini bo'g'maydi	Alohida pool/queue
10	**Observability**	Nima bo'layotgani ko'rinadi	Metric + log + trace

2. UMUMIY ARXITEKTURA (to'liq)



3. 📦 MODULLAR (Django app'lar)

```
armenu_project/
├── config/                  # settings, urls, asgi, wsgi, celery
├── core/                    # umumiy: base model, mixin, util, exception
├── apps/
│   ├── tenants/             # Restaurant, tarif, obuna, izolyatsiya
│   ├── accounts/            # User, Staff, rol, auth, JWT
│   ├── menu/                # Category, Dish, Modifier, stop-list
│   ├── ar3d/          ⚡     # DishModel3D, DishLibrary, pipeline, ARView
│   ├── tables/              # Hall, Table, QR, zal plani
│   ├── guest/               # Guest, GuestSession, mijoz view'lari
│   ├── orders/        ⚡     # Order, OrderItem, holat mashinasi, KDS
│   ├── payments/            # Payment, Tip, provayder adapterlar
│   ├── fiscal/              # OFD, IKPU, chek
│   ├── loyalty/             # Bonus, keshbek, promo
│   ├── reviews/             # Review, Google yo'naltirish
│   ├── analytics/           # Event, hisobot, menyu-injiniring
│   ├── ai/            ⚡     # MCP qatlami, chatbot, tarjima
│   ├── notifications/       # Telegram, push, SMS, email
│   ├── delivery/     (v2)   # Kuryer, zona
│   ├── inventory/    (v2)   # Ombor, retsept
│   └── platform_admin/      # Super-admin
├── static/
├── templates/
├── locale/                  # uz, ru, en
└── deploy/                  # systemd, nginx, docker, ansible
```

Modullar bog'liqligi (muhim — sikl bo'lmasin)

```
core ← hamma
tenants ← hamma (restaurant_id)
accounts ← orders, tables, payments
menu ← ar3d, orders, ai
tables ← orders, guest
guest ← orders, loyalty, reviews
orders ← payments, fiscal, analytics, notifications
ai ← menu, ar3d (MCP qatlami mustaqil)

⚠ QOIDA: yuqori qatlam pastga bog'liq bo'lishi mumkin, teskarisi YO'Q.
   Kerak bo'lsa — signal/event orqali (bo'shashgan bog'lanish).
```

---

4. 🚀 MASHTABLASH — MILLION FOYDALANUVCHI

4.1 Yuklama bosqichlari va arxitektura

Bosqich	Foydalanuvchi	Restoran	RPS	Arxitektura
S0 — Start	< 1 000/kun	1-20	~5	1 VPS (4 CPU, 8 GB): web+ws+db+redis
S1 — O'sish	10 000/kun	20-100	~50	2 VPS: (web+ws) + (db+redis)
S2 — Viloyat	100 000/kun	100-400	~300	4 VPS: 2 web, 1 ws, 1 db+replica
S3 — Respublika	500 000/kun	400-1500	~1 500	8-12 node, LB, Redis cluster, read replica
S4 — Region	**2 000 000/kun**	1500-5000	~6 000	20+ node, k8s, sharding, multi-region CDN
S5 — Pik	**1 mln bir vaqtda**	—	~25 000	Quyida batafsil ⬇

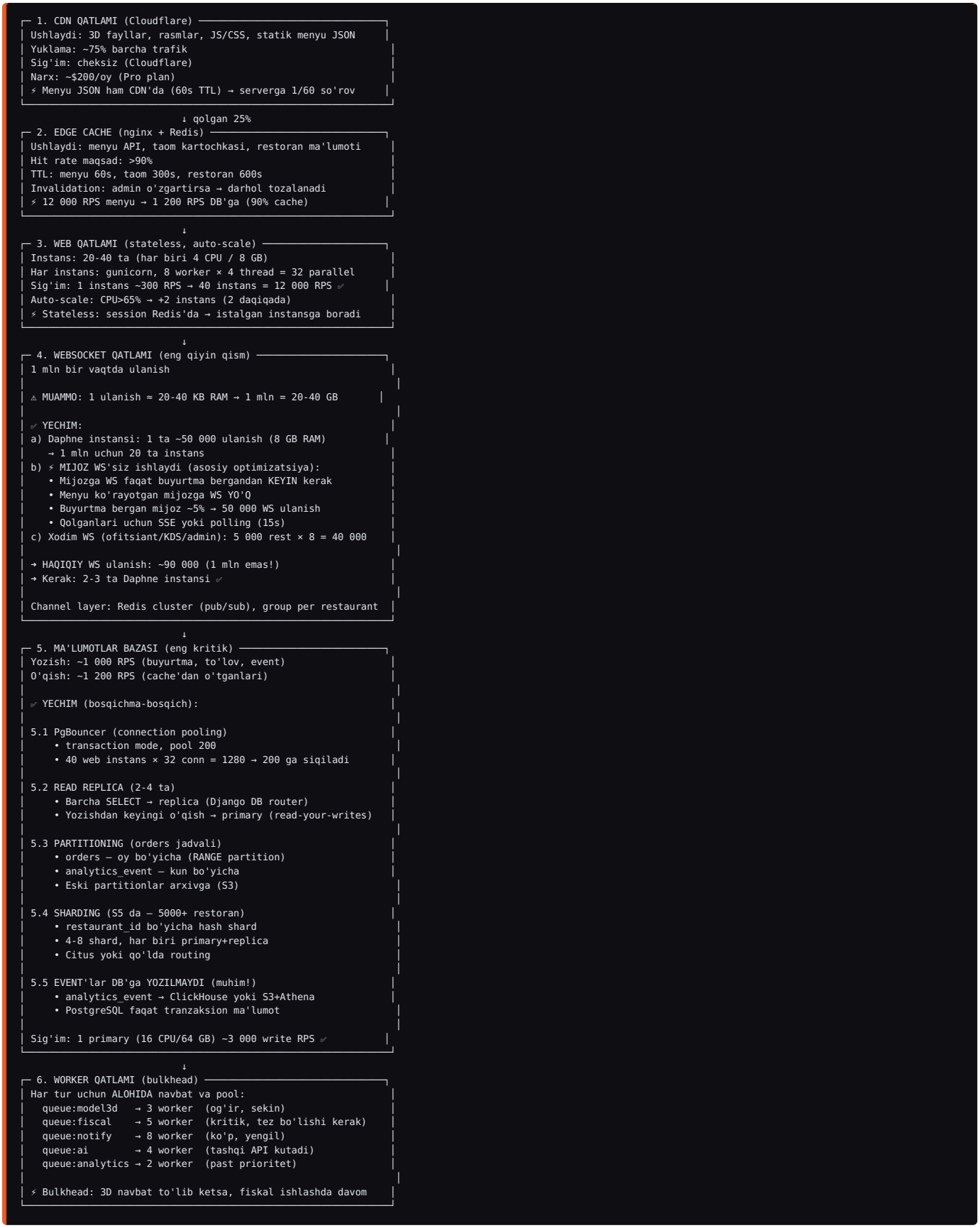
4.2 ⚡ 1 MILLION BIR VAQTDA — nima bo'ladi va nima qilinadi

Realistik ssenariy: 1 mln bir vaqtda faol foydalanuvchi ≈ 5 000 restoran × 200 stol.
Bu O'zbekiston uchun **butun bozorning 20%i** — ya'ni 4-5 yillik maqsad.

Yuklama profili (1 mln bir vaqtda)

Operatsiya	Ulush	RPS	Xususiyat
Menyu ko'rish	60%	~12 000	🟢 **Cache** — DB'ga bormaydi
3D/AR fayl yuklash	15%	~3 000	🟢 **CDN** — serverga bormaydi
WebSocket ulanish	10%	1 mln ulanish	🟡 WS qatlami
Buyurtma yaratish	5%	~1 000	🔴 **DB yozish** — eng og'ir
Boshqa API	10%	~2 000	🟡 Aralash

Har qatlam uchun yechim



📊 1 mln bir vaqtda — infratuzilma va narx

Komponent	Miqdor	Konfiguratsiya	Oylik narx
Web nodes	40	4 CPU / 8 GB	$1 600
WS nodes	3	8 CPU / 16 GB	$240
Worker nodes	6	4 CPU / 8 GB	$240
PostgreSQL primary	1	16 CPU / 64 GB / NVMe	$400
PostgreSQL replica	3	8 CPU / 32 GB	$600
Redis cluster	6	4 CPU / 16 GB	$480
Load balancer	2	4 CPU / 8 GB	$80
ClickHouse (analitika)	2	8 CPU / 32 GB	$400
S3/MinIO (3D fayl)	5 TB	—	$120
CDN (Cloudflare)	—	Pro/Business	$200
Monitoring	1	4 CPU / 8 GB	$40
JAMI			**~$4 400/oy**

Daromadga nisbatan: 5 000 restoran × 590k so'm ≈ 2,95 mlrd so'm/oy (~$233 000).
Infra = **1,9%** daromaddan. ✅ Sog'lom.

4.3 Masshtablash bo'yicha qat'iy kod qoidalari

```
# ❌ NOTO'G'RI — N+1 query, million foydalanuvchida o'ldiradi
for order in Order.objects.all():
    print(order.table.number)          # har safar DB so'rov

# ✅ TO'G'RI
for order in Order.objects.select_related('table', 'guest').all():
    print(order.table.number)

# ❌ NOTO'G'RI — butun jadval xotiraga
dishes = list(Dish.objects.all())

# ✅ TO'G'RI — iterator + chunk
for dish in Dish.objects.iterator(chunk_size=500): ...

# ❌ NOTO'G'RI — sinxron tashqi API web so'rovda
def create_order(request):
    order = Order.objects.create(...)
    requests.post(OFD_URL, ...)        # 3 soniya kutadi! ❌

# ✅ TO'G'RI — navbatga
def create_order(request):
    order = Order.objects.create(...)
    transaction.on_commit(lambda: enqueue('fiscal', order.id))
    return Response(...)               # darhol qaytadi ✅

# ❌ NOTO'G'RI — cache'siz issiq yo'l
def menu_api(request, slug):
    return Response(build_menu(slug))  # har so'rovda DB ❌

# ✅ TO'G'RI
def menu_api(request, slug):
    key = f"menu:{slug}:{lang}:{version}"
    data = cache.get(key)
    if data is None:
        data = build_menu(slug)
        cache.set(key, data, 60)
    return Response(data)
```

---

5. 🛡 XATOLARGA CHIDAMLILIK (fault tolerance)

> Talab: "*tizim bir qismi yiqilsa ham boshqa qismlar ishlashda davom etishi*"

5.1 Yiqilish matritsasi — nima ishlaydi, nima yo'q

Yiqilgan komponent	Ta'sir	Ishlashda davom etadi	Tiklanish
1 ta web node	🟢 Yo'q	Hammasi (LB boshqasiga yuboradi)	Avtomatik 30s
Barcha web node	🔴 To'liq to'xtash	—	Auto-restart + alert
1 ta WS node	🟡 Kichik	Sahifa ishlaydi, real-time → polling'ga o'tadi	Avto 15s
Barcha WS	🟠 O'rta	✅ Menyu, buyurtma, to'lov ishlaydi. KDS polling (10s)	Auto-restart
Redis	🟠 Katta	✅ Menyu (DB'dan), buyurtma. ❌ WS, cache sekin	Cluster failover 10s
PostgreSQL primary	🔴 Yozish yo'q	✅ **O'qish ishlaydi** (replica): menyu, AR ko'rish	Auto-failover 30-60s
PostgreSQL to'liq	🔴 To'liq	❌ Faqat statik CDN	Backup'dan tiklash
Worker (3D)	🟢 Kichik	Hammasi. Yangi 3D navbatda kutadi	Auto-restart
Worker (fiskal)	🟠 O'rta	✅ Hammasi. Chek navbatda (keyin yuboriladi)	Auto-restart + alert
AI provayder	🟢 Yo'q	Keyingi provayderga o'tadi (6 ta zanjir)	Circuit breaker
To'lov provayder	🟠 O'rta	Boshqa provayder yoki naqd	Circuit breaker
OFD (fiskal)	🟠 O'rta	✅ Buyurtma/to'lov ishlaydi, chek navbatda	Retry 24s
CDN	🟠 O'rta	Origin'dan beriladi (sekinroq)	Cloudflare o'zi
S3 (3D fayl)	🟠 O'rta	✅ Menyu ishlaydi, AR yo'q (rasm ko'rsatiladi)	Replica bucket
Telegram API	🟢 Kichik	Hammasi. Bildirishnoma navbatda	Retry

5.2 Circuit Breaker (har tashqi xizmat uchun)

```
# core/resilience.py

class CircuitBreaker:
    """
    3 holat:
      CLOSED    — normal, so'rovlar o'tadi
      OPEN      — xizmat yiqilgan, so'rov YUBORILMAYDI (tez xato)
      HALF_OPEN — sinov, 1 ta so'rov yuboriladi
    """
    def __init__(self, name, fail_threshold=5, timeout=60, success_threshold=2):
        self.name = name
        self.fail_threshold = fail_threshold      # 5 xato → OPEN
        self.timeout = timeout                    # 60s keyin HALF_OPEN
        self.success_threshold = success_threshold

    def call(self, fn, *args, fallback=None, **kwargs):
        state = self._get_state()                 # Redis'da

        if state == 'OPEN':
            if self._should_try(): self._set_state('HALF_OPEN')
            else:
                metrics.inc(f'cb.{self.name}.rejected')
                if fallback: return fallback()
                raise ServiceUnavailable(self.name)

        try:
            result = fn(*args, **kwargs)
            self._on_success()
            return result
        except Exception as e:
            self._on_failure()
            metrics.inc(f'cb.{self.name}.failure')
            if fallback: return fallback()
            raise
```

Qo'llanilishi:

```
ofd_breaker = CircuitBreaker('ofd', fail_threshold=3, timeout=120)

def send_fiscal_receipt(order):
    return ofd_breaker.call(
        _ofd_api_send, order,
        fallback=lambda: enqueue_retry('fiscal', order.id, delay=300)
    )
```

5.3 Retry siyosati

Operatsiya	Urinish	Kechikish	Jitter	Maks vaqt
To'lov webhook	8	eksponensial 2^n s	±30%	24 soat
Fiskal chek	10	5m, 15m, 1s, 6s...	±20%	48 soat
Telegram push	5	1s, 5s, 30s, 2m, 10m	±20%	1 soat
AI so'rov	2	500ms, 2s	—	30 s (keyin boshqa provayder)
3D generatsiya	3	1m, 5m, 15m	—	1 soat (keyin boshqa provayder)
DB deadlock	3	50ms, 200ms, 1s	±50%	5 s

⚠ **Retry faqat idempotent operatsiyalarda!** Har birida idempotency key.

5.4 Graceful degradation zanjirlari

```
🎯 AR KO'RISH:
   WebXR/QuickLook → 3D viewer → 360° rasm → oddiy rasm ✓

🤖 AI CHAT:
   Gemini → Haiku → GPT-mini → Groq → DeepSeek → Sonnet
   → hammasi yiqilsa: "Ofitsiantni chaqiring" + FAQ ✓

📡 REAL-TIME:
   WebSocket → Server-Sent Events → polling 10s → qo'lda yangilash ✓

🧾 FISKAL:
   OFD-1 → OFD-2 → navbat (keyin yuboriladi) → qo'lda chek ✓

💳 TO'LOV:
   Payme → Click → Uzum → naqd (ofitsiantga) ✓

🖼 3D FAYL:
   CDN → S3 origin → lokal cache → rasm ✓

💾 O'QISH:
   Redis cache → Read replica → Primary → statik snapshot ✓
```

---

6. 🔄 O'Z-O'ZINI TIKLASH (self-healing)

> Talab: "*tizim o'zini o'zi qayta tiklay olishi*"

6.1 Avtomatik tiklash mexanizmlari

#	Mexanizm	Nima qiladi	Vaqt
1	**systemd Restart=always**	Process o'lsa qayta ishga tushiradi	5s
2	**Health-check + LB**	Sog'lom bo'lmagan node trafikdan chiqariladi	3-9s
3	**Auto-scaling**	CPU>65% → node qo'shiladi; <25% → olib tashlanadi	2 daq
4	**DB auto-failover**	Primary yiqilsa replica primary bo'ladi (Patroni)	30-60s
5	**Redis Sentinel/Cluster**	Master yiqilsa replica ko'tariladi	10s
6	**Circuit breaker**	Yiqilgan xizmatni chetlab o'tadi, tiklansa qaytaradi	60s
7	**Dead Letter Queue**	Bajarilmagan vazifa DLQ'ga, qo'lda/avto qayta	—
8	**Orphan job recovery**	Worker o'lsa `running` job'lar qayta navbatga	60s
9	**WS auto-reconnect**	Klient uzilsa exponential backoff bilan ulanadi	1-30s
10	**Cache warm-up**	Restart'dan keyin issiq ma'lumot oldindan yuklanadi	30s
11	**Stuck order detector**	45 daq `cooking` da qolgan buyurtma → alert	5 daq
12	**Disk cleanup**	Disk >85% → eski log/temp o'chiriladi	10 daq
13	**Memory guard**	Worker RAM >80% → graceful restart	1 daq
14	**Connection leak detector**	Uzoq idle conn yopiladi	5 daq
15	**Backup verify**	Har kecha backup tiklanishi sinab ko'riladi	kunlik

6.2 Self-healing daemon (kod namunasi)

```
# core/healing/daemon.py — armenu-healer.service
"""
Har 60 soniyada tizimni tekshiradi va muammolarni O'ZI tuzatadi.
Tuzata olmasa — alert yuboradi.
"""

CHECKS = [
    ('orphan_jobs',    check_orphan_jobs,    heal_orphan_jobs),
    ('stuck_orders',   check_stuck_orders,   heal_stuck_orders),
    ('db_connections', check_db_conns,       heal_db_conns),
    ('redis_memory',   check_redis_mem,      heal_redis_mem),
    ('disk_space',     check_disk,           heal_disk),
    ('worker_alive',   check_workers,        heal_workers),
    ('fiscal_queue',   check_fiscal_queue,   heal_fiscal_queue),
    ('ws_connections', check_ws,             heal_ws),
    ('cdn_health',     check_cdn,            heal_cdn),
    ('ai_providers',   check_ai,             heal_ai),
]

def run_cycle():
    for name, check, heal in CHECKS:
        try:
            problem = check()
            if not problem:
                metrics.gauge(f'health.{name}', 1)
                continue

            logger.warning(f"[HEAL] Muammo: {name} — {problem}")
            metrics.gauge(f'health.{name}', 0)
            metrics.inc(f'heal.{name}.attempt')

            healed = heal(problem)

            if healed:
                logger.info(f"[HEAL] ✅ Tuzatildi: {name}")
                metrics.inc(f'heal.{name}.success')
                notify_telegram(f"🔧 Avtomatik tuzatildi: {name}")
            else:
                logger.error(f"[HEAL] ❌ Tuzatib bo'lmadi: {name}")
                metrics.inc(f'heal.{name}.failed')
                alert_oncall(f"☎️ QO'LDA ARALASHUV KERAK: {name}\n{problem}")
        except Exception as e:
            logger.exception(f"[HEAL] Check xatosi: {name}")


# --- Namuna: orphan job ---
def check_orphan_jobs():
    """Worker o'lganda `running` holatda qolgan job'lar"""
    cutoff = timezone.now() - timedelta(minutes=15)
    orphans = Job.objects.filter(status='running', heartbeat_at__lt=cutoff)
    return list(orphans) if orphans.exists() else None

def heal_orphan_jobs(orphans):
    for job in orphans:
        if job.attempts < job.max_attempts:
            job.status, job.attempts = 'queued', job.attempts + 1
            job.save(update_fields=['status', 'attempts'])
            enqueue(job.queue, job.id)
        else:
            job.status = 'failed'; job.save()
            move_to_dlq(job)
    return True

# --- Namuna: stuck order ---
def check_stuck_orders():
    cutoff = timezone.now() - timedelta(minutes=45)
    return list(Order.objects.filter(status='cooking', updated_at__lt=cutoff))

def heal_stuck_orders(orders):
    for o in orders:
        # O'zi hal qila olmaydi — odam kerak, lekin xabar beradi
        ws_send(f'rest_{o.restaurant_id}_admin', {
            'type': 'alert.stuck_order', 'order_id': o.id, 'minutes': 45
        })
        notify_restaurant_admin(o.restaurant, f"⚠ Buyurtma #{o.number} 45 daqiqadan beri tayyorlanmoqda")
    return False   # avtomatik tuzatilmadi, lekin xabar berildi
```

6.3 Chaos engineering (ishonchni tekshirish)

Oyiga 1 marta **atayin** yiqitib ko'riladi (production'ga o'xshash muhitda):

Test	Kutilgan natija
Tasodifiy web node o'chirish	Foydalanuvchi sezmaydi
Redis o'chirish	Menyu ishlaydi (DB'dan), WS uziladi va tiklanadi
DB primary o'chirish	30-60s yozish yo'q, keyin failover, o'qish uzilmaydi
AI provayder bloklash	Keyingi provayderga o'tadi, log'da ko'rinadi
Tarmoq kechikishi +2s	Timeout va circuit breaker ishlaydi
Disk to'ldirish 95%	Auto-cleanup ishlaydi

7. ⚡ ISHLASH TALABLARI (SLA/SLO)

7.1 Javob vaqti (p95)

Endpoint	Maqsad	Maks
`GET /t/<token>` (QR)	80 ms	200 ms
`GET /api/menu/<slug>/`	100 ms	250 ms
`GET /api/ar/<id>/`	60 ms	150 ms
`POST /api/orders/create/`	200 ms	500 ms
`POST /api/payments/create/`	400 ms	1 000 ms
WS xabar yetkazish	50 ms	200 ms
Admin dashboard	300 ms	800 ms
3D fayl (CDN)	1,5 s	4 s

7.2 Mavjudlik (availability)

Xizmat	SLO	Yiliga ruxsat etilgan uzilish
Mijoz menyu (o'qish)	**99,95%**	4,4 soat
Buyurtma yaratish	**99,9%**	8,8 soat
To'lov	**99,9%**	8,8 soat
Real-time (WS)	99,5%	43,8 soat
Admin panel	99,5%	43,8 soat
3D generatsiya	99,0%	87,6 soat

7.3 Ma'lumot yo'qotmaslik

Ma'lumot	RPO (yo'qotish)	RTO (tiklash)
Buyurtma / To'lov	**0** (sinxron replikatsiya)	5 daqiqa
Menyu / Restoran	5 daqiqa	15 daqiqa
3D fayllar	1 soat	1 soat
Analitika	1 soat	4 soat

---

8. 🔐 XAVFSIZLIK

8.1 Qatlamlar

Qatlam	Chora
Tarmoq	Cloudflare WAF, DDoS, faqat 80/443 ochiq, DB/Redis private tarmoqda
Ilova	CSRF, XSS (CSP), SQL injection (ORM), rate limiting
Auth	JWT (qisqa TTL) + refresh, PIN bcrypt, 2FA super-admin
Avtorizatsiya	Har so'rovda `restaurant_id` tekshiruvi ⚡
Ma'lumot	TLS 1.3, DB shifrlash (at-rest), maxfiy ma'lumot Vault'da
To'lov	PCI DSS — karta ma'lumoti **bizda saqlanmaydi**
Audit	Har muhim harakat AuditLog'da (kim, nima, qachon, IP)

8.2 ⚠ Multi-tenant izolyatsiya (eng muhim xavfsizlik masalasi)

```
# core/mixins.py — HAR view'da ishlatiladi

class TenantScopedMixin:
    """Restoran chegarasidan chiqishni imkonsiz qiladi"""
    def get_queryset(self):
        qs = super().get_queryset()
        rid = self.request.restaurant_id      # middleware o'rnatadi
        if rid is None:
            raise PermissionDenied("Restoran aniqlanmadi")
        return qs.filter(restaurant_id=rid)


# ✗ HECH QACHON:
dish = Dish.objects.get(pk=dish_id)              # boshqa restoranniki bo'lishi mumkin!

# ✓ DOIM:
dish = Dish.objects.get(pk=dish_id, category__restaurant_id=request.restaurant_id)
```

Avtomatik tekshiruv: CI'da test — har model uchun cross-tenant kirish urinishi **403 qaytarishi shart**. Test o'tmasa deploy bloklanadi.

8.3 Rate limiting

Endpoint	Limit	Kalit
QR kirish	60/daq	IP
Menyu API	120/daq	IP
Buyurtma yaratish	10/daq	session
Chaqiruv	1/daq	stol
AI chat	10/5daq	session
Staff login	5/15daq	IP+PIN
Admin API	300/daq	user
To'lov	5/daq	session

---

9. 📊 KUZATUV (observability)

9.1 Metrikalar (Prometheus)

```
# Biznes
armenu_orders_total{restaurant,status}
armenu_order_value_sum{restaurant}
armenu_ar_views_total{restaurant,dish}
armenu_ar_conversion_ratio{restaurant}
armenu_active_restaurants

# Texnik
armenu_http_request_duration_seconds{endpoint,method,status}
armenu_ws_connections{node}
armenu_db_query_duration_seconds{query_type}
armenu_cache_hit_ratio{cache_name}
armenu_queue_depth{queue}
armenu_job_duration_seconds{job_type}
armenu_circuit_breaker_state{service}

# Tashqi
armenu_external_api_duration_seconds{provider}
armenu_ai_cost_usd_total{provider,model}
armenu_fiscal_queue_depth
```

9.2 Alert qoidalari

Alert	Shart	Daraja	Kimga
Xatolik darajasi	5xx > 1% (5 daq)	🔴 P1	Telegram + qo'ng'iroq
Javob sekin	p95 > 1s (5 daq)	🟠 P2	Telegram
DB conn	> 85% pool	🟠 P2	Telegram
Navbat to'lgan	depth > 1000	🟠 P2	Telegram
Fiskal navbat	depth > 50 (10 daq)	🔴 P1	Telegram
Disk	> 85%	🟠 P2	Telegram
WS uzilish	> 20% (5 daq)	🟡 P3	Slack
AI xarajat	> $50/kun	🟡 P3	Telegram
Restoran nofaol	3 kun buyurtma yo'q	🟡 P3	Sotuv jamoasi

10. 💾 BACKUP VA FALOKATDAN TIKLASH

Ma'lumot	Chastota	Saqlash	Joy
PostgreSQL to'liq	Kunlik 03:00	30 kun	S3 (boshqa region)
PostgreSQL WAL	Uzluksiz	7 kun	S3
Redis snapshot	6 soatda	3 kun	S3
3D fayllar	Real-time replica	Doimiy	S3 versioning
Kod	Har commit	Doimiy	Git (GitHub + lokal)
Konfiguratsiya	Har o'zgarish	Doimiy	Git (shifrlangan)

Tiklash mashqi: har chorakda backup'dan to'liq tiklash sinovi (staging'da).

QISM IV

Ma'lumot modeli va API

1. MA'LUMOTLAR BAZASI SXEMASI (to'liq)

1.1 Bog'lanishlar diagrammasi



1.2 Jadvallar (Django modellari)

`tenants.Restaurant` — asosiy tenant

```
class Restaurant(models.Model):
    id            = models.BigAutoField(primary_key=True)
    slug          = models.SlugField(max_length=64, unique=True, db_index=True)
    name          = models.CharField(max_length=200)
    legal_name    = models.CharField(max_length=255, blank=True)   # yuridik nom
    inn           = models.CharField(max_length=20, blank=True)    # STIR
    phone         = models.CharField(max_length=20)                # 998XXXXXXXXX
    address       = models.TextField(blank=True)
    city          = models.CharField(max_length=100, db_index=True)
    lat           = models.DecimalField(max_digits=9,  decimal_places=6, null=True)
    lng           = models.DecimalField(max_digits=9,  decimal_places=6, null=True)

    # Brending
    logo          = models.ImageField(upload_to='logos/', null=True, blank=True)
    color_primary = models.CharField(max_length=7, default='#E85D2A')
    color_accent  = models.CharField(max_length=7, default='#1A1A2E')
    custom_domain = models.CharField(max_length=120, blank=True, db_index=True)

    # Til
    default_lang  = models.CharField(max_length=2, choices=LANGS, default='uz')
    langs_enabled = ArrayField(models.CharField(max_length=2), default=list_uz_ru_en)

    # Tarif
    plan          = models.CharField(max_length=20, choices=PLANS, default='free')
    plan_expires  = models.DateTimeField(null=True, blank=True)
    ar_model_limit= models.IntegerField(default=0)    # FREE=0, PRO=50
    table_limit   = models.IntegerField(default=3)    # FREE=3, boshqa=cheksiz(-1)

    # Xizmat haqi
    service_fee_pct = models.DecimalField(max_digits=5, decimal_places=2, default=0)

    # Fiskal
    ofd_provider  = models.CharField(max_length=20, blank=True)  # uzkassa|epos|qpos
    ofd_config    = models.JSONField(default=dict)              # shifrlangan
    fiscal_enabled= models.BooleanField(default=False)

    # Holat
    is_active     = models.BooleanField(default=True, db_index=True)
    created_at    = models.DateTimeField(auto_now_add=True, db_index=True)
    updated_at    = models.DateTimeField(auto_now=True)

    # Cache versiyasi (invalidation uchun) ⚡
    menu_version  = models.IntegerField(default=1)

    class Meta:
        indexes = [
            models.Index(fields=['is_active', 'plan']),
            models.Index(fields=['city', 'is_active']),
        ]
```

`tables.Hall` va `tables.Table`

```
class Hall(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=CASCADE, related_name='halls')
    name       = models.CharField(max_length=100)
    name_ru    = models.CharField(max_length=100, blank=True)
    name_en    = models.CharField(max_length=100, blank=True)
    order      = models.IntegerField(default=0)
    plan_json  = models.JSONField(default=dict)
    # plan_json: {"width":1000,"height":700,
    #             "objects":[{"type":"wall","x1":0,"y1":0,"x2":1000,"y2":0},
    #                        {"type":"bar","x":400,"y":600,"w":200,"h":60}]}
    # ⚡ v3 (3D) uchun tayyor: z, height maydonlari qo'shiladi
    is_active  = models.BooleanField(default=True)


class Table(models.Model):
    hall       = models.ForeignKey(Hall, on_delete=CASCADE, related_name='tables')
    restaurant = models.ForeignKey(Restaurant, on_delete=CASCADE, db_index=True)  # denorm ⚡
    number     = models.CharField(max_length=20)         # "5", "VIP-2", "A12"
    qr_token   = models.UUIDField(default=uuid.uuid4, unique=True, db_index=True)  ⚡ NOYOB
    qr_token_old = models.UUIDField(null=True, blank=True, db_index=True)  # grace 24s
    qr_rotated_at= models.DateTimeField(null=True, blank=True)

    capacity   = models.SmallIntegerField(default=4)
    shape      = models.CharField(max_length=10, default='rect')  # rect|circle
    x          = models.IntegerField(default=0)        # zal planidagi joyi
    y          = models.IntegerField(default=0)
    w          = models.IntegerField(default=80)
    h          = models.IntegerField(default=80)
    rotation   = models.SmallIntegerField(default=0)

    status     = models.CharField(max_length=20, default='free', db_index=True)
    # free | occupied | ordering | waiting | ready | bill | cleaning

    assigned_waiter = models.ForeignKey('accounts.Staff', null=True, blank=True, on_delete=SET_NULL)
    is_active  = models.BooleanField(default=True)

    class Meta:
        unique_together = [('restaurant', 'number')]
        indexes = [models.Index(fields=['restaurant', 'status'])]
```

`menu.Category` va `menu.Dish`

```
class Category(models.Model):
    restaurant = models.ForeignKey(Restaurant, on_delete=CASCADE, related_name='categories')
    name_uz    = models.CharField(max_length=120)
    name_ru    = models.CharField(max_length=120, blank=True)
    name_en    = models.CharField(max_length=120, blank=True)
    icon       = models.CharField(max_length=40, blank=True)
    image      = models.ImageField(upload_to='cat/', null=True, blank=True)
    order      = models.IntegerField(default=0, db_index=True)
    is_active  = models.BooleanField(default=True)


class Dish(models.Model):
    category   = models.ForeignKey(Category, on_delete=CASCADE, related_name='dishes')
    restaurant = models.ForeignKey(Restaurant, on_delete=CASCADE, db_index=True)  # denorm

    name_uz    = models.CharField(max_length=200)
    name_ru    = models.CharField(max_length=200, blank=True)
    name_en    = models.CharField(max_length=200, blank=True)
    desc_uz    = models.TextField(blank=True)
    desc_ru    = models.TextField(blank=True)
    desc_en    = models.TextField(blank=True)

    # Tarjima holati
    tr_auto    = models.JSONField(default=dict)   # {"ru":true,"en":false} — AI tarjimami
    tr_approved= models.JSONField(default=dict)   # {"ru":true,"en":false} — admin tasdiqladimi

    price      = models.DecimalField(max_digits=12, decimal_places=2)
    old_price  = models.DecimalField(max_digits=12, decimal_places=2, null=True, blank=True)
    image      = models.ImageField(upload_to='dish/')
    image_webp = models.CharField(max_length=255, blank=True)  # optimallashtirilgan

    weight_g   = models.IntegerField(null=True, blank=True)
    cook_min   = models.SmallIntegerField(default=15)
    calories   = models.IntegerField(null=True, blank=True)

    ingredients_uz = models.TextField(blank=True)
    ingredients_ru = models.TextField(blank=True)
    ingredients_en = models.TextField(blank=True)

    allergens  = ArrayField(models.CharField(max_length=30), default=list, blank=True)
    # ['nuts','gluten','dairy','eggs','fish','soy',...]
    diet_tags  = ArrayField(models.CharField(max_length=30), default=list, blank=True)
    # ['halal','vegetarian','vegan','spicy','no_spicy','lactose_free']

    ikpu_code  = models.CharField(max_length=30, blank=True, db_index=True)  ⚡ SOLIQ
    package_code = models.CharField(max_length=30, blank=True)               # o'lchov birligi

    is_available = models.BooleanField(default=True, db_index=True)  # stop-list
    is_hit     = models.BooleanField(default=False)
    is_new     = models.BooleanField(default=False)
    order      = models.IntegerField(default=0)

    # Denormalizatsiya (tez o'qish uchun) ⚡
    has_3d     = models.BooleanField(default=False, db_index=True)
    rating_avg = models.DecimalField(max_digits=3, decimal_places=2, default=0)
    rating_cnt = models.IntegerField(default=0)
    sold_count = models.IntegerField(default=0)    # menyu-injiniring uchun

    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        indexes = [
            models.Index(fields=['restaurant', 'is_available', 'order']),
            models.Index(fields=['category', 'is_active_order']),
            GinIndex(fields=['diet_tags']),
            GinIndex(fields=['allergens']),
        ]
```

⚡ `ar3d.DishLibrary` **— ENG MUHIM AKTIV (global, tenant'siz)**

```
class DishLibrary(models.Model):
    """
    Umumiy 3D taom kutubxonasi. Barcha restoranlar foydalanadi.
    ⚡ Bu — loyihaning eng qimmatli aktivi va himoya devori.
    """
    key        = models.SlugField(max_length=80, unique=True, db_index=True)
    # 'osh-toshkent', 'manti', 'somsa-goshtli', 'lagmon-quyruq'

    name_uz    = models.CharField(max_length=200)
    name_ru    = models.CharField(max_length=200)
    name_en    = models.CharField(max_length=200)
    aliases    = ArrayField(models.CharField(max_length=100), default=list)
    # ['palov','плов','pilaf','osh'] — moslashtirishda ishlatiladi

    category_key = models.CharField(max_length=50, db_index=True)  # 'national','salad',...
    cuisine    = models.CharField(max_length=50, default='uzbek')

    glb_url    = models.URLField(max_length=500)
    usdz_url   = models.URLField(max_length=500)
    poster_url = models.URLField(max_length=500)
    preview_360= models.URLField(max_length=500, blank=True)

    real_size_cm = models.DecimalField(max_digits=6, decimal_places=1)  ⚡ real o'lcham
    glb_bytes  = models.IntegerField()
    usdz_bytes = models.IntegerField()
    poly_count = models.IntegerField()

    quality    = models.SmallIntegerField(default=3)   # 1-5
    source     = models.CharField(max_length=20)       # photogrammetry|ai|manual
    embedding  = VectorField(dimensions=768, null=True)  # pgvector — semantik qidiruv ⚡

    used_by_count = models.IntegerField(default=0, db_index=True)
    created_at = models.DateTimeField(auto_now_add=True)
    is_active  = models.BooleanField(default=True)

    class Meta:
        indexes = [
            models.Index(fields=['category_key', 'is_active']),
            models.Index(fields=['-used_by_count']),
            GinIndex(fields=['aliases']),
        ]
```

`ar3d.DishModel3D` — restoran taomining modeli

```
class DishModel3D(models.Model):
    dish       = models.OneToOneField(Dish, on_delete=CASCADE, related_name='model3d')
    library    = models.ForeignKey(DishLibrary, null=True, blank=True, on_delete=SET_NULL)
    # library to'ldirilgan bo'lsa → kutubxonadan olingan (bepul)

    glb        = models.FileField(upload_to='ar/glb/', blank=True)
    usdz       = models.FileField(upload_to='ar/usdz/', blank=True)
    poster     = models.ImageField(upload_to='ar/poster/', blank=True)

    real_size_cm = models.DecimalField(max_digits=6, decimal_places=1, default=25)
    glb_bytes  = models.IntegerField(default=0)
    usdz_bytes = models.IntegerField(default=0)

    status     = models.CharField(max_length=20, default='none', db_index=True)
    # none | queued | processing | ready | failed
    source     = models.CharField(max_length=20, blank=True)  # library|ai|upload|photogrammetry
    error_msg  = models.TextField(blank=True)
    progress   = models.SmallIntegerField(default=0)   # 0-100

    # Analitika (denorm)
    view_count = models.IntegerField(default=0)
    convert_count = models.IntegerField(default=0)

    created_at = models.DateTimeField(auto_now_add=True)
    ready_at   = models.DateTimeField(null=True, blank=True)
```

`ar3d.ARView` — analitika (partitioned)

```
class ARView(models.Model):
    """Partition: oy bo'yicha. Eski partitionlar S3'ga arxivlanadi."""
    restaurant   = models.ForeignKey(Restaurant, on_delete=CASCADE, db_index=True)
    dish         = models.ForeignKey(Dish, on_delete=CASCADE)
    guest        = models.ForeignKey('guest.Guest', null=True, on_delete=SET_NULL)
    table        = models.ForeignKey(Table, null=True, on_delete=SET_NULL)
    session_id   = models.CharField(max_length=64, db_index=True)

    mode         = models.CharField(max_length=20)  # webxr|quick-look|scene-viewer|3d-only
    duration_sec = models.IntegerField(default=0)
    converted    = models.BooleanField(default=False)  # buyurtma qilindimi
    device       = models.CharField(max_length=20)     # ios|android|desktop
    created_at   = models.DateTimeField(auto_now_add=True, db_index=True)

    class Meta:
        indexes = [
            models.Index(fields=['restaurant', 'created_at']),
            models.Index(fields=['dish', 'created_at']),
        ]
```

`orders.Order` — buyurtma (partitioned)

```
class Order(models.Model):
    STATUS = [
        ('new','Yangi'), ('accepted','Qabul qilindi'), ('cooking','Tayyorlanmoqda'),
        ('ready','Tayyor'), ('served','Berildi'), ('paid','To\'landi'),
        ('cancelled','Bekor qilindi'),
    ]
    restaurant = models.ForeignKey(Restaurant, on_delete=PROTECT, db_index=True)
    table      = models.ForeignKey(Table, null=True, on_delete=SET_NULL)
    guest      = models.ForeignKey('guest.Guest', null=True, on_delete=SET_NULL)
    session_id = models.CharField(max_length=64, db_index=True)
    waiter     = models.ForeignKey('accounts.Staff', null=True, blank=True, on_delete=SET_NULL)

    number     = models.CharField(max_length=20, db_index=True)   # "#047" kunlik
    status     = models.CharField(max_length=20, choices=STATUS, default='new', db_index=True)
    source     = models.CharField(max_length=20, default='qr')    # qr|waiter|telegram|delivery

    subtotal   = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    service_fee= models.DecimalField(max_digits=14, decimal_places=2, default=0)
    discount   = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    total      = models.DecimalField(max_digits=14, decimal_places=2, default=0)

    note       = models.TextField(blank=True)
    eta_min    = models.SmallIntegerField(default=0)

    idempotency_key = models.CharField(max_length=64, unique=True, null=True, db_index=True) ⚡

    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    accepted_at= models.DateTimeField(null=True, blank=True)
    cooking_at = models.DateTimeField(null=True, blank=True)
    ready_at   = models.DateTimeField(null=True, blank=True)
    served_at  = models.DateTimeField(null=True, blank=True)
    paid_at    = models.DateTimeField(null=True, blank=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        indexes = [
            models.Index(fields=['restaurant', 'status', '-created_at']),
            models.Index(fields=['restaurant', '-created_at']),
            models.Index(fields=['table', 'status']),
        ]
```

⚠ Holat mashinasi (state machine) — qat'iy:

```
      new ──accept──► accepted ──cook──► cooking ──ready──► ready
       │                │                 │                 │
       │                │                 │              serve
       │                │                 │                 ▼
       └──cancel────────┴─────────────────┴──────────► served ──pay──► paid
                                                         │
                                                      cancel
                                                         ▼
                                                     cancelled

⚠ Ruxsat etilmagan o'tish → 409 Conflict
⚠ Har o'tishda: timestamp yoziladi + WS xabar + audit log
```

```
# apps/orders/state.py
ALLOWED = {
    'new':      {'accepted', 'cancelled'},
    'accepted': {'cooking', 'cancelled'},
    'cooking':  {'ready', 'cancelled'},
    'ready':    {'served', 'cancelled'},
    'served':   {'paid', 'cancelled'},
    'paid':     set(),
    'cancelled':set(),
}

def transition(order, new_status, actor=None):
    if new_status not in ALLOWED[order.status]:
        raise InvalidTransition(f"{order.status} → {new_status} mumkin emas")
    with transaction.atomic():
        old = order.status
        order.status = new_status
        setattr(order, f'{new_status}_at', timezone.now())
        order.save(update_fields=['status', f'{new_status}_at', 'updated_at'])
        AuditLog.log(order, old, new_status, actor)
    transaction.on_commit(lambda: broadcast_order_status(order))
```

Qolgan jadvallar (qisqacha)

Model	Asosiy maydonlar
`OrderItem`	order, dish, qty(int), price, modifiers(JSON), note, ar_viewed(bool)
`DishModifier`	dish, name_uz/ru/en, price_delta, is_default, group
`Guest`	restaurant, device_token(UUID), phone, name, tg_user_id, bonus, visits, total_spent
`GuestSession`	guest, table, started_at, ended_at, order_count
`Staff`	restaurant, user, role, pin_hash, tables[], is_active, tips_balance
`Payment`	order, provider, amount, status, ext_id, idempotency_key, paid_at
`Tip`	order, staff, amount, payment, created_at
`FiscalReceipt`	order, ofd_provider, fiscal_id, qr_url, status, sent_at, retry_count
`Call`	table, type, status, created_at, resolved_at, resolved_by
`Review`	order, guest, rating, rating_food, rating_service, text, redirected_to_google
`Job`	queue, type, payload(JSON), status, attempts, heartbeat_at, result
`AuditLog`	restaurant, actor, action, model, object_id, old, new, ip, ts
`AIUsage`	restaurant, provider, model, tokens_in, tokens_out, cost_usd, ts
`AnalyticsEvent`	(ClickHouse'ga) restaurant, type, payload, ts

---

2. 🔌 API SPETSIFIKATSIYASI

2.1 Umumiy qoidalar

Qoida	Qiymat
Baza URL	`https://api.armenu.uz/v1/`
Format	JSON (UTF-8)
Auth (mijoz)	yo'q (session cookie)
Auth (xodim)	`Authorization: Bearer <JWT>`
Auth (admin)	`Authorization: Bearer <JWT>`
Idempotency	`X-Idempotency-Key: <uuid>` (POST/PUT)
Til	`?lang=uz|ru|en` yoki `Accept-Language`
Versiyalash	URL'da `/v1/`
Sahifalash	`?page=1&page_size=50` (maks 200)
Rate limit	Header: `X-RateLimit-Remaining`

2.2 Xato formati (yagona)

```
{
  "error": {
    "code": "DISH_UNAVAILABLE",
    "message_uz": "Bu taom hozir mavjud emas",
    "message_ru": "Это блюдо сейчас недоступно",
    "message_en": "This dish is currently unavailable",
    "details": {"dish_id": 15, "dish_name": "Lag'mon"},
    "trace_id": "a3f2c891-..."
  }
}
```

Xato kodlari:

Kod	HTTP	Ma'no
`QR_INVALID`	404	QR token topilmadi
`RESTAURANT_INACTIVE`	403	Restoran nofaol
`TABLE_INACTIVE`	403	Stol nofaol
`DISH_UNAVAILABLE`	409	Taom stop-listda
`PRICE_CHANGED`	409	Narx o'zgargan
`INVALID_TRANSITION`	409	Holat o'tishi mumkin emas
`PLAN_LIMIT`	402	Tarif limiti
`RATE_LIMITED`	429	Ko'p so'rov
`TENANT_MISMATCH`	403	Boshqa restoran resursi ⚡
`VALIDATION_ERROR`	400	Ma'lumot noto'g'ri
`SERVICE_UNAVAILABLE`	503	Tashqi xizmat yiqilgan

2.3 Endpointlar (to'liq ro'yxat)

🧍 MIJOZ (auth yo'q)

```
GET    /t/{qr_token}                    # QR kirish → redirect
GET    /v1/menu/{slug}/?lang=uz         # To'liq menyu (cache 60s)
GET    /v1/menu/{slug}/search/?q=osh    # Qidiruv
GET    /v1/dish/{id}/?lang=uz           # Taom tafsiloti
GET    /v1/ar/{dish_id}/                # ⚡ 3D fayl URL'lari
POST   /v1/ar/view/                     # AR ko'rish yozuvi
POST   /v1/orders/                      # ⚡ Buyurtma yaratish
GET    /v1/orders/{id}/                 # Buyurtma holati
POST   /v1/orders/{id}/add-items/       # Qo'shimcha buyurtma
GET    /v1/orders/session/{sid}/        # Sessiya buyurtmalari
POST   /v1/calls/                       # Chaqiruv
GET    /v1/bill/{session_id}/           # Hisob
POST   /v1/payments/                    # To'lov boshlash
POST   /v1/payments/webhook/{provider}/ # Provayder callback
POST   /v1/tips/                        # Choychaqa
POST   /v1/reviews/                     # Otziv
POST   /v1/ai/chat/                     # AI chatbot
POST   /v1/lang/                        # Til o'zgartirish
```

👨‍🍳 XODIM (JWT)

```
POST   /v1/staff/login/                 # PIN bilan kirish
POST   /v1/staff/refresh/               # Token yangilash
GET    /v1/staff/me/                    # Profil
GET    /v1/staff/orders/?status=new     # Buyurtmalar
POST   /v1/staff/orders/{id}/accept/    # Qabul qilish
POST   /v1/staff/orders/{id}/cook/      # Tayyorlashni boshlash (KDS)
POST   /v1/staff/orders/{id}/ready/     # Tayyor (KDS)
POST   /v1/staff/orders/{id}/serve/     # Olib bordim
POST   /v1/staff/orders/{id}/cancel/    # Bekor qilish
POST   /v1/staff/orders/                # Ofitsiant buyurtma kiritadi
GET    /v1/staff/tables/                # Zal + stollar holati
GET    /v1/staff/calls/                 # Chaqiruvlar
POST   /v1/staff/calls/{id}/resolve/    # Chaqiruv bajarildi
POST   /v1/staff/dishes/{id}/stop/      # Stop-list
GET    /v1/staff/stats/                 # Mening statistikam
POST   /v1/staff/shift/start|end/       # Smena
```

👔 ADMIN (JWT)

```
GET    /v1/admin/dashboard/              # Real-time ko'rsatkichlar
CRUD   /v1/admin/categories/
CRUD   /v1/admin/dishes/
POST   /v1/admin/dishes/bulk-import/     # Excel/CSV import
CRUD   /v1/admin/modifiers/
CRUD   /v1/admin/halls/
CRUD   /v1/admin/tables/
POST   /v1/admin/tables/{id}/regen-qr/   # QR yangilash
GET    /v1/admin/tables/qr-pdf/          # ⚡ QR PDF yuklab olish
CRUD   /v1/admin/staff/
GET    /v1/admin/orders/                 # Filtr + eksport
GET    /v1/admin/reports/sales/          # Savdo hisoboti
GET    /v1/admin/reports/dishes/         # Taomlar hisoboti
GET    /v1/admin/reports/menu-engineering/# ⚡ Yulduz/Ot/Jumboq/It
GET    /v1/admin/reports/ar-analytics/   # ⚡ AR ta'siri
GET    /v1/admin/reports/staff/          # Xodim ish faoliyati
POST   /v1/admin/ar/generate/            # ⚡ 3D model yaratish
GET    /v1/admin/ar/jobs/                # Generatsiya holati
POST   /v1/admin/ar/upload/              # Qo'lda model yuklash
GET    /v1/admin/ar/library/search/      # Kutubxonadan qidirish
POST   /v1/admin/i18n/auto-translate/    # ⚡ AI tarjima
CRUD   /v1/admin/reviews/
CRUD   /v1/admin/promos/
GET/PUT /v1/admin/settings/
GET/PUT /v1/admin/fiscal/                # OFD sozlamalari
GET    /v1/admin/subscription/           # Tarif
POST   /v1/admin/subscription/upgrade/
GET    /v1/admin/export/                 # To'liq ma'lumot eksporti
```

🔧 SUPER-ADMIN

```
CRUD   /v1/platform/restaurants/
GET    /v1/platform/health/              # Tizim salomatligi
CRUD   /v1/platform/library/             # ⚡ 3D kutubxona
GET    /v1/platform/finance/             # Obunalar, daromad
GET    /v1/platform/ai-usage/            # AI xarajatlari
POST   /v1/platform/broadcast/           # Barcha restoranga xabar
```

2.4 Muhim endpointlar — to'liq spetsifikatsiya

`GET /v1/menu/{slug}/` — menyu (eng ko'p chaqiriladigan)

So'rov:

```
GET /v1/menu/registon/?lang=uz&table=5
Accept-Language: uz
If-None-Match: "v42-uz"
```

Javob (200):

```
{
  "restaurant": {
    "id": 1, "slug": "registon", "name": "Registon",
    "logo": "https://cdn.../logo.webp",
    "colors": {"primary": "#E85D2A", "accent": "#1A1A2E"},
    "langs": ["uz", "ru", "en"],
    "service_fee_pct": 10,
    "has_ar": true
  },
  "table": {"id": 5, "number": "5", "hall": "1-qavat"},
  "categories": [
    {"id": 1, "name": "Milliy taomlar", "icon": "🍲", "order": 1}
  ],
  "dishes": [
    {
      "id": 15,
      "cat": 1,
      "name": "Osh (Palov)",
      "desc": "Toshkent usulida...",
      "price": 45000,
      "old_price": null,
      "img": "https://cdn.../osh_400.webp",
      "weight_g": 400,
      "cook_min": 20,
      "allergens": [],
      "diet": ["halal", "no_spicy"],
      "has_3d": true,
      "poster_3d": "https://cdn.../osh_poster.webp",
      "is_hit": true,
      "rating": 4.8,
      "available": true,
      "modifiers": [
        {"id": 3, "name": "Qo'shimcha go'sht", "price": 25000, "group": "extra"}
      ]
    }
  ],
  "upsell_rules": [
    {"trigger": [15], "suggest": 42, "text": "Oshga achchiq-chuchuk salat?"}
  ],
  "version": 42,
  "cached_at": "2026-09-04T14:32:00Z"
}
```

Headers: `ETag: "v42-uz"`, `Cache-Control: public, max-age=60`

Kesh strategiyasi:

```
1. Brauzer localStorage (5 daq)
2. CDN (60s, ETag)
3. Redis: menu:{slug}:{lang}:v{version} (60s)
4. DB (oxirgi chora)

⚡ Invalidation: admin menyuni o'zgartirsa → Restaurant.menu_version += 1
   → barcha kesh kaliti o'zgaradi → yangi ma'lumot
```

`POST /v1/orders/` — buyurtma yaratish (eng kritik)

So'rov:

```
POST /v1/orders/
X-Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json

{
  "session_id": "gs_a3f2c891",
  "table_id": 5,
  "items": [
    {"dish_id": 15, "qty": 2, "modifiers": [3], "note": "achchiq qilmang", "ar_viewed": true},
    {"dish_id": 42, "qty": 1, "modifiers": [], "note": "", "ar_viewed": false}
  ],
  "note": "Tez bo'lsa yaxshi"
}
```

Javob (201):

```
{
  "id": 1047,
  "number": "#047",
  "status": "new",
  "subtotal": 158000,
  "service_fee": 15800,
  "total": 173800,
  "eta_min": 18,
  "created_at": "2026-09-04T14:32:15Z",
  "items": [...]
}
```

Backend validatsiya ketma-ketligi (qat'iy tartib):

```
# apps/orders/services.py::create_order

def create_order(session, table_id, items, note, idem_key):
    # 1. Idempotency (Redis, TTL 24s)
    existing = check_idempotency(idem_key)
    if existing: return existing                          # dublikat yo'q ✓

    # 2. Stol tekshiruvi (tenant scope!)
    table = Table.objects.select_related('restaurant').get(
        id=table_id, restaurant_id=session.restaurant_id, is_active=True)

    # 3. Restoran faolmi
    if not table.restaurant.is_active: raise RestaurantInactive()

    # 4. Taomlar tekshiruvi (BITTA so'rov, N+1 yo'q)
    dish_ids = [i['dish_id'] for i in items]
    dishes = {d.id: d for d in Dish.objects.filter(
        id__in=dish_ids, restaurant_id=session.restaurant_id
    ).prefetch_related('modifiers')}

    if len(dishes) != len(set(dish_ids)): raise TenantMismatch()

    # 5. Mavjudlik (stop-list)
    for i in items:
        d = dishes[i['dish_id']]
        if not d.is_available: raise DishUnavailable(d)

    # 6. ⚡ NARX SERVERDA HISOBLANADI (frontendga ISHONILMAYDI)
    subtotal = Decimal(0)
    order_items = []
    for i in items:
        d = dishes[i['dish_id']]
        qty = max(1, min(99, int(math.ceil(i['qty']))))    # butun son ⚡
        mod_sum = sum(m.price_delta for m in d.modifiers.filter(id__in=i['modifiers']))
        line = (d.price + mod_sum) * qty
        subtotal += line
        order_items.append(OrderItem(dish=d, qty=qty, price=d.price + mod_sum,
                                     modifiers=i['modifiers'], note=i['note'][:200],
                                     ar_viewed=i.get('ar_viewed', False)))

    service_fee = subtotal * table.restaurant.service_fee_pct / 100
    total = subtotal + service_fee

    # 7. Transaksiya
    with transaction.atomic():
        order = Order.objects.create(
            restaurant_id=session.restaurant_id, table=table, guest=session.guest,
            session_id=session.id, number=next_number(table.restaurant),
            subtotal=subtotal, service_fee=service_fee, total=total,
            note=note[:500], eta_min=max(d.cook_min for d in dishes.values()),
            idempotency_key=idem_key, source='qr',
        )
        for oi in order_items: oi.order = order
        OrderItem.objects.bulk_create(order_items)
        table.status = 'ordering'; table.save(update_fields=['status'])

    # 8. Post-commit (transaksiya muvaffaqiyatli bo'lgandan KEYIN)
    transaction.on_commit(lambda: _after_order_created(order))

    save_idempotency(idem_key, order)
    return order


def _after_order_created(order):
    r = order.restaurant_id
    ws_group_send(f'rest_{r}_kds',     {'type':'order.new',     'order': serialize(order)})
    ws_group_send(f'rest_{r}_waiters', {'type':'order.new',     'order': serialize_short(order)})
    ws_group_send(f'rest_{r}_admin',   {'type':'order.new',     'order': serialize_short(order)})
    ws_group_send(f'guest_{order.session_id}', {'type':'order.created', 'id': order.id})

    enqueue('notify',    'waiter_new_order', order_id=order.id)
    enqueue('notify',    'kitchen_print',    order_id=order.id)
    enqueue('analytics', 'order_created',    order_id=order.id)
```

---

3. 🔌 WEBSOCKET SPETSIFIKATSIYASI

3.1 Ulanish nuqtalari

URL	Kim	Guruh
`/ws/guest/{session_id}/`	Mijoz	`guest_{session_id}`
`/ws/staff/{token}/`	Ofitsiant	`rest_{id}_waiters` + `staff_{id}`
`/ws/kds/{token}/`	Oshxona	`rest_{id}_kds`
`/ws/admin/{token}/`	Admin	`rest_{id}_admin`

⚠ **Guruhlar HAR DOIM restoran-scoped** — cross-tenant xabar mumkin emas.

3.2 Xabar formati

```
{
  "type": "order.new",
  "ts": "2026-09-04T14:32:15Z",
  "data": { ... },
  "seq": 1047
}
```

3.3 Xabar turlari

Type	Yo'nalish	Ma'no
`order.new`	S→KDS/waiter/admin	Yangi buyurtma
`order.status`	S→guest/all	Holat o'zgardi
`order.ready`	S→waiter	🔔 Tayyor!
`order.cancelled`	S→all	Bekor qilindi
`call.new`	S→waiter	Mijoz chaqirdi
`call.resolved`	S→guest	Bajarildi
`table.status`	S→admin/waiter	Stol holati
`table.scanned`	S→admin	QR skanerlandi
`payment.success`	S→guest/cashier	To'lov o'tdi
`dish.stopped`	S→all	Taom tugadi
`model3d.ready`	S→admin	3D tayyor
`alert.*`	S→admin	Ogohlantirish
`ping` / `pong`	↔	Heartbeat (25s)

3.4 Klient tomonda ishonchlilik

```
class ReliableWS {
  constructor(url) {
    this.url = url; this.retries = 0; this.lastSeq = 0;
    this.connect();
  }
  connect() {
    this.ws = new WebSocket(this.url);
    this.ws.onopen = () => {
      this.retries = 0;
      this.startHeartbeat();
      this.syncMissed();                  // ⚡ o'tkazib yuborilganlarni olish
    };
    this.ws.onmessage = (e) => {
      const m = JSON.parse(e.data);
      if (m.seq && m.seq <= this.lastSeq) return;   // dublikat
      this.lastSeq = m.seq || this.lastSeq;
      this.handle(m);
    };
    this.ws.onclose = () => {
      this.stopHeartbeat();
      const delay = Math.min(1000 * 2 ** this.retries, 30000);  // exponential
      const jitter = delay * (0.8 + Math.random() * 0.4);
      this.retries++;
      setTimeout(() => this.connect(), jitter);
      if (this.retries > 3) this.startPolling();    // ⚡ fallback
    };
  }
  async syncMissed() {
    // WS uzilgan paytdagi o'zgarishlarni REST orqali olish
    const r = await fetch(`/v1/sync/?since_seq=${this.lastSeq}`);
    (await r.json()).events.forEach(m => this.handle(m));
  }
  startPolling() { /* har 10s REST so'rov */ }
}
```

QISM V

Kod yozish tartibi

Bu qism AI/dasturchi uchun to'g'ridan-to'g'ri bajariladigan yo'riqnoma.
Vazifalar bog'liqlik tartibida. Har vazifada: qaysi fayl, nima yoziladi, qanday tekshiriladi.

⚠ **Tartibni buzmang** — keyingi vazifa oldingisiga bog'liq.

---

0. FAYL STRUKTURASI (to'liq)

```
armenu/
├── manage.py
├── requirements.txt
├── requirements-dev.txt
├── .env.example
├── .gitignore
├── README.md
├── pytest.ini
├── Makefile
│
├── config/
│   ├── __init__.py
│   ├── settings/
│   │   ├── __init__.py
│   │   ├── base.py           # umumiy
│   │   ├── dev.py            # lokal
│   │   ├── prod.py           # ishlab chiqarish
│   │   └── test.py           # testlar
│   ├── urls.py
│   ├── asgi.py               # Channels
│   ├── wsgi.py
│   └── routing.py            # WS marshrutlar
│
├── core/
│   ├── models.py             # BaseModel, TimestampedModel
│   ├── mixins.py             # TenantScopedMixin ⚡
│   ├── middleware.py         # TenantMiddleware ⚡
│   ├── permissions.py        # IsStaff, IsAdmin, IsSuperAdmin
│   ├── exceptions.py         # Custom exception + handler
│   ├── pagination.py
│   ├── cache.py              # kesh yordamchilari
│   ├── resilience.py         # CircuitBreaker, retry ⚡
│   ├── queue.py              # Redis navbat
│   ├── ws.py                 # WS yordamchi (group_send)
│   ├── i18n.py               # tarjima yordamchi
│   ├── validators.py
│   └── healing/
│       ├── daemon.py         # self-healing ⚡
│       └── checks.py
│
├── apps/
│   ├── tenants/              # 1-VAZIFA
│   ├── accounts/             # 2
│   ├── menu/                 # 3
│   ├── tables/               # 4
│   ├── guest/                # 5
│   ├── orders/               # 6 ⚡
│   ├── ar3d/                 # 7 ⚡
│   ├── payments/             # 8
│   ├── fiscal/               # 9
│   ├── loyalty/              # 10
│   ├── reviews/              # 11
│   ├── analytics/            # 12
│   ├── ai/                   # 13 ⚡ MCP
│   ├── notifications/        # 14
│   └── platform_admin/       # 15
│
├── static/
│   ├── guest/{css,js,img}/
│   ├── staff/{css,js}/
│   ├── kds/{css,js}/
│   ├── admin_ui/{css,js}/
│   └── i18n/{uz,ru,en}.json   ⚡
│
├── templates/
│   ├── base.html
│   ├── guest/
│   ├── staff/
│   ├── kds/
│   └── admin_ui/
│
├── locale/{uz,ru,en}/LC_MESSAGES/
│
├── scripts/
│   ├── seed_demo.py
│   ├── import_library.py
│   └── healthcheck.py
│
├── tests/
│   ├── conftest.py
│   ├── test_tenant_isolation.py   ⚡ KRITIK
│   ├── test_orders.py
│   ├── test_ar.py
│   └── ...
│
└── deploy/
    ├── nginx/armenu.conf
    ├── systemd/*.service
    ├── docker-compose.yml
    └── ansible/
```

BOSQICH 0 — TAYYORGARLIK (3 kun, 12 vazifa)

V-001 — Loyiha yaratish

```
mkdir -p /home/user/armenu && cd /home/user/armenu
python3.12 -m venv venv && source venv/bin/activate
pip install django==5.0.* djangorestframework channels[daphne] \
    psycopg[binary] redis django-redis celery python-decouple \
    pillow qrcode[pil] reportlab drf-spectacular django-cors-headers \
    pytest pytest-django factory-boy sentry-sdk pgvector paytechuz
pip freeze > requirements.txt
django-admin startproject config .
git init && git add . && git commit -m "chore: loyiha skeleti"
```

✅ **Tekshirish:** `python manage.py runserver` ishlaydi

V-002 — Settings bo'lish

Fayl: `config/settings/{base,dev,prod,test}.py`

```
# base.py — asosiy qismlar
INSTALLED_APPS = [
    'daphne', 'django.contrib.admin', ..., 'rest_framework', 'channels',
    'corsheaders', 'drf_spectacular',
    'core', 'apps.tenants', 'apps.accounts', ...
]
MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    ...,
    'core.middleware.TenantMiddleware',        # ⚡ MUHIM
    'core.middleware.RequestIDMiddleware',
]
LANGUAGES = [('uz','O\'zbek'), ('ru','Русский'), ('en','English')]
LANGUAGE_CODE = 'uz'
USE_I18N = USE_L10N = USE_TZ = True
TIME_ZONE = 'Asia/Tashkent'
LOCALE_PATHS = [BASE_DIR / 'locale']
```

✅ `DJANGO_SETTINGS_MODULE=config.settings.dev python manage.py check`

V-003 — DB va Redis

```
sudo -u postgres psql -c "CREATE DATABASE armenu_db;"
sudo -u postgres psql -c "CREATE USER armenu WITH PASSWORD '...';"
sudo -u postgres psql -d armenu_db -c "CREATE EXTENSION IF NOT EXISTS vector;"
sudo -u postgres psql -d armenu_db -c "CREATE EXTENSION IF NOT EXISTS pg_trgm;"
```

✅ `python manage.py migrate` ishlaydi

V-004 — `core/models.py`

```
class TimestampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta: abstract = True

class TenantModel(TimestampedModel):
    restaurant = models.ForeignKey('tenants.Restaurant', on_delete=models.CASCADE, db_index=True)
    class Meta: abstract = True
```

V-005 — ⚡ `core/middleware.py` (KRITIK)

```
class TenantMiddleware:
    """Har so'rovda restaurant_id ni aniqlaydi. Multi-tenant xavfsizligining asosi."""
    def __call__(self, request):
        request.restaurant_id = None
        request.restaurant = None

        # 1. Custom domen
        host = request.get_host().split(':')[0]
        if host not in settings.PLATFORM_DOMAINS:
            r = Restaurant.objects.filter(custom_domain=host, is_active=True).first()
            if r: request.restaurant_id, request.restaurant = r.id, r

        # 2. URL'dagi slug: /m/<slug>/
        if not request.restaurant_id:
            m = re.match(r'^/m/([\w-]+)/', request.path)
            if m:
                r = cache_get_restaurant_by_slug(m.group(1))
                if r: request.restaurant_id, request.restaurant = r.id, r

        # 3. JWT ichidagi restaurant_id (xodim/admin)
        if not request.restaurant_id and hasattr(request, 'auth'):
            request.restaurant_id = getattr(request.auth, 'restaurant_id', None)

        return self.get_response(request)
```

V-006 — ⚡ `core/mixins.py`

```
class TenantScopedMixin:
    def get_queryset(self):
        qs = super().get_queryset()
        if not self.request.restaurant_id:
            raise PermissionDenied("Restoran aniqlanmadi")
        return qs.filter(restaurant_id=self.request.restaurant_id)
```

V-007 — `core/exceptions.py`

Barcha xato kodlari (Qism IV §2.2) + DRF `exception_handler` (3 tilda xabar).

V-008 — `core/resilience.py`

`CircuitBreaker` + `retry_with_backoff` (Qism III §5.2).

V-009 — `core/queue.py`

```
def enqueue(queue: str, job_type: str, delay: int = 0, **payload) -> str:
    job = Job.objects.create(queue=queue, type=job_type, payload=payload, status='queued')
    r = get_redis()
    if delay: r.zadd(f'q:{queue}:delayed', {str(job.id): time.time() + delay})
    else:     r.lpush(f'q:{queue}', str(job.id))
    return str(job.id)
```

V-010 — `core/ws.py`

```
def ws_send(group: str, msg_type: str, data: dict):
    layer = get_channel_layer()
    async_to_sync(layer.group_send)(group, {
        'type': 'broadcast', 'message': {
            'type': msg_type, 'ts': timezone.now().isoformat(),
            'data': data, 'seq': next_seq(group)
        }})
```

V-011 — `core/cache.py`

```
def menu_cache_key(slug, lang, version): return f'menu:{slug}:{lang}:v{version}'
def bump_menu_version(restaurant_id):
    Restaurant.objects.filter(id=restaurant_id).update(menu_version=F('menu_version')+1)
    cache.delete_pattern(f'menu:*')
```

V-012 — `tests/conftest.py`

Fixture'lar: `restaurant`, `restaurant2` (izolyatsiya testi uchun), `table`, `dish`, `staff`, `guest_session`.

BOSQICH 1 — TENANT VA AUTH (1 hafta, 18 vazifa)

#	Vazifa	Fayl	Bog'liq
V-013	`Restaurant` modeli (Qism IV §1.2)	`apps/tenants/models.py`	V-004
V-014	`Subscription`, `Plan` modellari	`apps/tenants/models.py`	V-013
V-015	Migratsiya + super-admin	—	V-014
V-016	`RestaurantSerializer`	`apps/tenants/serializers.py`	V-013
V-017	Tarif limitlari servisi	`apps/tenants/services.py`	V-014
V-018	`User` modeli (AbstractUser)	`apps/accounts/models.py`	V-004
V-019	`Staff` modeli (rol, PIN)	`apps/accounts/models.py`	V-018
V-020	PIN hash (bcrypt) + tekshirish	`apps/accounts/services.py`	V-019
V-021	JWT generatsiya (restaurant_id ichida ⚡)	`apps/accounts/tokens.py`	V-019
V-022	`POST /v1/staff/login/`	`apps/accounts/views.py`	V-021
V-023	Rate-limit (5 xato → 15 daq blok)	`apps/accounts/throttles.py`	V-022
V-024	`core/permissions.py`: IsWaiter, IsCook, IsCashier, IsRestAdmin	—	V-021
V-025	Admin login (email+parol)	`apps/accounts/views.py`	V-018
V-026	Super-admin 2FA (TOTP)	`apps/accounts/views.py`	V-025
V-027	Audit log modeli + `AuditLog.log()`	`apps/analytics/models.py`	V-004
V-028	⚡ **TEST: cross-tenant kirish 403**	`tests/test_tenant_isolation.py`	V-006
V-029	`POST /v1/staff/refresh/`	`apps/accounts/views.py`	V-021
V-030	Django admin ro'yxatga olish	`*/admin.py`	—

⚡ V-028 — eng muhim test

```
@pytest.mark.django_db
def test_cross_tenant_dish_access_forbidden(api_client, restaurant, restaurant2, staff_token):
    """Restoran A xodimi restoran B taomiga kira olmasligi SHART"""
    dish_b = DishFactory(restaurant=restaurant2)
    r = api_client.get(f'/v1/admin/dishes/{dish_b.id}/',
                       HTTP_AUTHORIZATION=f'Bearer {staff_token}')  # A ning tokeni
    assert r.status_code in (403, 404)      # ⚡ HECH QACHON 200 EMAS

@pytest.mark.parametrize('model_name', ALL_TENANT_MODELS)
def test_all_models_tenant_scoped(model_name, ...):
    """Har bir tenant modeli uchun avtomatik tekshiruv"""
    ...
```

BOSQICH 2 — MENYU (2 hafta, 24 vazifa)

#	Vazifa	Fayl
V-031	`Category` modeli (3 til)	`apps/menu/models.py`
V-032	`Dish` modeli (to'liq, Qism IV)	`apps/menu/models.py`
V-033	`DishModifier` modeli	`apps/menu/models.py`
V-034	`UpsellRule` modeli	`apps/menu/models.py`
V-035	Migratsiya + indekslar (GIN)	—
V-036	⚡ `core/i18n.py` : `t(obj, field, lang)`	`core/i18n.py`
V-037	Rasm optimizatsiyasi (WebP, 3 o'lcham)	`apps/menu/services/images.py`
V-038	`MenuSerializer` (kesh uchun tekis)	`apps/menu/serializers.py`
V-039	⚡ `GET /v1/menu/{slug}/` + Redis kesh	`apps/menu/views.py`
V-040	ETag + Cache-Control	`apps/menu/views.py`
V-041	`GET /v1/menu/{slug}/search/` (pg_trgm)	`apps/menu/views.py`
V-042	`GET /v1/dish/{id}/`	`apps/menu/views.py`
V-043	Kesh invalidation (signal)	`apps/menu/signals.py`
V-044	Admin CRUD: kategoriya	`apps/menu/admin_views.py`
V-045	Admin CRUD: taom	`apps/menu/admin_views.py`
V-046	Stop-list toggle	`apps/menu/admin_views.py`
V-047	Excel/CSV import	`apps/menu/services/import.py`
V-048	Tartib o'zgartirish (drag&drop)	`apps/menu/admin_views.py`
V-049	Allergen/diet filtri	`apps/menu/filters.py`
V-050	Upsell qoidalari CRUD	`apps/menu/admin_views.py`
V-051	IKPU kod maydoni + validatsiya	`apps/menu/validators.py`
V-052	`tasnif.soliq.uz` API qidiruvi	`apps/fiscal/services/ikpu.py`
V-053	Test: menyu API, kesh, filtr	`tests/test_menu.py`
V-054	Test: 3 til to'g'ri qaytadi	`tests/test_i18n.py`

⚡ V-036 — tarjima yordamchi (har joyda ishlatiladi)

```
# core/i18n.py
def t(obj, field: str, lang: str = 'uz') -> str:
    """
    t(dish, 'name', 'ru') → dish.name_ru yoki fallback dish.name_uz
    """
    val = getattr(obj, f'{field}_{lang}', None)
    if val: return val
    for fb in ('uz', 'ru', 'en'):                # fallback zanjiri
        val = getattr(obj, f'{field}_{fb}', None)
        if val: return val
    return ''
```

BOSQICH 3 — STOLLAR VA QR (1 hafta, 12 vazifa)

#	Vazifa	Fayl
V-055	`Hall` modeli	`apps/tables/models.py`
V-056	⚡ `Table` modeli (`qr_token` UUID unique)	`apps/tables/models.py`
V-057	Migratsiya	—
V-058	⚡ QR generatsiya (qrcode, ECC=H, logo)	`apps/tables/services/qr.py`
V-059	⚡ QR PDF (ReportLab, A4 1/2/4/6)	`apps/tables/services/pdf.py`
V-060	Admin CRUD: zal	`apps/tables/admin_views.py`
V-061	Admin CRUD: stol	`apps/tables/admin_views.py`
V-062	`POST /tables/{id}/regen-qr/` (grace 24s)	`apps/tables/admin_views.py`
V-063	`GET /tables/qr-pdf/`	`apps/tables/admin_views.py`
V-064	Zal plani saqlash (`plan_json`)	`apps/tables/admin_views.py`
V-065	Stol holati o'zgartirish + WS	`apps/tables/services/status.py`
V-066	Test: QR noyob, cross-tenant yo'q	`tests/test_tables.py`

V-058 — QR generatsiya kodi

```
# apps/tables/services/qr.py
import qrcode
from qrcode.image.styledpil import StyledPilImage
from qrcode.image.styles.moduledrawers import RoundedModuleDrawer

def generate_qr(table, logo_path=None):
    url = f"https://{settings.PUBLIC_DOMAIN}/t/{table.qr_token}"
    qr = qrcode.QRCode(version=None, error_correction=qrcode.ERROR_CORRECT_H,
                       box_size=12, border=2)
    qr.add_data(url); qr.make(fit=True)
    img = qr.make_image(image_factory=StyledPilImage,
                        module_drawer=RoundedModuleDrawer(),
                        embeded_image_path=logo_path)
    return img          # 300 DPI PNG
```

BOSQICH 4 — MIJOZ OQIMI (2 hafta, 26 vazifa)

#	Vazifa	Fayl
V-067	`Guest`, `GuestSession` modellari	`apps/guest/models.py`
V-068	⚡ `GET /t/{qr_token}` (M-01 zanjiri)	`apps/guest/views.py`
V-069	Device token cookie (1 yil)	`apps/guest/services.py`
V-070	Til aniqlash (4 bosqichli)	`core/i18n.py`
V-071	Menyu sahifasi shabloni	`templates/guest/menu.html`
V-072	⚡ `static/i18n/{uz,ru,en}.json` (barcha matn)	`static/i18n/`
V-073	JS i18n yuklovchi	`static/guest/js/i18n.js`
V-074	Menyu JS: yuklash, render, virtual scroll	`static/guest/js/menu.js`
V-075	Qidiruv (debounce 300ms)	`static/guest/js/menu.js`
V-076	Kategoriya + diet filtri	`static/guest/js/menu.js`
V-077	Taom modali (M-04)	`static/guest/js/dish.js`
V-078	Modifikatorlar + narx hisobi	`static/guest/js/dish.js`
V-079	Savat (localStorage)	`static/guest/js/cart.js`
V-080	⚡ Upsell mantiq	`static/guest/js/upsell.js`
V-081	Savat sahifasi (M-06)	`templates/guest/cart.html`
V-082	⚡ Dark + Light tema (CSS o'zgaruvchilar)	`static/guest/css/theme.css`
V-083	PWA manifest + service worker	`static/guest/sw.js`
V-084	Lazy image (IntersectionObserver)	`static/guest/js/lazy.js`
V-085	Offline rejim (cache-first)	`static/guest/sw.js`
V-086	Chaqiruv UI (M-09)	`static/guest/js/calls.js`
V-087	`POST /v1/calls/` + cooldown	`apps/orders/views.py`
V-088	Buyurtma holati sahifasi (M-08)	`templates/guest/order_status.html`
V-089	WS klient (ReliableWS)	`static/guest/js/ws.js`
V-090	Toast/bildirishnoma tizimi	`static/guest/js/toast.js`
V-091	Test: QR→menyu→savat oqimi	`tests/test_guest_flow.py`
V-092	Lighthouse audit (>90 ball)	—

BOSQICH 5 — BUYURTMA VA REAL-TIME (2 hafta, 22 vazifa)

#	Vazifa	Fayl
V-093	`Order` , `OrderItem` modellari	`apps/orders/models.py`
V-094	⚡ Holat mashinasi (`state.py`)	`apps/orders/state.py`
V-095	Kunlik buyurtma raqami (`#047`)	`apps/orders/services.py`
V-096	⚡ `create_order()` (Qism IV §2.4)	`apps/orders/services.py`
V-097	⚡ Idempotency (Redis)	`core/idempotency.py`
V-098	`POST /v1/orders/`	`apps/orders/views.py`
V-099	`GET /v1/orders/{id}/`	`apps/orders/views.py`
V-100	`POST /v1/orders/{id}/add-items/`	`apps/orders/views.py`
V-101	WS consumer: guest	`apps/orders/consumers.py`
V-102	WS consumer: staff	`apps/orders/consumers.py`
V-103	WS consumer: KDS	`apps/orders/consumers.py`
V-104	WS consumer: admin	`apps/orders/consumers.py`
V-105	`config/routing.py`	—
V-106	Heartbeat + reconnect (server)	`apps/orders/consumers.py`
V-107	⚡ `GET /v1/sync/?since_seq=`	`apps/orders/views.py`
V-108	Ofitsiant paneli sahifasi (O-02)	`templates/staff/orders.html`
V-109	Ofitsiant JS + WS + tovush	`static/staff/js/orders.js`
V-110	`POST /staff/orders/{id}/accept|serve/`	`apps/orders/staff_views.py`
V-111	⚡ KDS sahifasi (K-01, dark, katta)	`templates/kds/board.html`
V-112	KDS JS (kanban, taymer, ranglar)	`static/kds/js/board.js`
V-113	`POST /staff/orders/{id}/cook|ready/`	`apps/orders/staff_views.py`
V-114	Test: to'liq buyurtma oqimi + WS	`tests/test_order_flow.py`

BOSQICH 6 — ⚡ AR/3D MODUL (2,5 hafta, 28 vazifa) — ASOSIY

#	Vazifa	Fayl
V-115	`DishLibrary` modeli (pgvector)	`apps/ar3d/models.py`
V-116	`DishModel3D` modeli	`apps/ar3d/models.py`
V-117	`ARView` modeli (partition)	`apps/ar3d/models.py`
V-118	`Model3DJob` modeli	`apps/ar3d/models.py`
V-119	Migratsiya + pgvector indeks	—
V-120	⚡ `GET /v1/ar/{dish_id}/`	`apps/ar3d/views.py`
V-121	⚡ AR qo'llab-quvvatlash aniqlash (JS)	`static/guest/js/ar-detect.js`
V-122	⚡ `<model-viewer>` integratsiyasi	`static/guest/js/ar.js`
V-123	AR modal UI + progress	`templates/guest/ar_modal.html`
V-124	⚡ Fallback zanjiri (4 bosqich)	`static/guest/js/ar.js`
V-125	`POST /v1/ar/view/` (analitika)	`apps/ar3d/views.py`
V-126	⚡ Kutubxona qidiruvi (nom+alias)	`apps/ar3d/services/library.py`
V-127	⚡ Kutubxona semantik qidiruv (embedding)	`apps/ar3d/services/library.py`
V-128	Kutubxonadan nusxa olish	`apps/ar3d/services/library.py`
V-129	⚡ AI 3D generatsiya (MCP orqali)	`apps/ar3d/services/generate.py`
V-130	⚡ GLB optimizatsiya (gltf-transform)	`apps/ar3d/services/optimize.py`
V-131	⚡ Real o'lchamga masshtablash	`apps/ar3d/services/optimize.py`
V-132	⚡ USDZ konvertatsiya	`apps/ar3d/services/usdz.py`
V-133	Poster render (headless)	`apps/ar3d/services/poster.py`
V-134	CDN/S3 yuklash	`apps/ar3d/services/storage.py`
V-135	⚡ Worker: `run_model3d_worker`	`apps/ar3d/management/commands/`
V-136	Kutubxonaga qo'shish (avto)	`apps/ar3d/services/library.py`
V-137	`POST /v1/admin/ar/generate/`	`apps/ar3d/admin_views.py`
V-138	`POST /v1/admin/ar/upload/`	`apps/ar3d/admin_views.py`
V-139	Admin 3D sahifasi (A-04)	`templates/admin_ui/ar.html`
V-140	⚡ AR analitika (A-10)	`apps/ar3d/analytics.py`
V-141	Super-admin kutubxona (P-02)	`templates/admin_ui/library.html`
V-142	Test: AR API, fallback, o'lcham	`tests/test_ar.py`

⚡ V-130 — GLB optimizatsiya (kritik kod)

```
# apps/ar3d/services/optimize.py
import subprocess, json, os

def optimize_glb(input_path: str, output_path: str, real_size_cm: float) -> dict:
    """
    1. Draco siqish  2. Tekstura 1024px  3. Poligon ≤60k
    4. ⚡ Real o'lchamga masshtablash  5. Markazni Y=0 ga
    """
    tmp = output_path + '.tmp.glb'

    # gltf-transform pipeline
    cmds = [
        ['gltf-transform', 'resize', input_path, tmp, '--width','1024','--height','1024'],
        ['gltf-transform', 'simplify', tmp, tmp, '--ratio','0.5','--error','0.001'],
        ['gltf-transform', 'webp', tmp, tmp, '--quality','85'],
        ['gltf-transform', 'draco', tmp, output_path],
    ]
    for c in cmds:
        subprocess.run(c, check=True, timeout=300)

    # ⚡ Real o'lchamga masshtablash va markazlash (pygltflib)
    scale_and_center(output_path, real_size_cm / 100.0)   # sm → m

    size = os.path.getsize(output_path)
    if size > 3 * 1024 * 1024:                          # 3 MB dan katta
        aggressive_optimize(output_path)                 # qayta, agressiv
        size = os.path.getsize(output_path)

    return {'bytes': size, 'ok': size <= 3*1024*1024}
```

⚡ V-131 — real o'lcham (AR ning eng muhim qismi)

```
def scale_and_center(glb_path: str, target_size_m: float):
    """
    Model AR'da 1:1 chiqishi uchun:
      1. Bounding box hisoblanadi
      2. Eng katta o'lcham target_size_m ga tenglashtiriladi
      3. Model tagi Y=0 ga tushiriladi (stolga to'g'ri qo'yiladi)
    """
    gltf = GLTF2().load(glb_path)
    bbox = compute_bbox(gltf)
    current_max = max(bbox.size.x, bbox.size.z)   # gorizontal o'lcham
    scale = target_size_m / current_max

    root = gltf.nodes[gltf.scenes[0].nodes[0]]
    root.scale = [scale, scale, scale]
    root.translation = [
        -bbox.center.x * scale,
        -bbox.min.y   * scale,      # ⚡ tagi Y=0
        -bbox.center.z * scale,
    ]
    gltf.save(glb_path)
```

BOSQICH 7 — ZAL PLANI (1 hafta, 8 vazifa)

#	Vazifa	Fayl
V-143	Zal redaktor (drag&drop, canvas)	`static/admin_ui/js/hall-editor.js`
V-144	Stol shakli, o'lchami, burilishi	`static/admin_ui/js/hall-editor.js`
V-145	Devor/bar/eshik chizish	`static/admin_ui/js/hall-editor.js`
V-146	`plan_json` saqlash/yuklash	`apps/tables/admin_views.py`
V-147	Ofitsiant 2D plan ko'rinishi (O-05)	`static/staff/js/hall-view.js`
V-148	Stol rangi (holat) + WS yangilanish	`static/staff/js/hall-view.js`
V-149	Admin jonli zal ko'rinishi	`static/admin_ui/js/hall-live.js`
V-150	Test	`tests/test_halls.py`

> 📌 **v3 tayyorgarligi:** `plan_json` ga `z`, `height`, `model` maydonlari qo'shilsa — bu ma'lumot to'g'ridan-to'g'ri 3D zalga aylanadi.

BOSQICH 8 — ADMIN PANEL (1,5 hafta, 20 vazifa)

#	Vazifa
V-151	Admin layout + navigatsiya (dark/light)
V-152	⚡ Dashboard (A-01) + WS jonli oqim
V-153	Ko'rsatkich kartochkalari
V-154	Soatlik savdo grafigi (Chart.js)
V-155	TOP taomlar
V-156	Buyurtmalar ro'yxati + filtr
V-157	Buyurtma tafsiloti
V-158	Savdo hisoboti (kun/hafta/oy)
V-159	Taomlar hisoboti
V-160	⚡ Menyu-injiniring (4 kvadrant)
V-161	Xodimlar hisoboti
V-162	Excel/PDF eksport
V-163	Xodimlar CRUD
V-164	Sozlamalar sahifasi
V-165	Brending (logo, rang)
V-166	⚡ Tarjima boshqaruvi (A-15)
V-167	Tarif va to'lov sahifasi
V-168	Ma'lumot eksporti
V-169	Mobil moslashuv
V-170	Test

BOSQICH 9 — AI VA MCP (1,5 hafta, 16 vazifa)

> Batafsil: **Qism VII — AI/MCP integratsiyasi**

#	Vazifa	Fayl
V-171	⚡ MCP baza qatlami	`apps/ai/mcp/base.py`
V-172	Provayder: Claude (Anthropic)	`apps/ai/mcp/providers/claude.py`
V-173	Provayder: Gemini	`apps/ai/mcp/providers/gemini.py`
V-174	Provayder: OpenAI	`apps/ai/mcp/providers/openai.py`
V-175	Provayder: Groq	`apps/ai/mcp/providers/groq.py`
V-176	Provayder: DeepSeek	`apps/ai/mcp/providers/deepseek.py`
V-177	Provayder: Meshy (3D)	`apps/ai/mcp/providers/meshy.py`
V-178	Provayder: Tripo (3D)	`apps/ai/mcp/providers/tripo.py`
V-179	⚡ Router (prioritet + fallback)	`apps/ai/mcp/router.py`
V-180	Circuit breaker har provayderga	`apps/ai/mcp/router.py`
V-181	`AIUsage` + xarajat hisobi	`apps/ai/models.py`
V-182	⚡ Chatbot servisi	`apps/ai/services/chat.py`
V-183	`POST /v1/ai/chat/`	`apps/ai/views.py`
V-184	Chatbot UI (M-13)	`static/guest/js/chat.js`
V-185	⚡ Avto-tarjima servisi	`apps/ai/services/translate.py`
V-186	Test: fallback zanjiri	`tests/test_ai.py`

BOSQICH 10 — TO'LOV VA FISKAL (v1.5, 2 hafta, 18 vazifa)

#	Vazifa	Fayl
V-187	`Payment`, `Tip` modellari	`apps/payments/models.py`
V-188	`paytechuz` integratsiyasi	`apps/payments/providers/`
V-189	Payme adapter	`apps/payments/providers/payme.py`
V-190	Click adapter	`apps/payments/providers/click.py`
V-191	Uzum adapter	`apps/payments/providers/uzum.py`
V-192	`POST /v1/payments/`	`apps/payments/views.py`
V-193	⚡ Webhook + imzo tekshiruvi	`apps/payments/webhooks.py`
V-194	Idempotency (to'lov)	`apps/payments/services.py`
V-195	Hisobni bo'lish (split bill)	`apps/payments/services.py`
V-196	⚡ Choychaqa (tips)	`apps/payments/views.py`
V-197	Reconciliation cron	`apps/payments/tasks.py`
V-198	`FiscalReceipt` modeli	`apps/fiscal/models.py`
V-199	OFD adapter: UZKASSA	`apps/fiscal/providers/uzkassa.py`
V-200	OFD adapter: e-POS	`apps/fiscal/providers/epos.py`
V-201	⚡ Fiskal worker (retry 48s)	`apps/fiscal/tasks.py`
V-202	IKPU tekshiruvi	`apps/fiscal/services/ikpu.py`
V-203	Raqamli chek (QR + Telegram)	`apps/fiscal/services/receipt.py`
V-204	Test: to'lov + fiskal	`tests/test_payments.py`

BOSQICH 11 — SODIQLIK, OTZIV, ANALITIKA (1,5 hafta, 14 vazifa)

#	Vazifa
V-205	`Bonus`, `Promo` modellari
V-206	Keshbek hisoblash
V-207	Promokod qo'llash
V-208	Tug'ilgan kun bonusi (cron)
V-209	`Review` modeli
V-210	⚡ Otziv filtri (5★→Google)
V-211	Otziv UI (M-12)
V-212	Admin otzivlar (A-12)
V-213	`AnalyticsEvent` → ClickHouse
V-214	Event yig'ish (batch)
V-215	Menyu-injiniring hisobi
V-216	Kunlik agregatsiya (cron)
V-217	Telegram bildirishnoma
V-218	Test

BOSQICH 12 — DEPLOY VA KUZATUV (1 hafta, 16 vazifa)

#	Vazifa	Fayl
V-219	`deploy/nginx/armenu.conf`	—
V-220	`armenu.service` (gunicorn)	—
V-221	`armenu-ws.service` (daphne)	—
V-222	`armenu-worker@.service` (5 tur)	—
V-223	⚡ `armenu-healer.service`	—
V-224	Socket activation (zero-downtime)	—
V-225	Health endpoint `/health/`, `/ready/`	`core/views.py`
V-226	Prometheus metrikalar	`core/metrics.py`
V-227	Grafana dashboard	`deploy/grafana/`
V-228	Sentry integratsiyasi	`config/settings/prod.py`
V-229	Log (JSON, Loki)	`config/settings/prod.py`
V-230	Backup skript (cron)	`scripts/backup.sh`
V-231	Backup tekshirish (cron)	`scripts/verify_backup.sh`
V-232	CI/CD (GitHub Actions)	`.github/workflows/`
V-233	⚡ CI'da tenant izolyatsiya testi majburiy	—
V-234	Yuklama testi (k6/locust)	`tests/load/`

📊 VAZIFALAR XULOSASI

Bosqich	Vazifa	Muddat
0. Tayyorgarlik	12	3 kun
1. Tenant + Auth	18	1 hafta
2. Menyu	24	2 hafta
3. Stollar + QR	12	1 hafta
4. Mijoz oqimi	26	2 hafta
5. Buyurtma + real-time	22	2 hafta
6. ⚡ AR/3D	28	2,5 hafta
7. Zal plani	8	1 hafta
8. Admin panel	20	1,5 hafta
9. AI + MCP	16	1,5 hafta
v1 JAMI	**186**	**~15 hafta**
10. To'lov + fiskal (v1.5)	18	2 hafta
11. Sodiqlik + analitika	14	1,5 hafta
12. Deploy + kuzatuv	16	1 hafta
v1.5 JAMI	**234**	**~19,5 hafta**

✅ HAR VAZIFA UCHUN "TUGALLANGAN" TA'RIFI

Vazifa faqat quyidagilar bajarilgandagina tugallangan hisoblanadi:

1. ✅ Kod yozilgan va ishlaydi
2. ✅ Test yozilgan va o'tadi
3. ✅ ⚡ **Tenant izolyatsiyasi tekshirilgan** (agar tenant modeli bo'lsa)
4. ✅ 3 til qo'llab-quvvatlanadi (agar UI bo'lsa)
5. ✅ Dark + Light tema (agar UI bo'lsa)
6. ✅ Xato holatlari qayta ishlangan
7. ✅ Log/metrika qo'shilgan (agar muhim bo'lsa)
8. ✅ API hujjati yangilangan (drf-spectacular)

9. ✅ Kod ko'rib chiqilgan (review)

QISM VI

Tarjima tizimi va UI/UX

1. 🌐 3 TILLI TIZIM (uz / ru / en)

> Talab: "*3 ta tilda bo'lsin — o'zbek, rus, ingliz. Har bir harf, matn tarjima qilinishi kerak.*"

1.1 Tarjima qamrovi (100%)

Nima	Qayerda	Usul	Holat
Tugma, yorliq, xabar (UI)	Django + JS	Statik lug'at	Qo'lda, 100%
Xato xabarlari	Backend	Statik lug'at	Qo'lda, 100%
Taom nomi, tavsifi	DB (3 ustun)	Admin yoki AI	Aralash
Kategoriya nomi	DB (3 ustun)	Admin yoki AI	Aralash
Zal nomi	DB (3 ustun)	Admin yoki AI	Aralash
Modifikator nomi	DB (3 ustun)	Admin yoki AI	Aralash
Allergen/diet teglari	Statik lug'at	Qo'lda	100%
AI chatbot javobi	Real-time	Generatsiya	Avtomatik
Email/SMS/Telegram	Shablon	Qo'lda	100%
PDF (QR, chek, hisobot)	Shablon	Qo'lda	100%

1.2 Uch qatlamli arxitektura

```
┌──────────────────────────────────────────────────────┐
│ QATLAM 1 — STATIK INTERFEYS (100% qo'lda tarjima)    │
├──────────────────────────────────────────────────────┤
│ Backend:  locale/{uz,ru,en}/LC_MESSAGES/django.po    │
│           {% trans "Buyurtma berish" %}              │
│           gettext("Savat bo'sh")                     │
│                                                      │
│ Frontend: static/i18n/{uz,ru,en}.json                │
│           t('cart.empty')  →  "Savat bo'sh"          │
│                                                      │
│ ⚡ Sifat kafolati: har bir kalit 3 tilda MAVJUD bo'lishi │
│    SHART. CI'da tekshiriladi — bo'sh bo'lsa build FAIL. │
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│ QATLAM 2 — DINAMIK KONTENT (DB, 3 ustun)             │
├──────────────────────────────────────────────────────┤
│ Dish.name_uz / name_ru / name_en                     │
│ Dish.desc_uz / desc_ru / desc_en                     │
│                                                      │
│ Fallback: so'ralgan til bo'sh → uz → ru → en → ''    │
│                                                      │
│ 🤖 AI to'ldirish: bo'sh maydonlar avtomatik tarjima  │
│    → tr_auto={"ru":true} belgisi                     │
│    → admin tasdiqlaydi → tr_approved={"ru":true}     │
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│ QATLAM 3 — AI GENERATSIYA (real-time)                │
├──────────────────────────────────────────────────────┤
│ Chatbot javobi foydalanuvchi tilida GENERATSIYA qilinadi │
│ (tarjima emas — to'g'ridan-to'g'ri o'sha tilda yoziladi) │
└──────────────────────────────────────────────────────┘
```

1.3 `static/i18n/uz.json` — namuna (to'liq tuzilma)

```
{
  "common": {
    "back": "Orqaga", "close": "Yopish", "save": "Saqlash",
    "cancel": "Bekor qilish", "confirm": "Tasdiqlash", "delete": "O'chirish",
    "edit": "Tahrirlash", "add": "Qo'shish", "search": "Qidirish",
    "loading": "Yuklanmoqda...", "error": "Xatolik", "retry": "Qayta urinish",
    "yes": "Ha", "no": "Yo'q", "all": "Hammasi", "sum": "so'm",
```

```
    "min": "daqiqa", "gram": "g", "pcs": "ta"
  },
  "menu": {
    "title": "Menyu", "search_placeholder": "Taom qidirish...",
    "no_results": "Hech narsa topilmadi", "category_all": "Hammasi",
    "sold_out": "Tugagan", "hit": "Ommabop", "new": "Yangi",
    "cook_time": "{n} daqiqa", "weight": "{n} g", "calories": "{n} kkal"
  },
  "dish": {
    "ingredients": "Tarkibi", "allergens": "Allergenlar",
    "no_allergens": "Allergen yo'q", "modifiers": "Qo'shimchalar",
    "note_placeholder": "Izoh (masalan: achchiq qilmang)",
    "add_to_cart": "Savatga qo'shish", "view_ar": "AR'da ko'rish",
    "view_3d": "3D ko'rish", "rating": "{n} ({cnt} ta baho)"
  },
  "ar": {
    "button": "📱 Stolga qo'yib ko'rish",
    "loading": "3D model yuklanmoqda...",
    "not_supported": "Qurilmangiz AR'ni qo'llab-quvvatlamaydi",
    "not_supported_hint": "3D ko'rinishda aylantirib ko'rishingiz mumkin",
    "error": "3D model yuklanmadi",
    "instruction": "Telefoningizni stolga qarating",
    "real_size": "Haqiqiy o'lchamda",
    "preparing": "3D model tayyorlanmoqda..."
  },
  "cart": {
    "title": "Savat", "empty": "Savat bo'sh",
    "empty_hint": "Menyudan taom tanlang",
    "subtotal": "Taomlar", "service_fee": "Xizmat ({pct}%)",
    "discount": "Chegirma", "total": "JAMI",
    "note_placeholder": "Umumiy izoh",
    "submit": "Buyurtma berish", "cross_sell": "Bularni ham oling"
  },
  "order": {
    "created": "Buyurtmangiz qabul qilindi!",
    "number": "Buyurtma {n}", "table": "Stol {n}",
    "status_new": "Qabul qilindi", "status_accepted": "Qabul qilindi",
    "status_cooking": "Tayyorlanmoqda", "status_ready": "Tayyor",
    "status_served": "Berildi", "status_paid": "To'landi",
    "eta": "Taxminan {n} daqiqa qoldi",
    "ready_title": "🎉 Buyurtmangiz tayyor!",
    "add_more": "Yana buyurtma", "request_bill": "Hisob so'rash",
    "unavailable": "{dish} tugadi. Ofitsiant keladi."
  },
  "call": {
    "title": "Nima kerak?", "waiter": "Ofitsiant", "bill": "Hisob",
    "water": "Suv", "clean": "Tozalash",
    "sent": "Ofitsiant chaqirildi ✓",
    "cooldown": "Iltimos, biroz kuting",
    "resolved": "Ofitsiant keldi"
  },
  "payment": {
    "title": "To'lov", "method": "To'lov usuli", "cash": "Naqd (ofitsiantga)",
    "payme": "Payme", "click": "Click", "uzum": "Uzum",
    "use_bonus": "Bonusdan foydalanish", "pay": "To'lash — {amount}",
    "success": "✅ To'lov qabul qilindi", "failed": "To'lov amalga oshmadi",
    "split": "Hisobni bo'lish", "receipt": "Chek"
  },
  "tips": {
    "title": "Xizmat yoqdimi?", "waiter_label": "Sizning ofitsiantingiz",
    "give": "💝 Choychaqa berish", "skip": "O'tkazib yuborish",
    "custom": "Boshqa summa", "thanks": "Rahmat! 🎉"
  },
  "review": {
    "title": "Bizni baholang", "food": "Taom", "service": "Xizmat",
    "ambience": "Muhit", "comment_placeholder": "Izohingiz...",
    "submit": "Yuborish", "thanks": "Rahmat! 🎉",
    "google_prompt": "Google'da ham baholaysizmi?",
    "google_button": "Google'ga o'tish",
    "sorry": "Uzr so'raymiz. Nima yoqmadi?"
  },
  "chat": {
    "title": "Yordamchi", "greeting": "Salom! Menyu bo'yicha savolingiz bormi?",
    "placeholder": "Savolingizni yozing...", "thinking": "O'ylayapman...",
    "error": "Kechirasiz, hozir javob bera olmayman. Ofitsiantni chaqiring.",
    "view_dish": "Ko'rish", "order_dish": "Buyurtma"
  },
  "errors": {
    "qr_invalid": "QR kod yaroqsiz. Ofitsiantga murojaat qiling.",
    "restaurant_inactive": "Restoran vaqtincha xizmat ko'rsatmayapti",
    "dish_unavailable": "Bu taom hozir mavjud emas",
    "price_changed": "Narxlar yangilandi. Savatni tekshiring.",
    "network": "Ulanish yo'q. Qayta urinilmoqda...",
    "server": "Xatolik yuz berdi. Ofitsiantni chaqiring.",
    "rate_limited": "Juda ko'p so'rov. Biroz kuting."
  },
  "staff": {
    "login": "Xodim kirishi", "pin": "PIN kod", "wrong_pin": "PIN noto'g'ri",
    "blocked": "Bloklandi. {n} daqiqadan keyin urinib ko'ring.",
    "new_orders": "Yangi buyurtmalar", "my_tables": "Mening stollarim",
    "hall": "Zal", "calls": "Chaqiruvlar", "stats": "Statistika",
    "accept": "✅ Qabul qildim", "serve": "🍽 Olib bordim",
    "shift_start": "Smena boshlash", "shift_end": "Smena tugatish",
    "today": "Bugun", "tables_served": "{n} stol", "tips_earned": "Choychaqa"
  },
  "kds": {
    "title": "Oshxona", "new": "YANGI", "cooking": "TAYYORLANMOQDA",
    "ready": "TAYYOR", "start": "BOSHLASH", "mark_ready": "TAYYOR ✓",
    "served": "BERILDI", "late": "KECHIKDI", "avg_time": "O'rtacha"
  },
  "admin": {
```

```
    "dashboard": "Dashboard", "menu": "Menyu", "tables": "Stollar",
    "orders": "Buyurtmalar", "staff": "Xodimlar", "reports": "Hisobotlar",
    "ar_models": "3D/AR modellar", "settings": "Sozlamalar",
    "today_revenue": "Bugungi savdo", "orders_count": "Buyurtma",
    "avg_check": "O'rtacha chek", "tables_busy": "Band stollar",
    "ar_views": "AR ko'rish"
  },
  "diet": {
    "halal": "Halol", "vegetarian": "Vegetarian", "vegan": "Vegan",
    "spicy": "Achchiq", "no_spicy": "Achchiq emas",
    "lactose_free": "Laktozasiz", "gluten_free": "Glutensiz"
  },
  "allergen": {
    "nuts": "Yong'oq", "gluten": "Gluten", "dairy": "Sut mahsulotlari",
    "eggs": "Tuxum", "fish": "Baliq", "shellfish": "Dengiz mahsulotlari",
    "soy": "Soya", "sesame": "Kunjut"
  }
}
```

⚡ `ru.json` va `en.json` — **aynan shu tuzilma**, faqat qiymatlar tarjima qilingan.
CI tekshiruvi: 3 fayl kalitlari **bir xil** bo'lishi shart.

1.4 JS tarjima yordamchi

```
// static/guest/js/i18n.js
let TRANSLATIONS = {};
let CURRENT_LANG = 'uz';

async function loadI18n(lang) {
  CURRENT_LANG = lang;
  const cached = localStorage.getItem(`i18n_${lang}_v${I18N_VERSION}`);
  if (cached) { TRANSLATIONS = JSON.parse(cached); return; }

  const r = await fetch(`/static/i18n/${lang}.json?v=${I18N_VERSION}`);
  TRANSLATIONS = await r.json();
  localStorage.setItem(`i18n_${lang}_v${I18N_VERSION}`, JSON.stringify(TRANSLATIONS));
}

/**
 * t('cart.empty')            → "Savat bo'sh"
 * t('order.eta', {n: 18})    → "Taxminan 18 daqiqa qoldi"
 */
function t(key, params = {}) {
  const val = key.split('.').reduce((o, k) => o?.[k], TRANSLATIONS);
  if (val === undefined) { console.warn(`[i18n] Yo'q: ${key}`); return key; }
  return val.replace(/\{(\w+)\}/g, (_, p) => params[p] ?? `{${p}}`);
}

/** Sahifadagi barcha data-i18n elementlarini tarjima qilish */
function applyI18n(root = document) {
  root.querySelectorAll('[data-i18n]').forEach(el => {
    el.textContent = t(el.dataset.i18n);
  });
  root.querySelectorAll('[data-i18n-ph]').forEach(el => {
    el.placeholder = t(el.dataset.i18nPh);
  });
  root.querySelectorAll('[data-i18n-title]').forEach(el => {
    el.title = t(el.dataset.i18nTitle);
  });
  document.documentElement.lang = CURRENT_LANG;
}

/** Til almashtirish (sahifa qayta yuklanmasdan) */
async function switchLang(lang) {
  await loadI18n(lang);
  applyI18n();
  await reloadMenuContent(lang);        // DB kontentini qayta olish
  document.cookie = `armenu_lang=${lang};path=/;max-age=31536000`;
  fetch('/v1/lang/', {method:'POST', body: JSON.stringify({lang})});
}
```

1.5 CI tekshiruvi (tarjima to'liqligi)

```
# tests/test_i18n_completeness.py
import json, pathlib

def test_all_langs_have_same_keys():
    """3 tilda ham AYNAN bir xil kalitlar bo'lishi SHART"""
    base = pathlib.Path('static/i18n')
    uz = flatten(json.load(open(base/'uz.json')))
    ru = flatten(json.load(open(base/'ru.json')))
    en = flatten(json.load(open(base/'en.json')))

    assert set(uz) == set(ru), f"RU yetishmayapti: {set(uz)-set(ru)}"
    assert set(uz) == set(en), f"EN yetishmayapti: {set(uz)-set(en)}"

def test_no_empty_translations():
    for lang in ('uz','ru','en'):
        d = flatten(json.load(open(f'static/i18n/{lang}.json')))
        empty = [k for k, v in d.items() if not v.strip()]
        assert not empty, f"{lang} bo'sh: {empty}"

def test_placeholders_match():
    """{n}, {amount} kabi o'rinbosarlar 3 tilda mos bo'lishi kerak"""
    ...
```

1.6 🤖 AI avto-tarjima (admin uchun)

```
# apps/ai/services/translate.py

TRANSLATE_PROMPT = """Sen professional tarjimonsan. O'zbek restorani menyusini tarjima qilyapsan.

QOIDALAR:
1. Milliy o'zbek taomlari nomini TARJIMA QILMA — transliteratsiya qil.
   Masalan: "Norin" → en: "Norin", ru: "Норин"
2. Tavsifda taom nima ekanini QISQA tushuntir (chet ellik bilmaydi).
3. Umumiy taomlar tarjima qilinadi: "Salat" → "Salad" / "Салат"
4. Ohang: iliq, ishtaha ochadigan, lekin ortiqcha emas.
5. Faqat JSON qaytar, boshqa hech narsa.

Tarjima qilinadigan (o'zbekchadan {target_lang} ga):
{items}

Format:
[{"id": 15, "name": "...", "desc": "..."}]
"""

def auto_translate_dishes(restaurant_id: int, target_lang: str, batch_size: int = 40):
    field_name = f'name_{target_lang}'
    field_desc = f'desc_{target_lang}'

    qs = Dish.objects.filter(restaurant_id=restaurant_id).filter(
        Q(**{f'{field_name}': ''}) | Q(**{f'{field_desc}': ''})
    )[:batch_size]

    if not qs: return {'translated': 0}

    items = [{'id': d.id, 'name': d.name_uz, 'desc': d.desc_uz} for d in qs]

    # ⚡ MCP orqali (Qism VII) — fallback zanjiri bilan
    result = mcp_router.complete(
        task='translate',
        prompt=TRANSLATE_PROMPT.format(target_lang=LANG_NAMES[target_lang],
                                       items=json.dumps(items, ensure_ascii=False)),
        json_mode=True,
        restaurant_id=restaurant_id,
    )

    updated = 0
    for item in result:
        d = next((x for x in qs if x.id == item['id']), None)
        if not d: continue
        if not getattr(d, field_name): setattr(d, field_name, item['name'])
        if not getattr(d, field_desc): setattr(d, field_desc, item['desc'])
        d.tr_auto[target_lang] = True            # ⚡ AI tarjimasi belgisi
        d.tr_approved[target_lang] = False       # admin tasdiqlashi kerak
        d.save()
        updated += 1

    bump_menu_version(restaurant_id)
    return {'translated': updated}
```

---

2. 🎨 UI/UX DIZAYN TIZIMI

2.1 Dizayn printsiplari

#	Printsip	Amalda
1	**Taom rasmi — bosh qahramon**	Katta rasm, minimal matn
2	**3 tegishda buyurtma**	Menyu → taom → savatga → buyurtma
3	**AR jozibali ko'rinsin**	Tugma ko'zga tashlansin, animatsiya
4	**Katta tegish maydonlari**	Min 44×44 px (barmoq uchun)
5	**Tez**	Skeleton loader, optimistik UI
6	**Ochiq**	Kontrast ≥4,5:1, shrift ≥16px
7	**Dark + Light**	Ikkalasi ham to'liq

2.2 Rang tizimi (CSS o'zgaruvchilar)

```
/* static/guest/css/theme.css */

:root {
  /* Brend (restoran o'zgartirishi mumkin) */
  --brand:        #E85D2A;
  --brand-hover:  #D14E1F;
  --brand-light:  #FFF0EA;

  /* Neytral — LIGHT */
  --bg:           #FFFFFF;
  --bg-soft:      #F7F7F8;
  --bg-card:      #FFFFFF;
  --border:       #E5E5E8;
  --text:         #1A1A1F;
  --text-soft:    #6B6B76;
  --text-muted:   #9B9BA6;

  /* Semantik */
  --success:      #16A34A;
  --warning:      #F59E0B;
  --danger:       #DC2626;
  --info:         #2563EB;

  /* Holat ranglari (stol) */
  --st-free:      #16A34A;
  --st-occupied:  #2563EB;
  --st-waiting:   #F59E0B;
  --st-ready:     #DC2626;
  --st-bill:      #9333EA;

  /* O'lchamlar */
  --r-sm: 8px; --r-md: 12px; --r-lg: 16px; --r-xl: 24px;
  --sp-1: 4px; --sp-2: 8px; --sp-3: 12px; --sp-4: 16px;
  --sp-5: 24px; --sp-6: 32px; --sp-8: 48px;

  --shadow-sm: 0 1px 2px rgba(0,0,0,.05);
  --shadow-md: 0 4px 12px rgba(0,0,0,.08);
  --shadow-lg: 0 12px 32px rgba(0,0,0,.12);
}

/* DARK — tizim sozlamasi */
@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --bg:        #0F0F13;
    --bg-soft:   #17171D;
    --bg-card:   #1C1C24;
    --border:    #2A2A35;
    --text:      #F0F0F5;
    --text-soft: #A0A0B0;
    --text-muted:#6B6B7B;
    --brand-light: #2A1810;
    --shadow-sm: 0 1px 2px rgba(0,0,0,.3);
    --shadow-md: 0 4px 12px rgba(0,0,0,.4);
    --shadow-lg: 0 12px 32px rgba(0,0,0,.5);
  }
}

/* DARK — qo'lda tanlangan */
:root[data-theme="dark"] {
  --bg:        #0F0F13;
  --bg-soft:   #17171D;
  --bg-card:   #1C1C24;
  --border:    #2A2A35;
  --text:      #F0F0F5;
  --text-soft: #A0A0B0;
  --text-muted:#6B6B7B;
  --brand-light: #2A1810;
}

body { background: var(--bg); color: var(--text); }
```

2.3 Tipografiya

```
:root {
  --font: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
  --font-mono: 'JetBrains Mono', monospace;

  --fs-xs:  12px;   --fs-sm:  14px;   --fs-base: 16px;
  --fs-lg:  18px;   --fs-xl:  20px;   --fs-2xl:  24px;
  --fs-3xl: 32px;   --fs-4xl: 40px;   --fs-5xl:  56px;   /* KDS uchun */
}
```

⚠ **Kirill va lotin** — ikkalasi ham qo'llab-quvvatlansin (rus tili uchun).

2.4 KDS uslubi (oshxona TV) — `ai agent.mp4` ilhomi

```
/* static/kds/css/kds.css — dark, neon, 3 metrdan o'qiladigan */
.kds {
  --kds-bg:     #08080C;
  --kds-card:   #12121A;
  --kds-border: #1F1F2E;
  --kds-new:    #00E5FF;    /* neon moviy */
  --kds-cook:   #FFB300;    /* neon sariq */
  --kds-ready:  #00E676;    /* neon yashil */
  --kds-late:   #FF1744;    /* neon qizil */
  font-size: 20px;
}
.kds-order-table { font-size: 56px; font-weight: 900; letter-spacing: -1px; }
.kds-order-item  { font-size: 28px; font-weight: 600; }
.kds-timer       { font-size: 32px; font-variant-numeric: tabular-nums; }
.kds-card.late   { animation: pulse-red 1s infinite; box-shadow: 0 0 24px var(--kds-late); }
@keyframes pulse-red { 0%,100%{opacity:1} 50%{opacity:.6} }
```

2.5 Komponentlar kutubxonasi

Komponent	Fayl	Ishlatiladi
Button (primary/ghost/danger)	`components/button.css`	Hamma joyda
Card (dish/order/table)	`components/card.css`	Menyu, KDS
Modal (bottom-sheet mobil)	`components/modal.js`	Taom, AR, savat
Toast (success/error/info)	`components/toast.js`	Bildirishnoma
Skeleton loader	`components/skeleton.css`	Yuklanishda
Bottom sheet	`components/sheet.js`	Mobil modal
Stepper (−/+)	`components/stepper.js`	Miqdor
Rating (yulduz)	`components/rating.js`	Otziv
Progress bar	`components/progress.css`	Buyurtma holati
Badge	`components/badge.css`	HIT, YANGI, tugagan
Tabs	`components/tabs.js`	Kategoriya
Empty state	`components/empty.css`	Bo'sh savat

2.6 Animatsiyalar (o'lchovli, ortiqcha emas)

Hodisa	Animatsiya	Davomiylik
Savatga qo'shish	Rasm savatga uchadi	400ms
Modal ochilish	Pastdan ko'tarilish + fade	250ms
Buyurtma yuborildi	Checkmark chizilishi	600ms
Tayyor bildirishnoma	Pulsatsiya + vibratsiya	1s × 3
AR yuklanishi	Aylanuvchi 3D skeleton	—
Tugma bosish	Scale 0.97	100ms
Sahifa o'tishi	Slide	200ms

⚠ `prefers-reduced-motion` hurmat qilinadi.

2.7 Mobil moslashuv

Ekran	Layout
< 400px	1 ustun, bottom-sheet modal
400-768px	1-2 ustun
768-1024px	2-3 ustun (planshet)
> 1024px	Admin: sidebar + kontent; KDS: 3 ustun

2.8 Ochiqlik (accessibility)

- ✅ Kontrast ≥ 4,5:1 (matn), ≥ 3:1 (katta matn)
- ✅ Klaviatura navigatsiyasi (Tab, Enter, Esc)
- ✅ ARIA yorliqlar (`aria-label`, `role`)
- ✅ Focus ko'rinadigan (`:focus-visible`)
- ✅ Rasm `alt` matni
- ✅ Shrift o'lchamini oshirish buzmaydi (rem)
- ✅ Ekran o'quvchi bilan sinovdan o'tgan

---

3. 📱 PWA (ilova o'rniga)

3.1 Manifest

```
{
  "name": "ARMENU",
  "short_name": "ARMENU",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#FFFFFF",
  "theme_color": "#E85D2A",
  "icons": [
    {"src":"/static/icons/192.png","sizes":"192x192","type":"image/png"},
    {"src":"/static/icons/512.png","sizes":"512x512","type":"image/png"},
    {"src":"/static/icons/maskable.png","sizes":"512x512","purpose":"maskable"}
  ]
}
```

3.2 Service Worker strategiyalari

Resurs	Strategiya	TTL
HTML	Network-first, cache fallback	—
CSS/JS	Cache-first, versiyalangan	1 yil
Rasmlar	Cache-first	7 kun
3D fayllar	Cache-first	30 kun
API menyu	Stale-while-revalidate	60s
API buyurtma	Network-only	—

3.3 Offline rejim

```
// Offline'da nima ishlaydi
✅ Menyu ko'rish (keshdan)
✅ Taom tafsiloti
✅ Savatga qo'shish (localStorage)
✅ 3D ko'rish (kesh bo'lsa)
❌ Buyurtma yuborish → navbatga, internet kelganda yuboriladi
❌ To'lov
❌ Real-time holat
```

QISM VII

AI va MCP integratsiyasi

> Talab: "*eng kamida 6 ta AI ga MCP yozilgan va API key orqali muammosiz ishlashi kerak*"

---

1. MAQSAD VA PRINTSIP

1.1 Nima uchun 6+ provayder

Sabab	Tushuntirish
Ishonchlilik	Biri yiqilsa → keyingisi. Tizim hech qachon to'xtamaydi
Narx	Har vazifa uchun eng arzon mos provayder
Sifat	Murakkab vazifa → kuchli model; oddiy → arzon model
Tezlik	Groq juda tez, real-time chat uchun
Mustaqillik	Bitta kompaniyaga bog'lanmaslik (vendor lock-in yo'q)
Geografiya	Biri O'zbekistonda bloklanca → boshqasi

1.2 Asosiy printsip

```
Kod HECH QACHON aniq provayderni chaqirmaydi.
Kod faqat VAZIFA aytadi: "bu matnni tarjima qil", "3D model yarat".
Router o'zi eng mos provayderni tanlaydi va yiqilsa keyingisiga o'tadi.
```

---

2. 🔌 MCP ARXITEKTURASI



---

3. 📦 PROVAYDERLAR RO'YXATI (8 ta — talab 6+)

#	Provayder	Model	Vazifalar	Narx (1M token)	Env
1	**Anthropic Claude**	claude-haiku-4-5	chat, translate, vision	$0,80 / $4	`ANTHROPIC_API_KEY`
2	**Google Gemini**	gemini-2.0-flash	chat, translate, vision	$0,10 / $0,40	`GEMINI_API_KEY`
3	**OpenAI**	gpt-4o-mini	chat, translate, vision	$0,15 / $0,60	`OPENAI_API_KEY`
4	**Groq**	llama-3.3-70b	chat (juda tez)	$0,59 / $0,79	`GROQ_API_KEY`
5	**DeepSeek**	deepseek-chat	chat, translate (arzon)	$0,14 / $0,28	`DEEPSEEK_API_KEY`
6	**Mistral**	mistral-small	chat, translate	$0,20 / $0,60	`MISTRAL_API_KEY`
7	**Meshy**	meshy-4	image→3D	~$0,50/model	`MESHY_API_KEY`
8	**Tripo**	tripo-v2	image→3D	~$0,80/model	`TRIPO_API_KEY`
9	**Rodin** (ixtiyoriy)	rodin-gen1	image→3D (yuqori sifat)	~$1,50/model	`RODIN_API_KEY`

> ⚡ **Jami 8-9 ta provayder** — talab qilingan 6 tadan ko'p. Har biri API key orqali.

---

4. 💻 KOD — MCP QATLAMI

4.1 Baza sinf

```
# apps/ai/mcp/base.py
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any

@dataclass
class MCPRequest:
    task: str                      # chat|translate|vision|image_to_3d|embedding
    prompt: str = ''
    system: str = ''
    messages: list = None
    images: list = None            # rasm URL yoki base64
    json_mode: bool = False
    max_tokens: int = 1500
    temperature: float = 0.3
    timeout: int = 30
    extra: dict = None

@dataclass
class MCPResponse:
    ok: bool
    content: Any = None
    provider: str = ''
    model: str = ''
    tokens_in: int = 0
    tokens_out: int = 0
    cost_usd: float = 0.0
    latency_ms: int = 0
    error: str = ''
    raw: dict = None


class MCPProvider(ABC):
    """Har bir AI provayder shu interfeysni amalga oshiradi."""
    name: str
    supported_tasks: set
    priority: dict          # {'chat': 10, 'translate': 20}
    price_in: float         # $/1M token
    price_out: float

    @abstractmethod
    def is_configured(self) -> bool:
        """API key mavjudmi"""

    @abstractmethod
    def call(self, req: MCPRequest) -> MCPResponse:
        """Asosiy chaqiruv"""

    def health_check(self) -> bool:
        """Provayder tirikmi (yengil so'rov)"""
        try:
            r = self.call(MCPRequest(task='chat', prompt='ping', max_tokens=5, timeout=8))
            return r.ok
        except Exception:
            return False

    def estimate_cost(self, tin: int, tout: int) -> float:
        return (tin * self.price_in + tout * self.price_out) / 1_000_000
```

4.2 Provayder namunasi — Claude

```python
# apps/ai/mcp/providers/claude.py
import anthropic, time, json
from django.conf import settings
from ..base import MCPProvider, MCPRequest, MCPResponse

class ClaudeProvider(MCPProvider):
    name = 'claude'
    supported_tasks = {'chat', 'translate', 'vision', 'reasoning'}
    priority = {'chat': 20, 'translate': 10, 'vision': 15, 'reasoning': 5}
    price_in, price_out = 0.80, 4.00
    MODEL = 'claude-haiku-4-5-20251001'

    def __init__(self):
        self.key = settings.AI_KEYS.get('ANTHROPIC_API_KEY', '')
        self.client = anthropic.Anthropic(api_key=self.key) if self.key else None

    def is_configured(self): return bool(self.key)

    def call(self, req: MCPRequest) -> MCPResponse:
        t0 = time.time()
        try:
            msgs = req.messages or [{'role':'user','content': req.prompt}]
            if req.images:
                msgs[-1]['content'] = [
                    {'type':'image','source':{'type':'url','url': u}} for u in req.images
                ] + [{'type':'text','text': req.prompt}]

            kwargs = dict(model=self.MODEL, max_tokens=req.max_tokens,
                          temperature=req.temperature, messages=msgs)
            if req.system: kwargs['system'] = req.system

            resp = self.client.messages.create(**kwargs, timeout=req.timeout)
            text = resp.content[0].text
            content = json.loads(_strip_fences(text)) if req.json_mode else text

            return MCPResponse(
                ok=True, content=content, provider=self.name, model=self.MODEL,
                tokens_in=resp.usage.input_tokens, tokens_out=resp.usage.output_tokens,
                cost_usd=self.estimate_cost(resp.usage.input_tokens, resp.usage.output_tokens),
                latency_ms=int((time.time()-t0)*1000),
            )
        except Exception as e:
            return MCPResponse(ok=False, provider=self.name, error=str(e),
                               latency_ms=int((time.time()-t0)*1000))
```

4.3 Provayder namunasi — Meshy (3D)

```python
# apps/ai/mcp/providers/meshy.py
import requests, time
from ..base import MCPProvider, MCPRequest, MCPResponse

class MeshyProvider(MCPProvider):
    name = 'meshy'
    supported_tasks = {'image_to_3d'}
    priority = {'image_to_3d': 10}          # eng birinchi (arzon)
    price_in = price_out = 0
    BASE = 'https://api.meshy.ai/openapi/v1'
    COST_PER_MODEL = 0.50

    def __init__(self):
        self.key = settings.AI_KEYS.get('MESHY_API_KEY', '')

    def is_configured(self): return bool(self.key)

    def call(self, req: MCPRequest) -> MCPResponse:
        t0 = time.time()
        h = {'Authorization': f'Bearer {self.key}'}
        try:
            # 1. Vazifa yaratish
            r = requests.post(f'{self.BASE}/image-to-3d', headers=h, timeout=30, json={
                'image_url': req.images[0],
                'enable_pbr': True,
                'should_remesh': True,
                'target_polycount': 40000,
                'topology': 'triangle',
            })
            r.raise_for_status()
            task_id = r.json()['result']

            # 2. Poll (maks 10 daqiqa)
            deadline = time.time() + 600
            while time.time() < deadline:
                time.sleep(10)
                s = requests.get(f'{self.BASE}/image-to-3d/{task_id}', headers=h, timeout=20).json()
                st = s.get('status')
                if st == 'SUCCEEDED':
                    return MCPResponse(
                        ok=True, provider=self.name, model='meshy-4',
                        content={'glb_url': s['model_urls']['glb'],
                                 'thumbnail': s.get('thumbnail_url')},
                        cost_usd=self.COST_PER_MODEL,
                        latency_ms=int((time.time()-t0)*1000))
                if st in ('FAILED', 'EXPIRED'):
                    return MCPResponse(ok=False, provider=self.name,
                                       error=s.get('task_error', {}).get('message', st))
            return MCPResponse(ok=False, provider=self.name, error='timeout')
        except Exception as e:
            return MCPResponse(ok=False, provider=self.name, error=str(e))
```

4.4 ⚡ ROUTER — eng muhim komponent

```python
# apps/ai/mcp/router.py
import hashlib, json, logging
from django.core.cache import cache
from django.conf import settings
from core.resilience import CircuitBreaker
from .base import MCPRequest, MCPResponse
from .registry import ALL_PROVIDERS

log = logging.getLogger('ai.mcp')

class MCPRouter:
    """
    Vazifani mos provayderga yo'naltiradi.
    Yiqilsa — keyingisiga. Xarajatni hisoblaydi. Keshlaydi.
    """
    def __init__(self):
        self.providers = [p() for p in ALL_PROVIDERS]
        self.breakers = {p.name: CircuitBreaker(f'ai.{p.name}',
                         fail_threshold=3, timeout=120) for p in self.providers}

    # ---- Asosiy metod ----
    def run(self, req: MCPRequest, restaurant_id=None, use_cache=True) -> MCPResponse:
        # 1. Kesh
        ck = self._cache_key(req)
        if use_cache and req.task in ('translate', 'chat'):
            cached = cache.get(ck)
            if cached:
                log.debug(f'[MCP] kesh: {req.task}')
                return MCPResponse(**cached)

        # 2. Byudjet
        if restaurant_id and not self._check_budget(restaurant_id):
            return MCPResponse(ok=False, error='AI_BUDGET_EXCEEDED')

        # 3. Provayder ro'yxati (prioritet bo'yicha)
        candidates = self._candidates(req.task)
        if not candidates:
            return MCPResponse(ok=False, error=f'NO_PROVIDER_FOR_{req.task}')

        # 4. Zanjir bo'ylab urinish
        errors = []
        for p in candidates:
            br = self.breakers[p.name]
            if br.is_open():
                errors.append(f'{p.name}: circuit_open')
                continue

            log.info(f'[MCP] {req.task} → {p.name}')
            resp = p.call(req)

            if resp.ok:
                br.on_success()
                self._record(resp, restaurant_id, req.task)
                if use_cache and req.task in ('translate', 'chat'):
                    cache.set(ck, resp.__dict__, 3600)
                return resp

            br.on_failure()
            errors.append(f'{p.name}: {resp.error}')
            log.warning(f'[MCP] {p.name} xato: {resp.error} → keyingisi')

        log.error(f'[MCP] BARCHA provayder yiqildi: {errors}')
        return MCPResponse(ok=False, error='ALL_PROVIDERS_FAILED',
                           raw={'errors': errors})

    # ---- Yordamchilar ----
    def _candidates(self, task):
        ps = [p for p in self.providers
              if p.is_configured() and task in p.supported_tasks]
        return sorted(ps, key=lambda p: p.priority.get(task, 999))

    def _cache_key(self, req):
        raw = f'{req.task}|{req.system}|{req.prompt}|{req.temperature}'
        return f'mcp:{hashlib.sha256(raw.encode()).hexdigest()[:32]}'

    def _check_budget(self, restaurant_id):
        today = timezone.now().date()
        spent = AIUsage.objects.filter(
            restaurant_id=restaurant_id, created_at__date=today
        ).aggregate(s=Sum('cost_usd'))['s'] or 0
        limit = settings.AI_DAILY_LIMIT_PER_RESTAURANT   # masalan $2
        if spent >= limit:
            log.warning(f'[MCP] Byudjet tugadi: rest={restaurant_id} ${spent}')
            return False
        return True

    def _record(self, resp, restaurant_id, task):
        AIUsage.objects.create(
            restaurant_id=restaurant_id, provider=resp.provider, model=resp.model,
            task=task, tokens_in=resp.tokens_in, tokens_out=resp.tokens_out,
            cost_usd=resp.cost_usd, latency_ms=resp.latency_ms)
        metrics.inc(f'ai.{resp.provider}.success')
        metrics.observe('ai.latency_ms', resp.latency_ms, {'provider': resp.provider})
        metrics.add('ai.cost_usd', resp.cost_usd, {'provider': resp.provider})


mcp = MCPRouter()      # singleton
```

4.5 Ishlatilishi (juda oddiy)

```
from apps.ai.mcp import mcp, MCPRequest

# Chatbot
resp = mcp.run(MCPRequest(
    task='chat',
    system=CHAT_SYSTEM_PROMPT,
    prompt=user_question,
    max_tokens=400,
), restaurant_id=rid)

if resp.ok:
    answer = resp.content
else:
    answer = t('chat.error')      # "Ofitsiantni chaqiring"

# Tarjima
resp = mcp.run(MCPRequest(task='translate', prompt=..., json_mode=True))

# 3D model
resp = mcp.run(MCPRequest(task='image_to_3d', images=[photo_url]))
glb_url = resp.content['glb_url'] if resp.ok else None
```

---

5. 🔑 API KEY BOSHQARUVI

5.1 `.env` fayl

```
# --- AI provayderlar (kamida 3 tasi to'ldirilishi kerak) ---
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=AIza...
OPENAI_API_KEY=sk-proj-...
GROQ_API_KEY=gsk_...
DEEPSEEK_API_KEY=sk-...
MISTRAL_API_KEY=...

# --- 3D generatsiya ---
MESHY_API_KEY=msy_...
TRIPO_API_KEY=tsk_...
RODIN_API_KEY=...

# --- Limitlar ---
AI_DAILY_LIMIT_PER_RESTAURANT=2.0    # $/kun
AI_MONTHLY_LIMIT_TOTAL=500.0         # $/oy (butun platforma)
AI_CACHE_TTL=3600
```

5.2 Ishga tushishda tekshiruv

```
# apps/ai/apps.py
class AIConfig(AppConfig):
    def ready(self):
        from .mcp import mcp
        configured = [p.name for p in mcp.providers if p.is_configured()]
        chat_providers = [p for p in mcp.providers
                          if p.is_configured() and 'chat' in p.supported_tasks]
        three_d = [p for p in mcp.providers
                   if p.is_configured() and 'image_to_3d' in p.supported_tasks]

        log.info(f'[AI] Sozlangan provayderlar ({len(configured)}): {configured}')

        if len(chat_providers) < 2:
            log.warning('⚠ [AI] Kamida 2 ta chat provayderi tavsiya etiladi (fallback uchun)')
        if not three_d:
            log.warning('⚠ [AI] 3D provayder yo\'q — AI model generatsiyasi ishlamaydi')
        if not configured:
            log.error('✖ [AI] HECH BIR provayder sozlanmagan!')
```

5.3 Xavfsizlik

Chora	Amalda
Key kodda YO'Q	Faqat `.env` / Vault
`.env` git'da YO'Q	`.gitignore`
Log'da key ko'rinmaydi	Maskalash: `sk-ant-***`
Frontendga uzatilmaydi	Barcha AI chaqiruvi backendda
Rotatsiya	90 kunda bir marta
Har provayderga alohida limit	Provayder panelida

---

6. 🤖 AI VAZIFALARI (nima uchun ishlatiladi)

#	Vazifa	Provayder prioriteti	Chastota	Oylik xarajat (500 rest)
1	**Chatbot** (mijoz savoli)	Gemini→Groq→DeepSeek→Claude	~50k/oy	~$45
2	**Tarjima** (menyu)	Claude→Gemini→GPT	~5k/oy	~$12
3	**3D generatsiya**	Meshy→Tripo→Rodin	~800/oy	~$400
4	**Taom tavsifi yozish**	Claude→Gemini	~2k/oy	~$8
5	**Menyu tahlili** (tavsiya)	Claude	~500/oy	~$15
6	**Otziv tahlili** (sentiment)	Gemini→DeepSeek	~10k/oy	~$6
7	**Rasm sifatini baholash**	Gemini vision	~3k/oy	~$5
8	**Kutubxona moslashtirish** (embedding)	Gemini embedding	~5k/oy	~$2
	JAMI			**~$493/oy**

> 💰 500 restoran × 590k so'm ≈ 295 mln so'm/oy (~$23 300).
> AI xarajati = **2,1%** daromaddan. ✅ Sog'lom.

---

7. 💬 CHATBOT — TO'LIQ AMALGA OSHIRISH

7.1 System prompt

```
CHAT_SYSTEM = """Sen "{restaurant_name}" restoranining menyu yordamchisisan.

VAZIFANG: mijozga menyu bo'yicha yordam berish va taom tanlashga ko'maklashish.

QAT'IY QOIDALAR:
1. FAQAT quyidagi menyudagi taomlar haqida gapir. Menyuda yo'q taomni TAKLIF QILMA.
2. Narxlarni aynan menyudagidek ayt. O'zingdan narx O'YLAB TOPMA.
3. Javob QISQA bo'lsin — 2-4 gap. Mijoz och, uzun matn o'qimaydi.
4. {lang} tilida javob ber.
5. Milliy taom haqida so'ralsa — nima ekanini QISQA tushuntir (chet ellik bilmasligi mumkin).
6. Allergiya/parhez so'ralsa — DIQQAT BILAN javob ber, faqat teglar asosida.
   Ishonchsiz bo'lsang: "Ofitsiantdan aniqlashtirishingizni so'raymiz".
7. Menyudan tashqari savol (siyosat, ob-havo, dasturlash) → xushmuomala rad et.
8. Har javobda 1-2 ta taom taklif qil (upsell), lekin bosim o'tkazma.

MENYU:
{menu_json}

MIJOZ SAVATIDA: {cart_summary}
"""
```

7.2 Chatbot servisi

```
# apps/ai/services/chat.py

def answer_question(session, question: str, lang: str) -> dict:
    # 1. Rate limit
    if not rate_limit_ok(f'chat:{session.id}', limit=10, window=300):
        return {'error': 'rate_limited', 'text': t('errors.rate_limited', lang)}

    # 2. Kirish tozalash
    question = question.strip()[:500]
    if not question:
        return {'error': 'empty'}

    # 3. Menyu konteksti (kesh, kesilgan — token tejash)
    menu_ctx = build_menu_context(session.restaurant_id, lang)   # ~1500 token

    # 4. Suhbat tarixi (oxirgi 6 xabar)
    history = get_chat_history(session.id, limit=6)

    # 5. MCP chaqiruvi
    resp = mcp.run(MCPRequest(
        task='chat',
        system=CHAT_SYSTEM.format(
            restaurant_name=session.restaurant.name, lang=LANG_NAMES[lang],
            menu_json=menu_ctx, cart_summary=summarize_cart(session)),
        messages=history + [{'role': 'user', 'content': question}],
        max_tokens=350, temperature=0.4,
    ), restaurant_id=session.restaurant_id)

    if not resp.ok:
        return {'error': 'ai_unavailable', 'text': t('chat.error', lang)}

    text = resp.content

    # 6. ⚡ VALIDATSIYA — AI o'ylab topgan taomni bloklash
    mentioned = extract_dish_mentions(text, menu_ctx)
    invalid = [d for d in mentioned if d not in valid_dish_names(session.restaurant_id)]
    if invalid:
        log.warning(f'[AI] Menyuda yo\'q taom aytdi: {invalid}')
        text = remove_invalid_mentions(text, invalid)

    # 7. Tugmalar biriktirish
    buttons = build_action_buttons(mentioned, session.restaurant_id, lang)
    # [{'type':'ar','dish_id':15,'label':'AR\'da ko\'rish'},
    #  {'type':'order','dish_id':15,'label':'Buyurtma'}]

    # 8. Tarixga saqlash
    save_chat_message(session.id, 'user', question)
    save_chat_message(session.id, 'assistant', text)

    return {'text': text, 'buttons': buttons, 'provider': resp.provider}
```

7.3 Menyu konteksti (token tejash — muhim)

```
def build_menu_context(restaurant_id: int, lang: str) -> str:
    """
    To'liq menyu emas — faqat AI uchun zarur maydonlar.
    100 taom ≈ 1500 token (to'liq bo'lsa 8000+ bo'lardi).
    """
    key = f'ai:menuctx:{restaurant_id}:{lang}'
    ctx = cache.get(key)
    if ctx: return ctx

    dishes = Dish.objects.filter(
        restaurant_id=restaurant_id, is_available=True
    ).select_related('category').only(
        'id','name_uz','name_ru','name_en','price','diet_tags','allergens',
        'cook_min','category__name_uz','category__name_ru','category__name_en')

    lines = []
    for d in dishes:
        tags = ','.join(d.diet_tags) if d.diet_tags else '-'
        alrg = ','.join(d.allergens) if d.allergens else '-'
        lines.append(f"{d.id}|{t(d,'name',lang)}|{int(d.price)}|{tags}|{alrg}|{d.cook_min}min")

    ctx = "ID|Nom|Narx|Teglar|Allergen|Vaqt\n" + "\n".join(lines)
    cache.set(key, ctx, 300)
    return ctx
```

---

8. 📱 AI TESTLARI

```python
# tests/test_ai.py

def test_fallback_chain(monkeypatch):
    """Birinchi 2 provayder yiqilsa, 3-chisi ishlashi kerak"""
    monkeypatch.setattr(GeminiProvider, 'call',
                        lambda s, r: MCPResponse(ok=False, error='500'))
    monkeypatch.setattr(GroqProvider, 'call',
                        lambda s, r: MCPResponse(ok=False, error='timeout'))
    resp = mcp.run(MCPRequest(task='chat', prompt='salom'))
    assert resp.ok
    assert resp.provider not in ('gemini', 'groq')

def test_all_providers_down():
    """Hammasi yiqilsa — chiroyli xato, crash EMAS"""
    with mock_all_providers_fail():
        resp = mcp.run(MCPRequest(task='chat', prompt='salom'))
    assert not resp.ok
    assert resp.error == 'ALL_PROVIDERS_FAILED'

def test_budget_limit(restaurant):
    AIUsageFactory.create_batch(100, restaurant=restaurant, cost_usd=0.05)  # $5
    resp = mcp.run(MCPRequest(task='chat', prompt='x'), restaurant_id=restaurant.id)
    assert resp.error == 'AI_BUDGET_EXCEEDED'

def test_chatbot_does_not_invent_dishes(restaurant, menu):
    """AI menyuda yo'q taomni taklif qilmasligi SHART"""
    r = answer_question(session, "Sushi bormi?", 'uz')
    assert 'sushi' not in r['text'].lower() or 'yo\'q' in r['text'].lower()

def test_circuit_breaker_opens():
    """3 xatodan keyin provayder chetlab o'tiladi"""
    for _ in range(3):
        mcp.breakers['gemini'].on_failure()
    assert mcp.breakers['gemini'].is_open()

def test_cost_recorded(restaurant):
    mcp.run(MCPRequest(task='chat', prompt='salom'), restaurant_id=restaurant.id)
    assert AIUsage.objects.filter(restaurant=restaurant).exists()
```

9. 📊 AI KUZATUVI (P-05 sahifasi)

```
┌──────────────────────────────────────────────────────────┐
│ 🤖 AI xarajatlari                        [Bu oy ▾]       │
├──────────────────────────────────────────────────────────┤
│ Jami: $487  |  Byudjet: $500  |  ████████████▒▒ 97%      │
├──────────────────────────────────────────────────────────┤
│ Provayder    Chaqiruv   Xarajat   O'rt.vaqt   Xato  Holat │
│ ──────────────────────────────────────────────────────── │
│ Meshy          782       $391      4m 12s      2,1%  🟢   │
│ Gemini      48 210       $ 42      0,8s        0,3%  🟢   │
│ Claude       6 104       $ 31      1,4s        0,1%  🟢   │
│ Groq        12 890       $ 15      0,3s        1,8%  🟢   │
│ DeepSeek     3 201       $  5      1,1s        0,4%  🟢   │
│ Tripo           89       $  3      6m 40s      8,9%  🟡   │
│ OpenAI           0       $  0      —           —     ○ zaxira│
├──────────────────────────────────────────────────────────┤
│ ⚠ Tripo xato darajasi yuqori (8,9%) — prioritet pasaytirildi │
│ 💡 Gemini eng samarali: 48k chaqiruv, atigi $42          │
└──────────────────────────────────────────────────────────┘
```

10. 🔮 KELAJAKKA TAYYORLIK

MCP qatlami tufayli quyidagilar **kod o'zgartirmasdan** qo'shiladi:

Kelajak	Nima qilinadi
Yangi AI model chiqdi	Provayder faylida `MODEL` o'zgartiriladi
Yangi provayder	1 ta fayl (~80 qator) + registry'ga qo'shish
Ovozli buyurtma	`task='speech_to_text'` + Whisper provayder
Rasm bo'yicha qidiruv	`task='vision'` (allaqachon bor)
Video generatsiya	`task='text_to_video'` + provayder
Lokal model (arzon)	Ollama provayderi, prioritet #1
Narx tushdi	`priority` config'da o'zgartiriladi

> 🎯 **Bu — 5 yillik chidamlilikning texnik asosi.**
> AI dunyosi tez o'zgaradi, lekin bizning kod o'zgarmaydi.

QISM VIII

Deploy, reja, risklar

1. 🚀 DEPLOY

1.1 Muhitlar

Muhit	Maqsad	Domen	Ma'lumot
dev	Lokal ishlab chiqish	`localhost:8060`	Demo seed
staging	Sinov, pilot demo	`stage.armenu.uz`	Anonimlashtirilgan nusxa
prod	Ishlab chiqarish	`armenu.uz`	Real

1.2 Systemd servislar

```
# /etc/systemd/system/armenu.socket
[Unit]
Description=ARMENU web socket
[Socket]
ListenStream=127.0.0.1:8060
NoDelay=true
[Install]
WantedBy=sockets.target
```

```
# /etc/systemd/system/armenu.service
[Unit]
Description=ARMENU web (gunicorn)
Requires=armenu.socket
After=network.target postgresql.service redis.service

[Service]
Type=notify
User=armenu
WorkingDirectory=/home/armenu/app
EnvironmentFile=/home/armenu/app/.env
ExecStart=/home/armenu/app/venv/bin/gunicorn config.wsgi:application \
  --workers 8 --threads 4 --worker-class gthread \
  --timeout 60 --graceful-timeout 30 --keep-alive 5 \
  --max-requests 2000 --max-requests-jitter 200 \
  --access-logfile - --error-logfile - --log-level info
ExecReload=/bin/kill -s HUP $MAINPID
Restart=always
RestartSec=5
KillSignal=SIGTERM
LimitNOFILE=65535

[Install]
WantedBy=multi-user.target
```

```
# /etc/systemd/system/armenu-ws.service
[Unit]
Description=ARMENU WebSocket (daphne)
[Service]
ExecStart=/home/armenu/app/venv/bin/daphne -b 127.0.0.1 -p 8061 \
  --proxy-headers --access-log - config.asgi:application
Restart=always
RestartSec=5
LimitNOFILE=65535
```

```
# /etc/systemd/system/armenu-worker@.service  (parametrli — 5 tur)
[Unit]
Description=ARMENU worker (%i)
[Service]
ExecStart=/home/armenu/app/venv/bin/python manage.py run_worker --queue=%i
Restart=always
RestartSec=10
MemoryMax=2G
```

Ishga tushirish:

```
systemctl enable --now armenu.socket armenu.service armenu-ws.service
systemctl enable --now armenu-worker@model3d armenu-worker@fiscal \
                       armenu-worker@notify armenu-worker@ai armenu-worker@analytics
systemctl enable --now armenu-healer.service
```

1.3 Nginx

```
# /etc/nginx/sites-available/armenu.conf
upstream armenu_web { server 127.0.0.1:8060; keepalive 32; }
upstream armenu_ws  { server 127.0.0.1:8061; }

server {
    listen 443 ssl http2;
    server_name armenu.uz *.armenu.uz;
    ssl_certificate     /etc/letsencrypt/live/armenu.uz/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/armenu.uz/privkey.pem;

    client_max_body_size 50M;          # 3D fayl yuklash
    gzip on; gzip_types text/css application/javascript application/json;

    # Statik
    location /static/ {
        alias /home/armenu/app/staticfiles/;
        expires 1y; add_header Cache-Control "public, immutable";
    }
    # 3D fayllar (CDN'ga o'tguncha)
    location /media/ar/ {
        alias /var/armenu/media/ar/;
        expires 30d; add_header Cache-Control "public";
        add_header Access-Control-Allow-Origin *;   # AR uchun
    }
    # WebSocket
    location /ws/ {
        proxy_pass http://armenu_ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_read_timeout 3600s;
    }
    # Web
    location / {
        proxy_pass http://armenu_web;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

1.4 CI/CD (GitHub Actions)

```
# .github/workflows/deploy.yml
name: CI/CD
on: {push: {branches: [main]}}

jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres: {image: pgvector/pgvector:pg16, env: {POSTGRES_PASSWORD: test}}
      redis:    {image: redis:7}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: python manage.py migrate
      - name: ⚡ Tenant izolyatsiya testi (MAJBURIY)
        run: pytest tests/test_tenant_isolation.py -v --tb=short
      - name: ⚡ Tarjima to'liqligi
        run: pytest tests/test_i18n_completeness.py -v
      - run: pytest --cov=apps --cov-fail-under=75
      - run: ruff check . && black --check .

  deploy:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - name: Deploy
        run: |
          ssh armenu@$HOST '
            cd /home/armenu/app &&
            git pull &&
            venv/bin/pip install -r requirements.txt &&
            venv/bin/python manage.py migrate --noinput &&
            venv/bin/python manage.py collectstatic --noinput &&
            venv/bin/python manage.py compilemessages &&
            sudo systemctl reload armenu &&
            sudo systemctl restart armenu-ws armenu-worker@*
          '
      - name: Smoke test
        run: curl -f https://armenu.uz/health/ || exit 1
```

> ⚠ **Tenant izolyatsiya testi o'tmasa deploy BLOKLANADI.**

---

2. 🗓 TO'LIQ REJA (bosqichma-bosqich)

2.1 v1 — MVP + AR (15 hafta)

Hafta	Bosqich	Vazifalar	Natija
1	Tayyorgarlik + Tenant	V-001…V-030	Skelet, auth ishlaydi
2-3	Menyu	V-031…V-054	Menyu API, 3 til
4	Stollar + QR	V-055…V-066	QR chop etiladi
5-6	Mijoz oqimi	V-067…V-092	Menyu ko'rish, savat
7-8	Buyurtma + real-time	V-093…V-114	Buyurtma, KDS, ofitsiant
9-11	⚡ **AR/3D**	V-115…V-142	**AR ishlaydi**
12	Zal plani	V-143…V-150	2D zal
13-14	Admin panel	V-151…V-170	To'liq nazorat
15	AI + MCP	V-171…V-186	Chatbot, avto-tarjima
			✅ **v1 TAYYOR**

2.2 v1.5 — Monetizatsiya (+4,5 hafta)

Hafta	Bosqich	Natija
16-17	To'lov + fiskal	Payme/Click/Uzum, OFD chek
18-19	Sodiqlik, otziv, analitika	Keshbek, Google filtri, menyu-injiniring
19,5	Deploy + kuzatuv	Production, monitoring, backup

2.3 Parallel ish (pilot bilan)



2.4 6/12/24 oylik yo'l xaritasi

Oy	Bosqich	Mijoz	Oylik daromad	Holat
1-3	Ishlab chiqish + 5 pilot	5 (bepul)	0	Investitsiya
4	Sotuv start (Toshkent)	18	6 mln	Investitsiya
5	Samarqand + Buxoro	42	16 mln	Investitsiya
6	**Viloyat miqyosi**	**75**	**32 mln**	✅ **Operatsion foyda**
9	8 viloyat	165	68 mln	Foyda
11	—	230	88 mln	✅ **Investitsiya qoplandi**
12	**Respublika miqyosi**	**280**	**105 mln**	Foyda
14	🇰🇿 Qozog'iston	340	132 mln	Kengayish
17	🇰🇬 Qirg'iziston	480	186 mln	Kengayish
19	🇹🇯 Tojikiston	580	224 mln	Kengayish
21	🇦🇿 Ozarbayjon	680	262 mln	Kengayish
24	**Qo'shni respublikalar**	**750**	**283 mln**	✅ **~3,4 mlrd/yil**

---

3. 👥 JAMOA

3.1 Minimal jamoa (0-6 oy)

Rol	Soni	Vazifa	Oylik
Backend dasturchi	1	Django, API, worker	$1 200
Frontend dasturchi	1	PWA, admin, AR	$1 000
3D dizayner (part-time)	1	Kutubxona, model sifati	$500
Sotuv menejeri	2	Restoranlarga sotuv	$400 + %
JAMI	**5**		**$3 500/oy**

3.2 O'sgan jamoa (12-oy)

Rol	Soni
Backend	2
Frontend	2
DevOps (part-time)	1
3D dizayner	2
Sotuv	4
Qo'llab-quvvatlash	2
PM	1
JAMI	**14**

---

4. ⚠ RISKLAR VA YUMSHATISH (to'liq)

4.1 Texnik risklar

#	Risk	Ehtimol	Ta'sir	Yumshatish	Egasi
T1	AR real o'lcham noto'g'ri	O'rta	Yuqori	Har model qo'lda tekshiriladi; sinov ro'yxati	3D dizayner
T2	iOS USDZ konvertatsiya buziladi	O'rta	Yuqori	3 xil konvertor; iPhone'da avtotest	Frontend
T3	3D fayl juda katta (sekin)	Yuqori	O'rta	Qat'iy 3 MB limit, Draco, LOD, CDN	Backend
T4	WS 1 mln ulanishda yiqiladi	Past	Yuqori	Mijozga WS faqat buyurtmadan keyin (§III.4.2)	Arxitektor
T5	DB yozish bo'g'ilishi	O'rta	Yuqori	Partition, replica, event→ClickHouse	Arxitektor
T6	Cross-tenant ma'lumot sizishi	Past	**Kritik**	Middleware + mixin + CI test majburiy	Barcha
T7	AI provayderlar bir vaqtda yiqilishi	Past	O'rta	8 provayder, circuit breaker, fallback	Backend
T8	To'lov webhook yo'qolishi	O'rta	Yuqori	Idempotency + kunlik reconciliation	Backend
T9	Fiskal OFD API o'zgarishi	O'rta	O'rta	Adapter pattern, versiyalash	Backend

4.2 Biznes risklar

#	Risk	Ehtimol	Ta'sir	Yumshatish
B1	**3D kontent narxi bashoratdan yuqori**	O'rta	**Yuqori**	Kutubxona + AI (80/20); 1-oyda o'lchanadi; yomon bo'lsa AR faqat premium tarifda
B2	Restoranlar to'lamaydi	O'rta	Yuqori	FREE tarif; "delivery 0%" argumenti; pilotda ROI isbotlanadi
B3	iiko/Zoomda AR qo'shadi	O'rta	O'rta	Kutubxona 2-3 yillik ustunlik; tez harakat
B4	Xodim qarshiligi (ofitsiant)	O'rta	Yuqori	⚡ **Choychaqa** — ular foyda ko'radi; 15 daq o'qitish
B5	Churn yuqori (wow tugaydi)	O'rta	Yuqori	Fiskal+sodiqlik+analitika "yopishqoq"; AR analitika ROI ko'rsatadi
B6	Yandex/Uzum bosim o'tkazadi	Past	O'rta	Biz agregator emasmiz — huquqiy jihatdan boshqa segment
B7	Pilot restoran topilmaydi	Past	Yuqori	Bepul + 10 taom 3D taklifi; 15 tadan 5 tasi rozi bo'ladi
B8	Asosiy dasturchi ketadi	O'rta	Yuqori	**Bu TZ** — hujjatlashtirilgan; kod standarti; option
B9	Valyuta/inflyatsiya	Yuqori	Past	Narx so'mda, yiliga qayta ko'riladi

4.3 Huquqiy risklar

#	Risk	Yumshatish
H1	Shaxsiy ma'lumot qonuni buzilishi	Ma'lumot O'zbekistonda; minimal yig'ish; rozilik; DPA hujjat
H2	Fiskal talabga mos emaslik	v1.5 da OFD; yuridik maslahat; Soliq bilan maslahat
H3	To'lov litsenziyasi	Biz to'lov qabul qilmaymiz — provayder qiladi (agent modeli)
H4	Kontent mualliflik huquqi	3D modellar bizniki; restoran rasmlari — shartnomada ruxsat
H5	Chegaralararo ma'lumot (24-oy)	Har davlatda lokal server yoki adekvatlik

4.4 Risk matritsasi



Ustuvor 5 ta risk (birinchi navbatda hal qilinadi):
1. 🟠 **T6** — cross-tenant sizish (CI test majburiy)
2. 🟠 **B1** — 3D kontent narxi (1-oyda o'lchanadi)
3. 🟠 **B4** — xodim qarshiligi (choychaqa v1.5 da SHART)
4. 🟡 **T1/T2** — AR o'lcham/USDZ (har model tekshiriladi)
5. 🟡 **B5** — churn (AR analitika ROI ko'rsatadi)

---

5. ✅ SIFAT NAZORATI

5.1 Test qamrovi (majburiy minimum)

Modul	Qamrov	Kritik testlar
`core`	85%	middleware, mixin, resilience
`tenants`	90%	⚡ izolyatsiya
`orders`	90%	⚡ holat mashinasi, idempotency, narx hisobi
`payments`	90%	⚡ webhook, imzo, idempotency
`ar3d`	80%	fayl limiti, o'lcham, fallback
`menu`	80%	kesh, 3 til, filtr
`ai`	75%	fallback zanjiri, byudjet
Umumiy	**≥75%**	CI'da majburiy

5.2 Majburiy testlar (deploy bloklaydi)

```
tests/test_tenant_isolation.py    # ⚡ har model uchun cross-tenant 403
tests/test_i18n_completeness.py   # ⚡ 3 til to'liq
tests/test_order_state.py         # ⚡ noto'g'ri holat o'tishi 409
tests/test_payment_idempotency.py # ⚡ dublikat to'lov yo'q
tests/test_price_calculation.py   # ⚡ narx serverda hisoblanadi
```

5.3 Yuklama testi (har relizdan oldin)

```
// tests/load/menu.js (k6)
export const options = {
  stages: [
    {duration: '2m', target: 500},    // ko'tarilish
    {duration: '5m', target: 2000},   // barqaror
    {duration: '2m', target: 5000},   // pik
    {duration: '3m', target: 0},      // tushish
  ],
  thresholds: {
    http_req_duration: ['p(95)<250'],
    http_req_failed:   ['rate<0.01'],
  },
};
```

5.4 Kod standarti

Vosita	Maqsad
`ruff`	Linting
`black`	Formatlash
`mypy` (asta-sekin)	Tip tekshiruvi
`bandit`	Xavfsizlik skani
`pip-audit`	Zaifliklar
Pre-commit hook	Har commit'da avtomatik

---

6. 🫙 QABUL MEZONLARI (v1 tayyor deb hisoblanishi uchun)

6.1 Funksional

- [ ] Mijoz QR skanerlab menyuni 3 tilda ko'ra oladi
- [ ] ⚡ Mijoz taomni **AR'da real o'lchamda** stolga qo'yib ko'ra oladi (iOS + Android)
- [ ] AR qo'llab-quvvatlamaydigan qurilmada 3D/rasm fallback ishlaydi
- [ ] Mijoz savatga qo'shib buyurtma bera oladi
- [ ] Buyurtma holati real vaqtda yangilanadi
- [ ] ⚡ Ofitsiant **stol raqamini** darhol ko'radi va tovushli bildirishnoma oladi
- [ ] Oshxona KDS monitorida buyurtmalar kanban ko'rinishida
- [ ] Admin barcha buyurtmalarni real vaqtda nazorat qiladi
- [ ] ⚡ Har stol uchun **noyob** QR generatsiya qilinadi va PDF chop etiladi
- [ ] Mijoz ofitsiant/hisob/suv chaqira oladi
- [ ] AI chatbot menyu bo'yicha javob beradi (3 tilda)
- [ ] Admin 3D model yarata oladi (kutubxona/AI/yuklash)
- [ ] AR analitikasi ko'rsatiladi (ko'rish, konversiya, chek ta'siri)

6.2 Nofunksional

- [ ] Menyu sahifasi 3G'da < 2,5 s ochiladi
- [ ] API p95 < 250 ms
- [ ] Lighthouse (mobil) > 90
- [ ] 3D fayl ≤ 3 MB (GLB), ≤ 5 MB (USDZ)
- [ ] Dark + Light tema ikkalasi ham to'liq
- [ ] 3 tilda 100% tarjima (CI tekshiradi)
- [ ] Test qamrovi ≥ 75%
- [ ] ⚡ Tenant izolyatsiya testi 100% o'tadi
- [ ] 1000 parallel foydalanuvchi yuklama testi o'tadi
- [ ] Web node o'chirilganda foydalanuvchi sezmaydi
- [ ] Redis o'chirilganda menyu ishlashda davom etadi
- [ ] Self-healing daemon ishlaydi va log yozadi

6.3 Biznes

- [ ] 5 ta pilot restoran o'rnatilgan va ishlayapti
- [ ] Kamida 100 ta 3D model kutubxonada
- [ ] AR ta'siri o'lchangan (chek farqi %)
- [ ] Sotuv materiallari tayyor (demo, taqdimot, video)
- [ ] Narx va shartnoma tasdiqlangan

---

7. ? RUXSATDAN OLDIN JAVOB KERAK (yakuniy)

#	Savol	Nima uchun muhim	Tavsiyam
1	**Server:** shu server (8060) yoki alohida VPS?	Izolyatsiya kafolati	⚡ **Alohida VPS** — mebelcity'ga 0 xavf
2	**Domen va brend nomi?**	QR kodlarda doimiy qoladi	`armenu.uz` yoki sizning taklif
3	**Pilot restoran bormi?**	Butun strategiya shunga bog'liq	5 ta kerak: 2 TSH + 2 SMQ + 1 BXR
4	**Byudjet (1-bosqich)?**	Jamoa hajmi	~$45 000 (6 oy)
5	**Jamoa: kim yozadi?**	Muddat	2 dasturchi kerak (parallel)
6	**3D kontent:** kim qiladi?	Eng katta xarajat	AI 80% + qo'lda 20%
7	**v1 qamrovi to'g'rimi?** (15 hafta)	Reja	Ha desangiz boshlaymiz
8	**AI API kalitlari bormi?**	MCP ishlashi uchun	Kamida 3 ta (Gemini bepul limit bor)
9	**FREE tarif bo'ladimi?**	Bozorga kirish tezligi	⚡ Ha — viral kanal
10	**Fiskal (OFD) v1.5 da yetarlimi?**	Muddat	Ha — avval AR ishlasin
11	**To'lov:** ilovada yoki kassada?	v1 qamrovi	v1.5 da ilovada
12	**Qo'shni davlatlar:** qaysi biri birinchi?	14-oy rejasi	🇰🇿 Qozog'iston (eng katta)

8. 🎯 YAKUNIY XULOSA

8.1 Loyiha bahosi (5 nuqtai nazardan)

Nuqtai nazar	Baho	Izoh
🏗 **Arxitektor**	8,5/10	Standart, sinovdan o'tgan stack; masshtablanadi; 5-yil chidamli
📋 **PM**	8/10	234 vazifa aniq; 15 hafta realistik; bog'liqliklar aniq
💰 **Investor**	8,5/10	LTV/CAC 40:1; break-even 11 oy; TAM 118k nuqta; exit $3-5 mln
🍽 **Restoran egasi**	9/10	Oyiga 590k to'lab 16-45 mln foyda; risk past
🏛 **Davlat**	9/10	Soliq shaffofligi, turizm sifati, IT-eksport
👤 **Foydalanuvchi**	9/10	Kutish yo'q, xato yo'q, taomni ko'radi
O'RTACHA	**8,7/10**	

8.2 Nima uchun bu loyiha ishlaydi

1. ✅ Bozorda AR/3D qiluvchi HECH KIM YO'Q (O'zb + 5 qo'shni davlat)
2. ✅ Sotib olish sababi AR emas — PUL (delivery 0%) + QONUN (fiskal)
3. ✅ 3D kutubxona = takrorlanmaydigan aktiv (2-3 yillik ustunlik)
4. ✅ Turizm 7,75 mln (+22,6%) — eng kuchli use-case
5. ✅ QR odati shakllangan (to'lov QR tufayli)
6. ✅ Telegram 27 mln — ilova kerak emas
7. ✅ Birlik iqtisodiyoti sog'lom (LTV/CAC 40:1)
8. ✅ Texnologiya tayyor va arzon (model-viewer, Meshy, paytechuz)

8.3 Nima loyihani o'ldirishi mumkin

1. 🟡 3D kontent narxi nazoratdan chiqishi → kutubxona + AI bilan hal qilinadi
2. 🟡 Xodim qarshiligi → choychaqa bilan hal qilinadi
3. 🟡 Cross-tenant ma'lumot sizishi → CI test bilan oldi olinadi
4. 🔴 Churn → fiskal + analitika "yopishqoqligi" bilan
5. 🔴 Sekin sotuv → FREE tarif + viral QR bilan

8.4 Birinchi qadam

> ⛔ **KOD YOZILMAYDI.** Avval §7 dagi 12 ta savolga javob.
>
> Javob kelgach, **V-001** dan boshlab, TZ bo'yicha ketma-ket ishlaymiz.

9. 🖷 HUJJATLAR RO'YXATI

Fayl	Mazmun
`01_KIRISH_VA_BIZNES.md`	Biznes-model, bozor, investor, 5 nuqtai nazar
`02_SAHIFALAR_VA_TUGMALAR.md`	⚡ Har sahifa, har tugma, jarayon zanjirlari
`03_ARXITEKTURA.md`	⚡ Masshtablash, fault-tolerance, self-healing
`04_DATA_MODEL_VA_API.md`	DB sxema, API spetsifikatsiya, WebSocket
`05_KOD_YOZISH_TARTIBI.md`	⚡ 234 vazifa, ketma-ketlik, fayl-fayl
`06_TARJIMA_VA_UI.md`	3 til, dizayn tizimi, PWA
`07_AI_MCP.md`	⚡ 8 provayder, router, fallback
`08_DEPLOY_REJA_RISK.md`	Deploy, reja, risklar, savollar (bu fayl)

Oldingi tahlillar:
`00_BOZOR_TAHLIL.md`	Birinchi bozor tahlili
`02_CHUQUR_BOZOR_TAHLIL_v2.md`	Fiskalizatsiya, delivery, turizm topilmalari
`03_TEXNIK_TZ_v2.md`	3 rol, AR asosiy

Hujjat tugadi.
Versiya 3.0 MASTER · 2026-09-04 · ARMENU loyihasi